DESIGN AND IMPLEMENTATION OF A DIGITAL TWIN ORCHESTRATION PLATFORM FOR INDUSTRIAL IoT MANUFACTURING

A Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

in

Industrial IoT

to

PRIYADARSHINI COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur

Under the Guidance of

[Project Guide Name and Designation]

Submitted by

[Student Name 1]	**[Student Name 2]**
[Student Name 3]	**[Student Name 4]**
[Student Name 5]	**[Add or remove student fields as required]**



DEPARTMENT OF INDUSTRIAL IoT

PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR-440019

2026-2027

INSTITUTION AND DEPARTMENT VISION AND MISSION

VISION OF THE INSTITUTE

To become one of India's leading engineering institutes in both education and research. We are committed to providing quality and state-of-the-art technical education to our students so that they become technologically superior and contribute to creating a great society.

MISSION OF THE INSTITUTE

Fostering a dynamic learning environment that equips students with technical expertise, problem-solving skills, and a deep commitment to ethical practices.

Cultivating a culture of innovation, incubation, research, and entrepreneurship that drives technological advancement.

Upholding the spirit of mutual excellence while interacting with stakeholders of the institutional ecosystem.

Promoting lifelong learning, professional growth, holistic student development, and the well-being of society.

VISION OF THE DEPARTMENT

To develop a centre of excellence that produces graduates with strong technical and industry-oriented knowledge needed to achieve distinction in the discipline.

MISSION OF THE DEPARTMENT

To guide, motivate, and enrich students' knowledge and skills towards development of Industrial IoT.

To support excellence in workforce participation, higher studies, or lifelong learning.

To strengthen links with industry through partnership and collaborative development work.

PROGRAM OUTCOMES

PO1 Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop solutions to complex engineering problems.

PO2 Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems, reaching substantiated conclusions with consideration for sustainable development.

PO3 Design and Development of Solutions: Design creative solutions for complex engineering problems and develop systems, components, or processes to meet identified needs with consideration for public health and safety, whole-life cost, net-zero carbon, culture, society, and environment.

PO4 Conduct Investigations of Complex Problems: Conduct investigations using research-based knowledge, including experiment design, modelling, analysis, and interpretation of data to provide valid conclusions.

PO5 Engineering Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, while recognizing their limitations.

PO6 The Engineer and the World: Analyze societal and environmental aspects while solving engineering problems, including impacts on sustainability, economy, health, safety, law, culture, and environment.

PO7 Ethics: Apply ethical principles, commit to professional ethics and human values, and adhere to applicable laws.

PO8 Individual and Collaborative Teamwork: Function effectively as an individual and as a member or leader in diverse and multidisciplinary teams.

PO9 Communication: Communicate effectively within the engineering community and society through technical reports, documentation, and presentations.

PO10 Project Management and Finance: Apply engineering management principles and economic decision-making to one's work and to projects in multidisciplinary environments.

PO11 Lifelong Learning: Recognize the need and ability for independent lifelong learning, adaptability to emerging technologies, and critical thinking.

PROGRAM EDUCATIONAL OBJECTIVES AND PROGRAM SPECIFIC OUTCOMES

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO 1. To provide graduates with a solid foundation in mathematical, scientific, and engineering fundamentals required to formulate, analyze, and solve complex engineering problems.

PEO 2. To create graduates with technical expertise to handle multidisciplinary projects and excel in professional careers or higher education.

PEO 3. To inculcate leadership qualities among graduates with ethical responsibilities and global security awareness.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1. Develop problem-solving skills through programming techniques using appropriate principles and concepts of the Internet of Things.

PSO 2. Understand IoT application development tools and their implementation for industrial applications.

PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR

(An Autonomous Institute Affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur)

Department of Industrial IoT

CERTIFICATE

This is to certify that the project report entitled “Design and Implementation of a Digital Twin Orchestration Platform for Industrial IoT Manufacturing” is a bonafide work carried out by [Student Name(s)].

The project report is submitted to Priyadarshini College of Engineering, Nagpur, an Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Industrial IoT for Session 2026-2027.

Project Guide: _____

Head of Department: _____

Principal: _____

Student Name(s): _____

DECLARATION

We, the undersigned, declare that the project entitled “Design and Implementation of a Digital Twin Orchestration Platform for Industrial IoT Manufacturing”, submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Industrial IoT at Priyadarshini College of Engineering, Nagpur, is work carried out by us.

We have identified the project sources and references used in this report. The implementation evidence and limitations are represented to the best of our knowledge. Any remaining institutional declaration requirements shall be completed and signed by the project team before submission.

Place: Nagpur

Date: _____

Project team: _____

Student signatures: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, [Project Guide Name], for guidance, technical feedback, and support throughout the design and implementation of this work.

We thank the Department of Industrial IoT and the faculty of Priyadarshini College of Engineering for the academic environment and facilities that supported this project.

We acknowledge the contributions of the project team members and all those who provided useful discussions, review, and encouragement. Their input helped refine the system architecture, implementation priorities, and presentation of the work.

We also thank our families and friends for their patience and support during the project. Names and additional acknowledgements will be completed by the project team before submission.

ABSTRACT

This project presents a simulator-backed Digital Twin Orchestration Platform for a discrete five-station automotive-component manufacturing line. The platform combines deterministic reduced-order models for Stamping, CNC machining, Welding, Inspection, and Packaging with a versioned event contract, a Live Twin state reducer, a process-level directed acyclic graph, production-token lineage, isolated scenario simulation, a FastAPI service, and a React dashboard. The design separates the model that generates observations from the service that materializes current twin state. Events carry stable identity, machine sequence, schema version, and timezone-aware timestamps. The reducer validates the payload, records observed receipt time, prevents duplicate writes from changing state, and preserves current state when a late sequence arrives. At the line level, the simulator links adjacent stations, enforces upstream starvation and a maximum downstream buffer of five units, and maintains the output conservation relation across the sequential route. Production tokens carry selected quality and process flags so that an inspection outcome can be traced to upstream model signals. The Scenario Worker captures a frozen line snapshot and runs baseline and counterfactual factories using the same seed. It computes comparative output, quality, energy, health, scrap, bottleneck, and trajectory fields and checks that the Live Twin state remains unchanged. The web application exposes machine, process, event, scenario, and system views. Docker Compose and Kubernetes resources describe a multi-service deployment. The project repository records a final backend verification summary of 48 tests with two skipped and a successful frontend production build. These results support software-behavior claims only. The station coefficients and cycle times are model configuration; physical calibration, real factory connectivity, measured latency benchmarks, and production improvement have not been established. The work provides a traceable prototype and an implementation basis for future hardware integration, security hardening, calibration, and empirical evaluation.

Keywords: Industrial IoT, digital twin, manufacturing, MQTT, event processing, causal simulation, production flow, counterfactual analysis

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 16
1.1 Background 16
1.2 Industrial problem context 17
1.3 Problem statement 17
1.4 Aim and objectives 18
1.5 Research questions 18
1.6 Scope 19
1.7 Significance 19
1.8 Assumptions and limitations 19
1.9 Contributions 20
1.10 Organization of the thesis 20
CHAPTER 2 LITERATURE REVIEW 21
2.1 Digital twin concepts in manufacturing 21
2.2 Standardized manufacturing architecture 21
2.3 Digital twin and simulation relationship 21
2.4 Industrial communication and event transport 22
2.5 Versioned data contracts and event identity 22
2.6 State reduction, ordering, and idempotency 23
2.7 Causal and physics-informed modeling 23
2.8 Process-level digital twins and material flow 23
2.9 Bottleneck reasoning 24
2.10 Counterfactual simulation and immutability 24
2.11 Dashboards and decision support 24
2.12 Container deployment and operational security 25
2.13 Literature synthesis and research gap 25
CHAPTER 3 METHODOLOGY 27
3.1 Engineering approach 27

3.2 Requirements elicitation and traceability 27
3.3 System boundary and actors 27
3.4 Station model and deterministic execution 28
3.5 Telemetry envelope and validation 29
3.6 Live Twin reducer method 29
3.7 Material-flow method 30
3.8 Snapshot and scenario method 31
3.9 API and user-interface method 32
3.10 Verification and evaluation design 32
3.11 Research ethics and operational constraints 33
3.12 Reproducibility record 33
CHAPTER 4 SOFTWARE AND HARDWARE REQUIREMENTS AND DESIGN 34
4.1 Functional requirements 34
4.2 Non-functional requirements 34
4.3 Software and hardware environment 34
4.4 High-level architecture 35
4.5 Station configuration and nominal capacity 35
4.6 Causal station models 36
4.7 Material constraints and buffer semantics 37
4.8 Event and twin data model 37
4.9 Live Twin state transitions 38
4.10 Process Twin, topology, and snapshots 38
4.11 Production token and defect trace 38
4.12 Bottleneck calculation 39
4.13 Scenario Worker and presets 39
4.14 REST API design 40
4.15 Frontend design 40
4.16 Deployment architecture 41
4.17 User workflows 42

4.18 Design trade-offs 42
CHAPTER 5 RESULTS AND DISCUSSION 44
5.1 Implementation status 44
5.2 Functional outcomes 44
5.3 Model configuration results 44
5.4 Verification results in the repository log 45
5.5 Test behavior coverage 46
5.6 Determinism and state consistency 46
5.7 Material-flow behavior 47
5.8 Scenario outcomes and interpretation 47
5.9 Dashboard and operational observability 47
5.10 Deployment results and boundary 48
5.11 Limitations of evaluation 48
5.12 Discussion 48
CHAPTER 6 CONCLUSION AND FUTURE SCOPE 49
6.1 Conclusion 49
6.2 Objective attainment 49
6.3 Limitations 49
6.4 Future work: physical and semantic integration 50
6.5 Future work: empirical system evaluation 50
6.6 Future work: security and reliability 50
6.7 Future work: process and user features 51
6.8 Final statement 51
REFERENCES 52
APPENDIX A REQUIREMENTS AND TRACEABILITY 53
A.1 Functional requirement register 53
A.2 Non-functional requirement register 53
A.3 Acceptance interpretation 54
APPENDIX B DATA DICTIONARY AND TOPIC REFERENCE 55

B.1 Core event envelope 55
B.2 Telemetry signal dictionary 55
B.3 Documented MQTT topic taxonomy 55
APPENDIX C API REFERENCE 56
C.1 Health and compatibility routes 56
C.2 Versioned twin and event routes 56
C.3 Process and token routes 56
C.4 Scenario routes 56
APPENDIX D SCENARIO AND MODEL REFERENCE 58
D.1 Available scenario presets 58
D.2 Station model variables 58
APPENDIX E DEPLOYMENT AND OPERATOR GUIDE 59
E.1 Local development mode 59
E.2 Docker Compose topology 59
E.3 Kubernetes topology 59
E.4 Operator interpretation checklist 59
E.5 Configuration and security checklist for future integration 60
APPENDIX F VERIFICATION CATALOGUE 61
F.1 Test names in the source suite 61
F.2 Verification summary from dated project record 61
F.3 Suggested field evaluation protocol 61
APPENDIX G COURSE OUTCOME MAPPING 63
G.1 Course outcomes 63
G.2 CO to PO and PSO matrix 63
G.3 Outcome contribution 63
APPENDIX H EDITABLE SUBMISSION DETAILS 64
H.1 Student and guide details 64
H.2 Project photographs 64
H.3 Guide photograph 64

H.4 Certificate and publication evidence 64
H.5 Plagiarism report 64
H.6 Poster 64
APPENDIX I MEASUREMENT RECORD TEMPLATES 65
I.1 Event timing log 65
I.2 Model calibration record 65
I.3 Scenario experiment record 65
I.4 Deployment observation record 65
I.5 Evidence discipline 65
APPENDIX J SELECTED SOURCE LISTINGS 66

LIST OF FIGURES

No table of figures entries found.

ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
CPS	Cyber-Physical System
DAG	Directed Acyclic Graph
IIoT	Industrial Internet of Things
KPI	Key Performance Indicator
MQTT	Message Queuing Telemetry Transport
OEM	Original Equipment Manufacturer
OPC UA	OPC Unified Architecture
REST	Representational State Transfer
WIP	Work in Process

CHAPTER 1 INTRODUCTION

1.1 Background

Manufacturing lines are cyber-physical systems: machines transform material while sensors, controllers, operators, and enterprise applications record and influence that work. A digital twin gives a defined manufacturing element a digital representation that is synchronized with information about the element. In manufacturing, the element may be a machine, product, process, facility, or another observable entity. The representation is useful only when its scope, data, models, and intended decisions are made explicit. The NIST and ISO literature therefore treats a twin as a fit-for-purpose engineering system rather than a three-dimensional picture or a generic dashboard [1], [2].

Small production cells provide a practical setting for studying the design questions behind a twin. Even a short line can contain several data and coordination problems: machine telemetry has different meanings at each station; repeated messages can arrive after a retry; a downstream process cannot work without upstream material; a buffer can fill and stop an upstream station; a defect detected at inspection may have originated earlier; and an operator may need to compare an intervention with the current operating plan. These concerns connect communication, state management, physical modeling, production flow, quality, and decision support.

This project develops a software prototype for a discrete five-station automotive-component manufacturing line. The line contains Stamping, CNC machining, Welding, Inspection, and Packaging. Its command center combines a seeded reduced-order causal simulator, a message event pathway, persistent Live Twin records, a process-level graph and token history, isolated what-if scenarios, versioned APIs, and a React dashboard. The models are explicit enough to demonstrate causal relationships and flow constraints, while remaining small enough to run as a local development application.

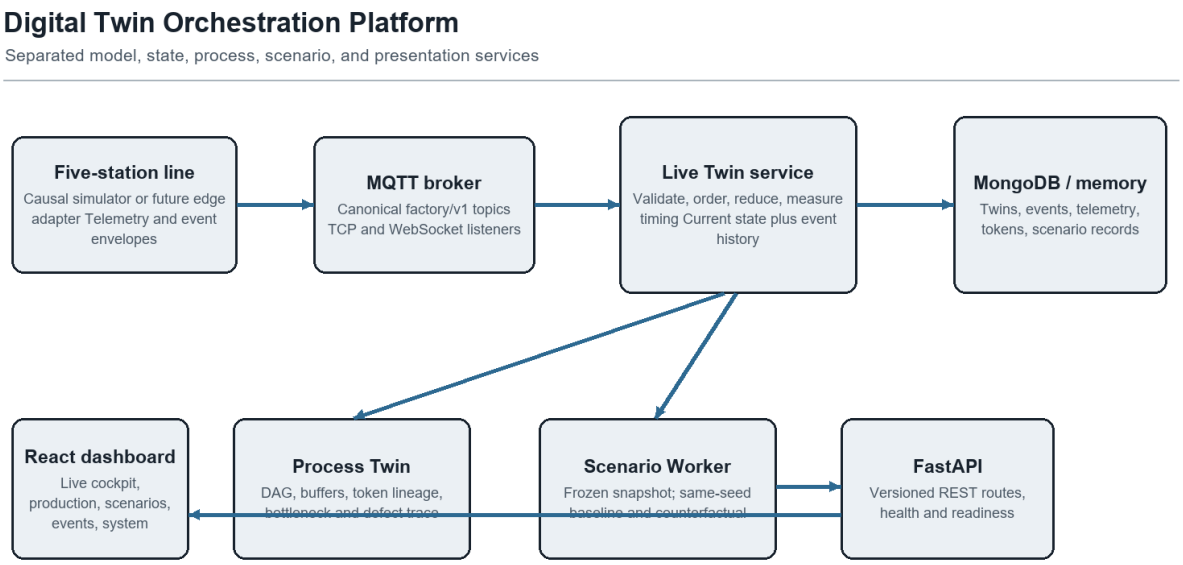


Figure 1.1: Reference architecture of the Digital Twin Orchestration Platform

1.2 Industrial problem context

Machine-level telemetry does not by itself provide a line-level explanation. A vibration warning at a machining center may matter because it changes quality, effective cycle time, or the timing of work reaching Welding. A fault at one station can cause another to starve, while a blocked upstream buffer may prevent further production. A dashboard that shows five independent cards misses those dependencies. A line twin must therefore preserve machine state while also representing process order, buffers, parts, inspection outcomes, and relationships among stations.

The prototype addresses this gap by separating physical observations from the services that consume them. The simulator emits timestamped telemetry and event envelopes. The Live Twin reducer validates the envelope, records the actual receive time, rejects invalid state transitions, tracks event ordering, and persists the latest state. The Process Twin composes station twins into a directed acyclic graph and carries production tokens through that order. Scenario runs fork from a frozen snapshot and evaluate baseline and overridden models independently. This separation enables a user to inspect the current simulated line, review traceable parts, and compare a proposed scenario without rewriting the live twin.

1.3 Problem statement

The engineering problem is to design and implement a maintainable Industrial IoT command center that represents a multi-station manufacturing line at both the machine and process levels. The system must accept versioned telemetry, preserve a consistent current state when events are duplicated or late, make material-flow constraints visible, support traceable discrete production tokens, and permit

reproducible counterfactual simulation from a captured state. It must also present these functions through a usable web interface and be deployable with common container orchestration tools.

The project does not claim a production installation or a validated physical plant model. The repository implements a simulator-backed prototype, with repository adapters and deployment manifests intended to support development and integration. It provides a concrete software foundation and testable invariants, but does not provide sensor calibration, shop-floor commissioning, or measured factory performance data.

1.4 Aim and objectives

The aim is to design and implement a digital twin orchestration platform for an Industrial IoT manufacturing line that joins telemetry, causal simulation, process-flow state, scenario comparison, and operational visualization.

The specific objectives are to define stable, versioned contracts for event and twin data; implement deterministic, station-specific reduced-order causal models; separate simulator output from Live Twin state reduction; measure event timing from observed timestamps; implement idempotent and monotonic event processing; represent production order through a directed acyclic graph; enforce upstream starvation and downstream buffer blocking; preserve part lineage and quality flags through a production token; run baseline and counterfactual scenarios from the same frozen snapshot; expose the platform through versioned REST interfaces; provide live status and analysis in the dashboard; and package the application for local, Docker Compose, and Kubernetes deployment.

1.5 Research questions

The project is guided by four design questions. First, how can a small manufacturing twin combine machine-specific causal models with one stable telemetry contract? Second, how can a state reducer retain audit history while preventing duplicate or late events from rolling current state backward? Third, how can line-level material constraints and part lineage be added without allowing a scenario simulation to mutate the live twin? Fourth, how can implementation claims be evaluated transparently when a repository contains simulator tests but no calibrated physical-line data or recorded performance benchmark?

The questions are architectural and engineering questions. They do not ask whether the prototype increases production or predicts physical failures in a real factory. Those claims require real equipment, calibrated models, a controlled evaluation, and evidence outside the current repository. The evaluation in this thesis consequently focuses on implemented behavior, contract validation, deterministic replay, flow invariants, scenario isolation, build evidence, and the boundary between observed and unevaluated performance.

1.6 Scope

The system models five station types and a sequential material path: STAMPING-01, CNC-01, WELDING-01, INSPECTION-01, and PACKAGING-01. The backend is written in Python and exposes FastAPI routes. Pydantic contracts define station events and twin records. The simulator includes station-specific degradation and telemetry. MQTT provides an asynchronous transport path, MongoDB is the durable repository option, and a thread-safe in-memory repository supports local development. The frontend is a React and TypeScript application with live polling, process views, telemetry, scenario controls, event records, and system status.

Scope includes scenario presets, deterministic seeds, snapshot-based execution, basic fault and maintenance commands, throughput and bottleneck summaries, quality-oriented token flags, container images, Docker Compose, and Kubernetes manifests. The models derive electrical energy from modeled current and a fixed assumed supply, and derive machine responses from reduced-order equations and seeded noise. The project supports a simulator-backed dashboard and an API path for adapter events; it does not include a certified device driver or a commissioned physical machine.

1.7 Significance

The engineering value of the prototype is in its explicit boundaries. A simulator does not directly overwrite a twin; the reducer owns the current state. A scenario does not run against mutable live objects; it forks from an immutable snapshot. A production line is not treated as five unrelated counters; it includes ordered material movement, queue limits, starvation, blocking, and tokens. Latency is not filled with randomized values; timing values are calculated when corresponding timestamps are available. These decisions improve traceability and make later validation easier.

The same boundaries are useful in teaching and in early-stage system integration. They give a team separable work packages: device and broker integration, data contracts, persistence, model calibration, operational analytics, front-end interaction, and deployment. Because the simulator and service interfaces are decoupled, an industrial adapter can be introduced later while retaining the same reducer and dashboard contracts, subject to field mapping and safety review.

1.8 Assumptions and limitations

The five stations and their nominal cycle times are illustrative model configuration. The simulator's mathematical coefficients are calibration-ready prototype parameters. They are not claimed to match a particular plant or machine. The line is modeled as a single sequential path with a maximum inter-station buffer of five units. The code's base model uses seeded pseudo-random variation for repeatability, not a probability distribution fitted from field data. Model-state labels and fault responses represent software behavior rather than a safety-rated machine control system.

The repository records a verification log, but the thesis does not equate unit tests with physical fidelity. No measured end-to-end latency percentiles, concurrency limits, uptime figures, energy savings, or prediction accuracy figures are available in the examined project material. The optional AI reasoning integration is not required for the core twin and is gated when its provider is unavailable. The current Mosquitto example configuration permits anonymous connections and must be hardened before connection to a real production network.

1.9 Contributions

The project contributes a reusable event envelope and strict data contracts; five station-specific deterministic models; an event reducer that handles event identity, sequence order, receive time, and idempotent persistence; a process twin that captures snapshots, analyzes bottlenecks, traces token flags, and enforces the sequential-line constraint; a scenario worker that compares baseline and counterfactual outcomes while checking live-state immutability; a versioned API surface; a dashboard that exposes the machine and process views; and a deployment package for a multi-service environment.

These contributions are concrete software artifacts. Their significance is bounded by the evidence available: repository records and its dated verification log. Physical model calibration, cybersecurity evaluation, load testing, operations research, and field usability testing remain further work.

1.10 Organization of the thesis

Chapter 1 establishes the project context, problem, objectives, scope, and limitations. Chapter 2 reviews manufacturing digital twins, industrial communication, causal simulation, event reduction, process flow, and counterfactual evaluation. Chapter 3 describes the engineering methodology and verification design. Chapter 4 presents requirements, architecture, data contracts, station models, services, user interface, and deployment. Chapter 5 reports the implementation evidence and discusses what can and cannot be concluded from it. Chapter 6 summarizes outcomes and recommends a staged path toward hardware integration and measured validation. The appendices provide traceability, API and data references, operational guidance, evidence templates, and selected source listings.

CHAPTER 2 LITERATURE REVIEW

2.1 Digital twin concepts in manufacturing

Digital twin terminology is used across engineering domains, and the literature has not always applied one uniform definition. Reviews describe a progression from virtual product and lifecycle models toward operational representations synchronized with data from a physical counterpart [3], [4]. For a manufacturing use case, NIST presents a fit-for-purpose digital representation of an observable manufacturing element with synchronization between that element and its representation [1]. This definition stresses purpose and synchronization; it does not prescribe one modeling technique or one communications protocol.

This distinction matters for the present project. A static 3D model is not sufficient to support event ordering or trace material lineage. A database record without a relationship to a modeled station is also incomplete as a process twin. A useful prototype has to identify what it represents, what events update the representation, what computations are performed, what decisions are supported, and what evidence confirms the intended behavior. The platform treats a station twin as the latest validated operational state plus telemetry, health, production, fault, desired-state, synchronization, and identity information.

2.2 Standardized manufacturing architecture

ISO 23247 provides a reference framework for digital twins in manufacturing. Its parts address general principles and requirements, a reference architecture, information attributes for observable manufacturing elements, and technical information exchange requirements [2]. NIST work on applying the framework describes relevant use cases including analytics, health monitoring, scheduling, virtual commissioning, and predictive maintenance [7]. These sources support an architecture organized around a manufacturing element, a digital representation, data acquisition, and user or application functions.

The project adopts these ideas as architectural guidance rather than claiming conformance certification. It models five machine elements and exposes functional views that resemble common manufacturing twin concerns: telemetry and health, process sequence, token lineage, bottleneck analysis, and scenario evaluation. The platform does not implement all reference architecture entities, nor does it provide a formal conformance test. Its API contract and database shape are project-specific and would need mapping to a standards-based model before interoperability claims could be made.

2.3 Digital twin and simulation relationship

Simulation is one possible capability inside a digital twin, but simulation alone does not provide synchronization, identity, history, or operational context. A model may run as an isolated experiment, while the live representation must remain tied to a stream of observations. Literature on digital twins emphasizes integration among physical entities, models, data, and intended use, and recognizes simulation as one of several analytical capabilities [1], [4].

In this project, the causal factory model and the Live Twin reducer are separate modules. The simulator advances station state and emits telemetry envelopes. The reducer consumes versioned events and materializes a current twin. This design allows the same reducer to receive MQTT events or controlled HTTP ingestion, while the model can be replayed under seeded conditions. It also makes an important limitation visible: a deterministic replay verifies repeatable software output, not accuracy against an industrial machine.

2.4 Industrial communication and event transport

MQTT is a client-server publish/subscribe transport designed for lightweight, decoupled messaging and constrained environments [5]. A publisher sends a message to a topic and subscribers select topics of interest. This pattern allows the simulator, broker, Live Twin consumer, and browser-facing systems to evolve independently. The platform uses a canonical factory topic namespace under factory/v1 and includes separate telemetry, state, fault, production, command, and summary categories in its documented taxonomy.

The MQTT consumer currently subscribes to canonical telemetry topics and forwards valid event envelopes to the same Live Twin service used by the internal ingestion route. The project configuration exposes conventional TCP and WebSocket listeners. The reference Mosquitto configuration uses anonymous access, which is acceptable only in an isolated demonstration environment. Production use requires authenticated identities, authorization by topic, encrypted transport, network segmentation, secrets management, and review of broker persistence and availability.

OPC Unified Architecture is a platform-independent industrial information and communication framework covering information, message, communication, and conformance models [6]. It is relevant as a future device integration option, but it is not the implemented telemetry protocol in this repository. The architecture currently uses an abstract adapter-capable event contract and an MQTT consumer. A real OPC UA adapter would need to map node identities, units, timestamps, quality flags, security identities, and machine-specific semantics into the defined event model.

2.5 Versioned data contracts and event identity

Data contracts provide a stable boundary between producers and consumers. The project defines Pydantic models with strict handling of extra fields, typed enumerations, numeric bounds, and aware timestamps. The event envelope includes a schema version, event identifier, run identifier, machine identifier, kind, per-run sequence, generation timestamp, optional receive timestamp, correlation identifier, and a payload. The telemetry payload covers shared measurements and optional station-specific values.

Stable identity is central to safe event processing. A deterministic event identifier is derived from run, machine, event kind, and sequence. This supports replay and duplicate detection. The contract

separates generated time from received time: the simulator cannot claim an observed receive timestamp. The Live Twin assigns receive time at consumption, which permits delivery and source-age calculations from clocks that actually exist at the service boundary. Contract validation cannot prove sensor calibration or clock synchronization; it only checks structure and declared value constraints.

2.6 State reduction, ordering, and idempotency

Distributed message delivery can involve retries, duplicate delivery, delay, and out-of-order events. A reducer turns accepted input events into a current operational view while retaining an event history. The platform stores monotonic sequence state for each machine and uses an event identity barrier in the repository. A repeated event is treated as a no-op. A late event can remain in event history without replacing a newer current state. These behaviors separate auditability from current-state materialization.

The design uses a repository abstraction so that in-memory development storage and MongoDB persistence present the same service-facing operations. The MongoDB implementation creates indexes and uses the event identifier as a unique insertion barrier. It stores current twin state separately from events and telemetry. This structure allows recent state to be queried without replaying all events, while preserving an event record for trace and troubleshooting use cases.

2.7 Causal and physics-informed modeling

Data-driven models learn associations from historical observations; physics-based models encode relationships from a system's mechanics; reduced-order models retain selected causal structure while avoiding the cost of a high-fidelity simulation. The current code implements deterministic reduced-order models with separate station classes. It includes degradation variables, operating-state effects, bounded metrics, and seeded noise. A nominal model may be repeatable and causally legible while still requiring calibration before being used to infer real equipment behavior.

For example, the CNC model separates tool wear, bearing degradation, coolant failure, cutting force, temperature, current, vibration, target RPM, actual RPM, quality, and health. A tool-change operation resets tool wear but need not erase bearing degradation. A coolant failure contributes a thermal penalty. Such separations are preferable to one unconstrained random health value because they support directional tests and interpretable counterfactuals. Coefficients in these equations are model parameters, not externally validated machine constants.

2.8 Process-level digital twins and material flow

A production system twin must represent relationships among workstations and parts. The project represents the five-station route as a directed acyclic graph and records order explicitly. Each station has an upstream and downstream relationship. The simulator prevents a station from completing work when upstream production has not supplied a part and prevents an upstream station from creating work

beyond its downstream buffer limit. This is a software representation of starvation and blocking, not a full discrete-event scheduling system.

The additional production token represents one part moving through the process. It accumulates station identifiers and selected quality or process flags. Inspection determines whether the part is accepted for packaging or rejected. The token lineage makes a downstream quality outcome inspectable against upstream signals. The token model is deliberately lightweight; it does not represent a full bill of materials, operation routing with alternative paths, rework, lot genealogy, labor, or a validated quality control plan.

2.9 Bottleneck reasoning

Throughput in a serial production line is constrained by station capacity and state. The platform estimates a binding station from reported cycle time adjusted by an operational-state penalty. It calculates slack relative to the maximum effective cycle time and derives a nominal per-hour capacity from the binding time. For the configured nominal cycle times, CNC-01 has the largest cycle time at 42 seconds. In a purely nominal calculation, this is approximately 85.7 cycles per hour before availability, yield, setup, queueing, or shift effects.

That calculation is a configured capacity proxy, not a measured production rate. In practice, bottleneck location can change when a station is degraded, faulted, idling, or running under an override. The scenario worker recomputes the binding station for both baseline and counterfactual outcomes. Such comparison can expose a bottleneck shift inside the model, but the result should not be described as a factory improvement until the model parameters and operating conditions are validated.

2.10 Counterfactual simulation and immutability

What-if analysis compares a baseline model with a modified model under shared initial conditions. If initial state or random seed differs, the output difference may reflect uncontrolled variation rather than the intervention. The Scenario Worker resolves a frozen line snapshot, creates separate baseline and counterfactual factories using the same seed, applies overrides only to the latter, advances both for a defined horizon, computes deltas, and checks a fingerprint of Live Twin state before and after.

This design provides a software-level guard against scenario leakage. It also enables repeatable experiments for tool replacement, coolant failure, cadence change, welding electrode wear, line outage, and alternative CNC capacity assumptions. A scenario response remains conditional on the current model and its assumptions. It can support operator discussion or research planning but does not directly actuate production equipment or provide a safety authorization.

2.11 Dashboards and decision support

Dashboards are operational interfaces that turn data and model state into views suited to different decisions. The frontend separates fleet overview, 3D factory view, production flow, machine detail, twin

studio, maintenance, analytics, events, and system health. The Twin Studio includes the live twin cockpit, scenario studio, and process graph and lineage views. The production page surfaces queue counts, starvation, blocking, token status, and upstream defect attribution.

The UI should preserve distinctions between measured values, model estimates, and simulation results. The current app signals backend readiness and MQTT status, while API requests are live-first and do not silently switch to mock data by default. Any interface values captured during a simulator session should be labeled as simulated observations. Usability tests with operators and accessibility checks are not recorded in the repository, so this thesis describes implemented interaction patterns without claiming validated usability.

2.12 Container deployment and operational security

The repository includes a multi-service Docker Compose stack with MQTT, MongoDB, FastAPI backend, and React frontend, along with Kubernetes manifests for a namespace, configuration, database, broker, backend, frontend, and ingress. Health and readiness probes distinguish process liveness from application readiness. The architecture is portable at the service and environment-variable level, but availability, scaling, backup, secrets, and security policies still need environment-specific configuration.

Security is especially important where a twin can influence operator actions or expose process data. The current prototype does not document user authentication, fine-grained authorization, TLS configuration, a threat model, or a security test. MQTT anonymous access is enabled in the provided broker file. These facts limit safe deployment to an isolated demonstration setup until authentication, network controls, auditing, secrets, and a production operations model are designed and reviewed.

2.13 Literature synthesis and research gap

The literature and standards point to a consistent set of design concerns: fit the twin to its purpose; represent manufacturing elements and information exchange explicitly; keep model behavior connected to operational data; preserve the semantics of events; make process relationships visible; and use evaluation evidence that corresponds to the claim being made [1], [2], [3], [5], [6], [7]. The existing project implements a subset of those concerns in a compact, testable stack.

The gap addressed here is not a lack of another generic dashboard. It is the need for a demonstrable integration pattern that joins station-level telemetry, causal model state, line-level material constraints, event-order protection, token lineage, and isolated scenario comparison. The prototype's contribution is architectural cohesion and software verification. Its unresolved research gap is the empirical bridge from model behavior to a real line: parameter estimation, clock quality, sensor calibration, throughput and latency measurement, predictive validity, human factors, and cybersecurity.

Table 2.1 Comparison of relevant approaches

Approach	Main strength	Limitation for this project	Design response

Approach	Main strength	Limitation for this project	Design response
Manufacturing framework [1], [2]	Shared vocabulary and composable architecture	Framework is not a complete application implementation	Implement focused station and line services
Publish/subscribe transport [5]	Producer-consumer decoupling	Transport delivery alone does not define current state	Add versioned envelope and reducer
OPC UA information exchange [6]	Industrial information and interoperability model	Not currently used as a device protocol	Keep adapter boundary and document future mapping
Static dashboard	Low effort and clear visual summary	No event history, material lineage, or isolated scenarios	Integrate event, process, and scenario services
Standalone simulation	Repeatable model exploration	Does not maintain a live operational view	Fork snapshots and isolate scenario state

CHAPTER 3 METHODOLOGY

3.1 Engineering approach

This work follows an artifact-centered engineering method. The target artifact is a simulator-backed command center for a bounded manufacturing line. Requirements are derived from the repository's project definition and the intended Industrial IoT use case. The implementation is divided into domain contracts, physical model behavior, live-state reduction, process composition, scenario execution, user interface, persistence, and deployment. Each subsystem exposes contracts that can be exercised independently.

The method combines system decomposition with iterative verification. A change is treated as complete only when the relevant behavior is implemented and the repository's recorded checks support the claim. The delivery log shows staged development from contracts and causal simulation through persistence, process flow, scenario work, UI, containers, and material-flow corrections. The thesis uses this chronology as project history; it does not infer measured performance from the sequence of implementation phases.

3.2 Requirements elicitation and traceability

Functional requirements were derived from the project objective and organized around observable behaviors. Examples include validating telemetry, preserving event identity, refusing an invalid machine-payload pairing, listing the five twins, analyzing process bottlenecks, tracing defects, and running scenarios. Non-functional requirements include deterministic replay, state isolation, bounded memory, deployability, clear health checks, and a single source of truth for interface contracts.

Traceability links each requirement to at least one module and one verification method. Unit tests are appropriate for deterministic model direction, event order, token progression, and scenario immutability. API tests verify route contracts and status behavior. Frontend compilation verifies TypeScript and bundler compatibility. Container manifests establish deployability intent but do not prove deployment success in a target cluster. Field calibration, throughput scaling, latency percentiles, and recovery under hardware or network faults require experiments not represented by the current log.

3.3 System boundary and actors

The system boundary begins at a simulator or adapter that creates station observations and ends at dashboard users or a client consuming the API. In the current build, the simulator is the concrete telemetry source. A future edge adapter can publish the same envelope to MQTT. The broker distributes events to a Live Twin consumer; repositories store current twins and history; the Process Twin composes station state; the Scenario Worker runs isolated model forks; FastAPI exposes functions; and the browser renders operational views.

Primary actors are an operator who inspects the line and issues supported simulation commands, an engineer who reviews telemetry and scenario deltas, and a developer or administrator who configures storage and deployment. The current prototype has no identity and access management model, so these are logical user roles rather than authenticated accounts. The APIs are not safety PLC interfaces and do not replace interlocks, safe operating procedures, or machine protection.

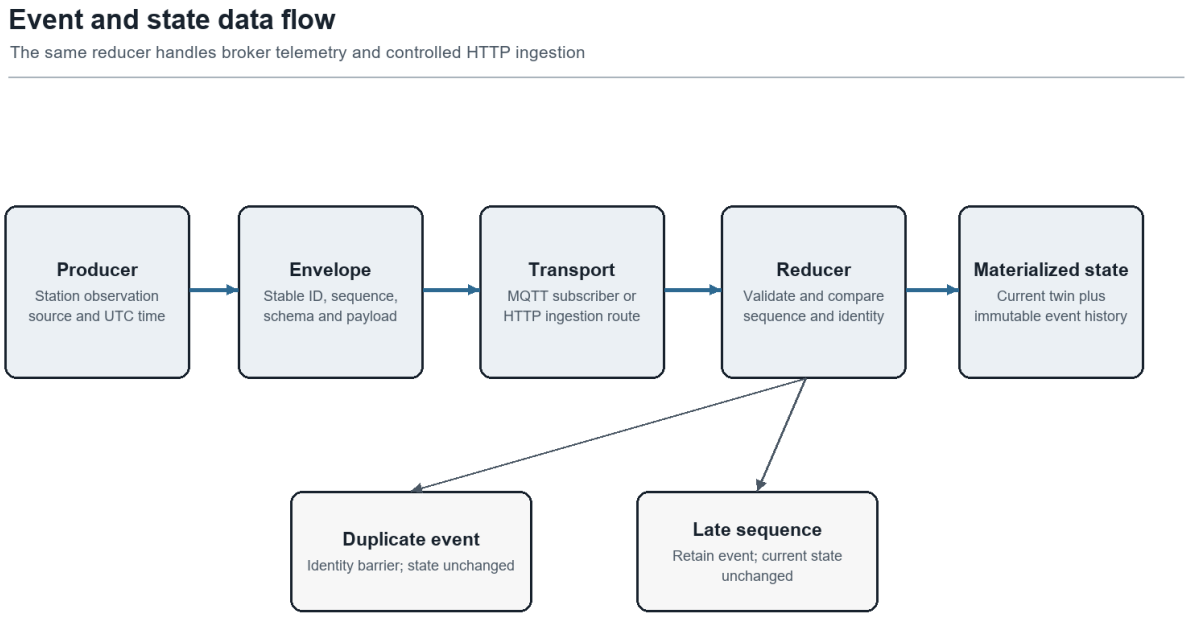


Figure 3.1: Telemetry event and state reduction data flow

3.4 Station model and deterministic execution

Each station is represented by a station configuration, an operational state, degradation variables, production counters, energy accumulation, cycle progress, and a seeded random generator. The factory config establishes the nominal sequence and cycle time. The configured values are 8.5 seconds for Stamping, 42 seconds for CNC, 15 seconds for Welding, 6 seconds for Inspection, and 10 seconds for Packaging. These values provide the baseline for the software demonstration.

Station subclasses map different mechanisms into telemetry. Stamping uses die wear and fault level to influence force, vibration, current, temperature, cycle time, quality, and health. CNC distinguishes tool and bearing degradation and models coolant failure. Welding models electrode and clamp wear with voltage, current, temperature, and quality response. Inspection represents quality review; Packaging has conveyor wear and throughput-related telemetry. The use of separate state variables makes model direction inspectable and enables focused tests.

The simulator uses independent seeded random streams for station models. A fixed seed and equal initial state produce repeatable generated variation, allowing regression comparisons. Seeded

randomness is not a substitute for statistical characterization. The implementation does not assert that a particular seed or noise distribution represents natural plant variation.

3.5 Telemetry envelope and validation

Every canonical event carries a schema version, deterministic event ID, run ID, machine ID, kind, positive sequence, generation time, optional receive time, optional correlation ID, and a JSON payload. The telemetry payload contains a station type, operational state, status, simulated time, temperature, vibration, current, optional voltage, force, pressures and RPM, cycle time, energy, output, WIP, health, anomaly, quality, wear, fault code, and source.

Pydantic validation constrains values and rejects unknown fields at strict model boundaries. Timestamps must include timezone information and are normalized to UTC. Optional fields are omitted when a station does not expose that measurement; the contract avoids manufacturing nonexistent signals. The event identity function uses a namespaced UUID derived from run, machine, event kind, and sequence, so a deterministic run can reconstruct stable identifiers.

3.6 Live Twin reducer method

The reducer accepts telemetry envelopes only. It records the consumption time, validates the payload, confirms that the outer machine identifier matches the payload, and compares the sequence with the current twin. If the sequence is newer, it constructs a replacement twin record with identity, operational state, telemetry, health, production, fault, desired-state, synchronization, revision, and event metadata. If the event is out of order, it retains the event while preserving the existing current state. The repository's event identity barrier prevents duplicate writes from changing state.

Delivery latency is the nonnegative difference between receive time and generated time. Processing latency is measured around the reducer's validation and reduction work using a monotonic performance counter. Source age is based on the same generation-to-receive time gap in the current record. These fields are meaningful only when clocks and timestamps are meaningful; the repo does not publish a benchmark distribution. Figure 3.2 summarizes the update decision.

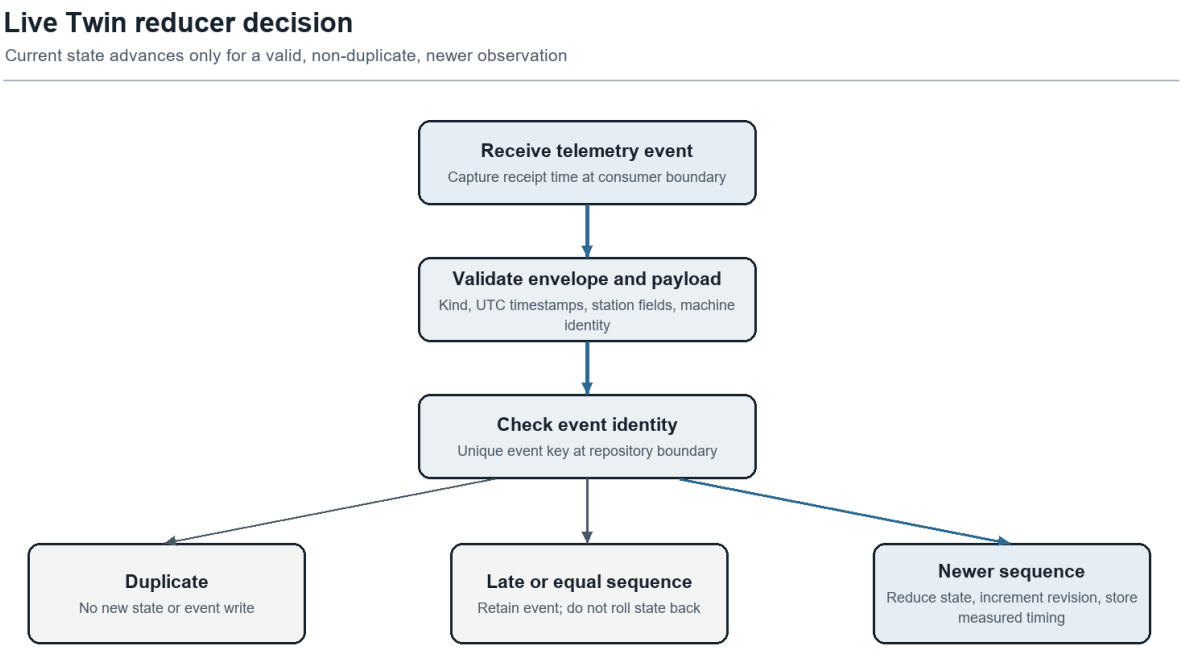


Figure 3.2: Live Twin reducer decision flow

3.7 Material-flow method

The Process Twin defines a versioned topology with five ordered nodes and material-flow edges. Its snapshot function reads available twin documents and orders them by station index. Production tokens represent a part and carry quality flags, signal values, and station lineage. Inspection calculates a bounded defect probability from upstream quality inputs; it sets a pass or fail status and sends accepted parts to Packaging while failed parts remain scrapped.

The simulator links adjacent station models. A downstream station pauses when its upstream output has not produced a corresponding part. An upstream station pauses when the difference between its output and downstream output reaches the maximum buffer of five. The system-level conservation invariant is therefore Packaging output less than or equal to Inspection output, which is less than or equal to Welding, CNC, and Stamping output. This invariant holds for the model's sequential pipeline; it is not a claim about all production systems.

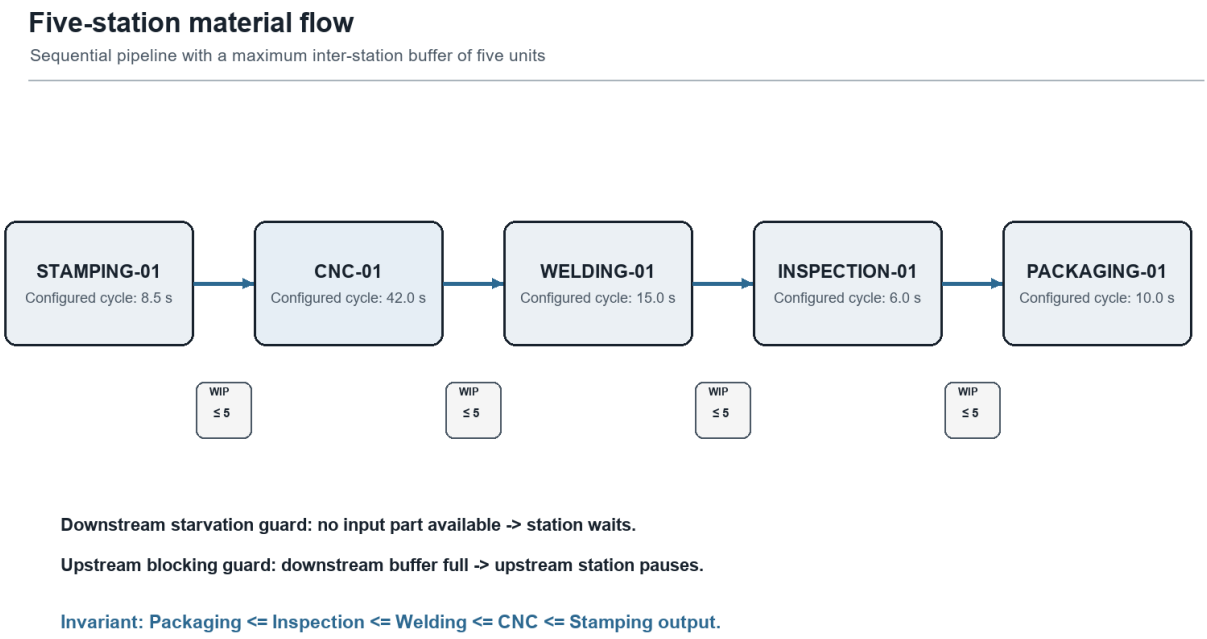


Figure 3.3: Five-station material pipeline and flow constraints

3.8 Snapshot and scenario method

A scenario request includes a name, description, horizon, time step, seed, optional snapshot ID, and station overrides. The worker captures or resolves a frozen line snapshot, calculates a fingerprint of Live Twin state, creates a baseline factory from the snapshot, and runs it forward. It then creates a second factory from the same snapshot and seed, applies the requested changes only to that factory, and runs the same horizon. The worker calculates output, scrap, quality, energy, health, bottleneck, and trajectory comparisons.

After both runs, the worker recalculates the live-state fingerprint. If it differs, the scenario is rejected as a critical immutability violation. Completed results are persisted through the scenario repository. This flow controls a specific risk—accidental modification of live twin state. It does not validate model fidelity or guarantee that all external side effects would be isolated if new integrations were added.

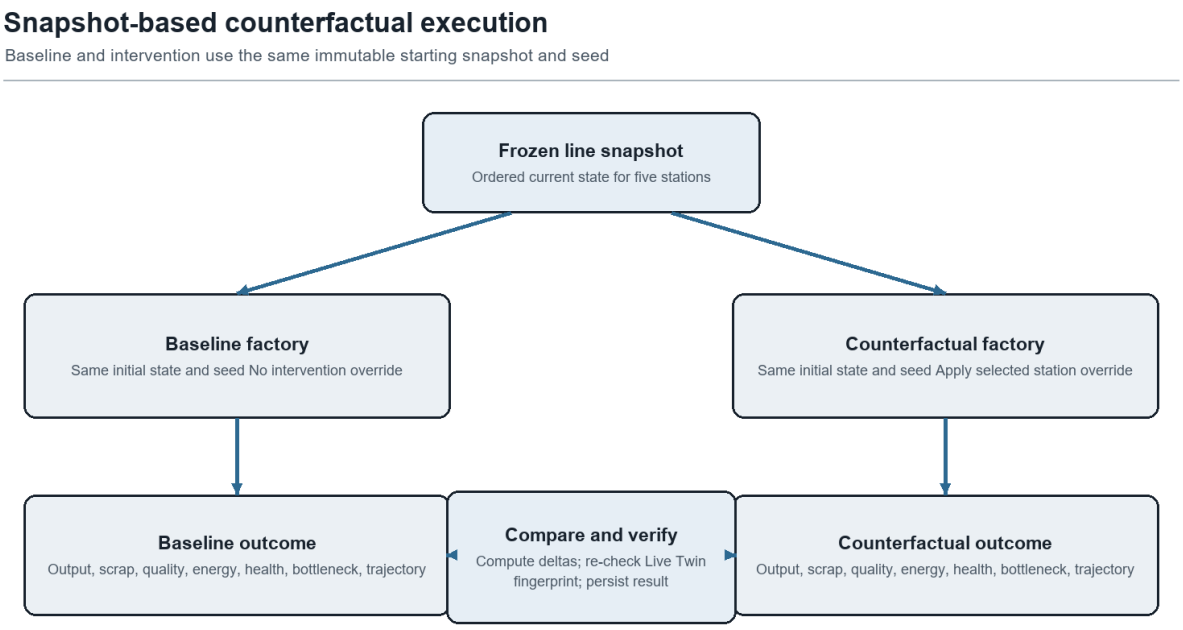


Figure 3.4: Frozen snapshot baseline and counterfactual execution

3.9 API and user-interface method

FastAPI routes expose readiness, legacy machine telemetry, current twins, event history, process topology and views, production tokens, defect traces, scenario execution, and simulator controls. The domain contracts are mirrored into TypeScript types. The dashboard uses polling hooks for versioned endpoints and has an MQTT hook for browser streaming. The live-first policy requires API health in the operational UI rather than silently making simulated mock state appear live.

The interface is organized around decisions. Overview answers whether the line appears active and what its totals are. Production shows station sequence, buffers, tokens, and defects. Machine views show station-level state. Twin Studio combines the live cockpit, what-if comparisons, and process lineage. Events supports history review; System displays service readiness and synchronization instrumentation. This organization is described from the source routes and components, not from a user study.

3.10 Verification and evaluation design

Verification is separated into levels. Unit tests target model direction, determinism, validation, reducer idempotency, snapshot consistency, token flow, bottleneck logic, and scenario isolation. API tests exercise routes and error responses. Frontend build evidence verifies type checking and bundling. Deployment manifests and health probes establish an integration design, but a successful frontend build or unit test does not prove that Docker or Kubernetes was started in a target environment.

The dated delivery log reports 48 backend test cases in the final material-flow verification entry, with two skipped, and a successful frontend production build with no errors. That is repository-reported

evidence, not a fresh test run performed for this thesis. The available log does not provide raw test output beyond its summary or measured runtime across hardware. Accordingly, this thesis reports implementation checks and designs a future empirical evaluation rather than inventing latency, fidelity, or throughput results.

3.11 Research ethics and operational constraints

The prototype uses no personal manufacturing data or customer records in the described project scope. It models a generic automotive-component line. If a real site is added, operational telemetry may be commercially sensitive and can reveal production capacity or maintenance condition. Data collection therefore requires organizational authorization, data minimization, retention rules, access control, and a threat model. The application should not be used to issue machine-control commands until control ownership, safe states, network segmentation, human approval, and safety certification are defined.

3.12 Reproducibility record

Reproduction requires the source repository, its Python and Node dependency lock/configuration, the chosen persistence mode, MQTT configuration, and fixed random seed. The API accepts seed parameters for scenario presets and the simulator assigns a run identifier to event identity. The repository log names the verification commands, but environment versions and a full experimental dataset are not included here. A complete scientific replication package should archive the source commit, dependency versions, configuration, generated traces, test output, benchmark harness, and hardware details.

CHAPTER 4 SOFTWARE AND HARDWARE REQUIREMENTS AND DESIGN

4.1 Functional requirements

The design shall generate or accept telemetry for five named stations; validate event envelopes; maintain one current twin record per machine; retain event history; ignore duplicate events; prevent late sequences from rolling state back; calculate observed timing fields; expose the station graph and a frozen snapshot; identify the effective bottleneck; emit and query part tokens; preserve lineage and quality flags; trace upstream defect contributors; run baseline and counterfactual simulations; persist scenario results; provide a live-first dashboard; and expose liveness and readiness routes.

4.2 Non-functional requirements

Determinism is required for controlled replay. Modularity is required so model, reducer, process, scenario, API, storage, and UI concerns can be evolved independently. Contracts must constrain values and reject schema drift. The event processor must be idempotent at the persistence boundary. State comparisons must use common initialization and seeds. Readiness information must distinguish startup from healthy execution. The application should remain runnable without cloud credentials and support a local in-memory repository while retaining a durable MongoDB path.

4.3 Software and hardware environment

The backend uses Python, Pydantic, FastAPI, a MongoDB driver, and an MQTT client. The frontend uses React 19, TypeScript, Vite, browser polling, and an MQTT-over-WebSocket hook. Mosquitto is the MQTT broker and MongoDB is the durable document store. Docker Compose orchestrates the full local stack, while Kubernetes manifests describe a multi-service deployment.

The model can be developed on a conventional workstation. No PLC, sensor, industrial PC, CNC controller, welding controller, or hardware-in-the-loop test equipment is part of the supplied repository evidence. This is a software prototype. A future installation requires edge devices or gateways, physical machine signal definitions, network interfaces, clocks, calibrated sensors, protective network infrastructure, and approval by the facility's engineering and cybersecurity teams.

Table 4.1 Main software components

Component	Technology or module	Responsibility
Event contract	Pydantic domain models	Typed messages and stored twin records
Simulator	Pure-Python causal models	Station state and telemetry generation
Event transport	Eclipse Mosquitto and Paho consumer	Asynchronous event distribution
Live state	LiveTwinService	Validation, sequencing, reduction, timing
Persistence	Repository interface, MongoDB, in-memory adapter	Current state, events, telemetry, tokens
Process layer	ProcessTwinService	Topology, snapshots, tokens, bottleneck and defect trace
Scenario layer	ScenarioWorkerService	Isolated forward comparisons and results
API	FastAPI	Versioned application boundary

Component	Technology or module	Responsibility
Dashboard	React and TypeScript	Monitoring, production, twin, and diagnostics views
Packaging	Docker Compose and Kubernetes	Repeatable service deployment structure

4.4 High-level architecture

The architecture separates generation, transport, reduction, process composition, scenario analysis, storage, API, and presentation. Telemetry can use MQTT or the controlled event ingestion route. Both routes call the same Live Twin service. The process layer queries the twin service and repository. Scenario execution uses snapshots and isolated factories. The browser consumes HTTP endpoints and may subscribe to the MQTT WebSocket stream. Storage is abstracted so local development can run in memory and container deployments can use MongoDB.

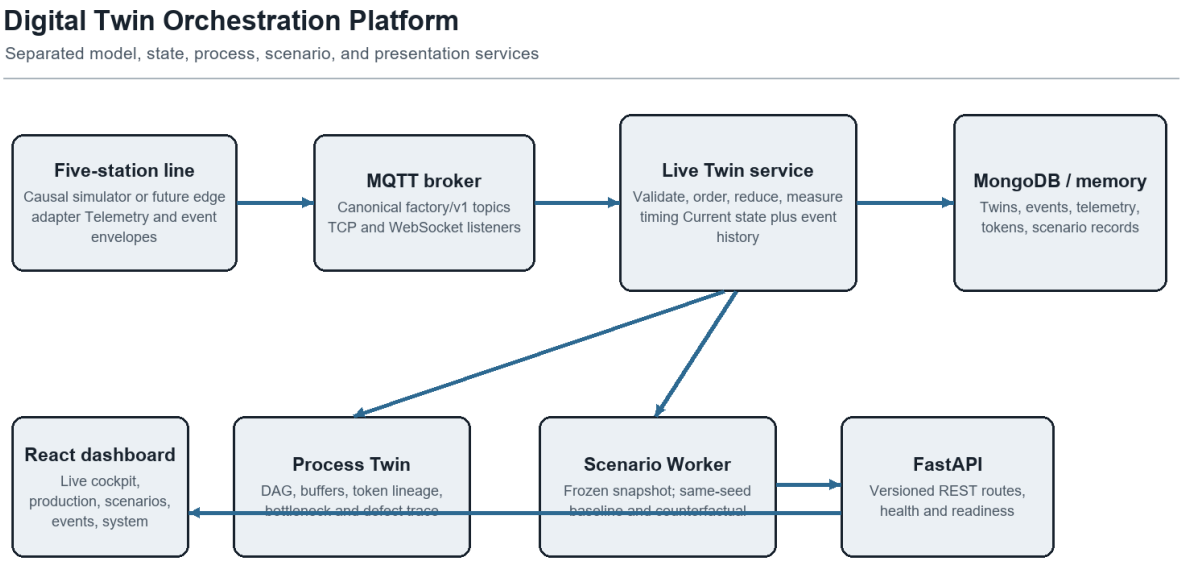


Figure 4.1: Reference architecture of the Digital Twin Orchestration Platform

4.5 Station configuration and nominal capacity

The station configuration is declared in code rather than loaded from an external plant database. The configured cycle times are Stamping 8.5 seconds, CNC 42 seconds, Welding 15 seconds, Inspection 6 seconds, and Packaging 10 seconds. Stamping, CNC, and Packaging also expose nominal RPM values in the model configuration. The 42-second CNC cycle defines the nominal line constraint in the default configuration.

For comparison only, theoretical station capacity is calculated as 3,600 seconds per hour divided by configured cycle time. This produces approximately 423.5 cycles/hour for Stamping, 85.7 for CNC, 240 for Welding, 600 for Inspection, and 360 for Packaging. This model-derived comparison omits

downtime, buffers, yield, shift breaks, changeovers, operator interaction, and real cycle-time variation. It must not be used as measured factory output.

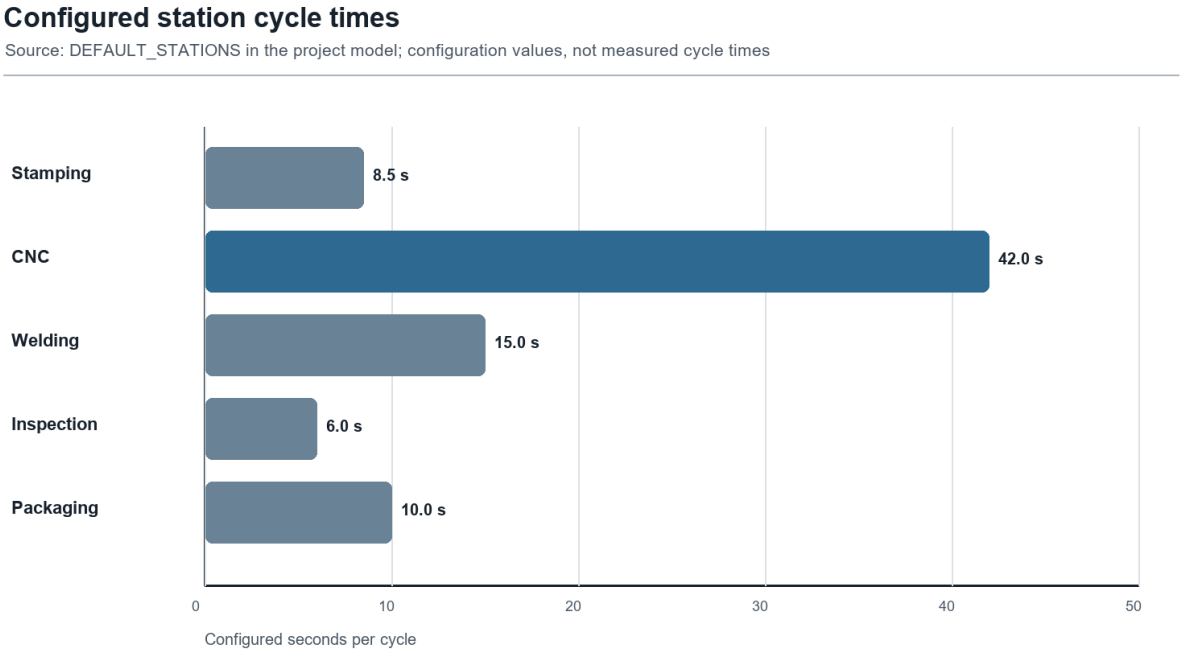


Figure 4.2: Configured station cycle times

4.6 Causal station models

The common station base class owns the model clock, station state, fault state, cycle accumulator, energy total, seeded random stream, station connections, and buffer limit. Each concrete station defines its own degradation variables and equations. All models return bounded telemetry fields and distinguish idle or stopped behavior from active production. The model attenuates activity while idle, faulted, offline, or in maintenance so that zero production does not coincide with continuing cutting force or active wear.

The Stamping model links die wear and fault severity to required force and resulting current, vibration, temperature, cycle time, quality, health, and anomaly indicators. The CNC model separately represents tool wear and bearing degradation. Cutting force is influenced by load and tool wear; current reflects force and bearing wear; temperature includes wear and a coolant failure penalty; RPM tracking error changes with load and degradation; and health and quality are bounded. A tool change resets tool wear and clears relevant fault state without pretending that every underlying component has been renewed.

Welding models electrode wear and clamp degradation with current, voltage, temperature, and quality consequences. Inspection supplies quality-related station behavior and does not mimic a full machine-vision classifier. Packaging models conveyor wear, energy, vibration, RPM, and cycle time.

These formulations produce causal direction for scenario and unit tests, but the repository itself notes that coefficients require calibration against public or site datasets before physical validation.

4.7 Material constraints and buffer semantics

The factory wires adjacent station models into a sequential chain and sets a maximum buffer of five. A station with an upstream relation cannot advance when it has no upstream part available. A station with downstream relation pauses when the gap between its own completed count and the downstream completed count reaches the buffer limit. The flow-control code therefore handles starvation and blocking at the production-advance step, before the next telemetry summary is emitted.

The model's count invariant is explicit: packaging output cannot exceed inspection, welding, CNC, or stamping output. Work in progress is derived from differences between adjacent cumulative counts. The dashboard shows queue indicators and STARVED/BLOCKED tags. The current representation is adequate for a single path with uniform tokens; alternative routings, rework loops, batch operations, changeover setup, and variable scrap destinations need an extended process model.

4.8 Event and twin data model

The EventEnvelope stores transport metadata separately from the TelemetryPayload. The latter includes common measurements plus optional fields such as force, hydraulic or pneumatic pressure, voltage, RPM, and fault code. The Live Twin stores nested identity, operational, telemetry, health, production, faults, desired state, synchronization, last event ID, revision, and update time. This nested shape reduces the need to reconstruct a current station view from many independent records.

MongoDB stores current twins, event envelopes, telemetry history, scenarios, and production tokens in separate collections. The Mongo twin repository creates indexes for station order, event machine/time, telemetry event identity and machine/time, and token identity, emitted time, and quality. The repository uses a unique event key as a duplicate barrier. A comment in the source notes that stronger transaction guarantees may require a MongoDB replica set when write sequences span collections.

Table 4.2 Event timing fields

Field	Source	Meaning	Limitation
generated_at	Producer	Timestamp when observation was formed	Depends on producer clock
received_at	Live Twin consumer	Timestamp at service receipt	Not present until consumption
delivery_latency_ms	Receive minus generated time	Observed message delay proxy	Clock skew can distort value
processing_latency_ms	Monotonic counter in reducer	Local validation and reduction time	Not full API or database latency
source_age_ms	Current implementation from generation-to-receive difference	Age of the source observation at receive	No value for absent or untrusted timestamps

4.9 Live Twin state transitions

The reducer receives a telemetry event and first checks kind and time-zone validity. It validates the nested payload and machine identity. A newer sequence produces a new materialized state and an updated revision. An older or equal sequence preserves the current state while retaining the event record. If the same event ID is repeated, repository insertion is not applied again. This architecture distinguishes sequence monotonicity from event identity: both checks matter because a message can be repeated or arrive late.

The synchronization object records the latest sequence, generation and receipt times, delivery latency, processing latency, and source age. Desired-state commands are represented separately from observed operational state so that requested control and actual state can later be distinguished. The current implementation's event reducer consumes telemetry; it does not implement a complete command acknowledgement lifecycle for real equipment.

4.10 Process Twin, topology, and snapshots

The canonical topology version is five-station-v1. Nodes carry machine ID, station type, index, nominal cycle time, and upstream/downstream machine IDs. Edges have a material_flow transition type. The Process Twin builds a view from the current station records and derives production line metrics. Snapshot capture orders the available twins by station index and records a snapshot ID, capture time, and topology version.

The snapshot is an immutable value passed to scenario creation. It contains the current twin documents and ensures baseline and counterfactual factories start from an equivalent state. The worker maintains a bounded in-memory cache of snapshots. Long-term scenario results are stored separately so a completed analysis can be retrieved without mutating current station records.

4.11 Production token and defect trace

The token has an identifier, part type, quality state, source station, emission time, carried flags, and ordered station lineage. The implementation records stamping quality and force when present; machining quality and CNC wear values; welding quality and voltage; inspection defect probability and verdict; and disposition. A pass adds Packaging to lineage; a failure is marked scrapped. The token repository allows retrieval and filtering by quality or station.

Defect attribution examines recent tokens and their carried flags to identify upstream contributors. The function reports the number of tokens evaluated, defective count, defect rate, primary source, and station-level contribution. This is a model-based attribution rule using the recorded token values. It is not a causal inference study on production data, nor should the percentages be presented as statistically validated process attribution.

4.12 Bottleneck calculation

For each station, the process service reads the current reported cycle time or uses the nominal configuration when needed. It applies a status factor: 1.00 for running, 1.15 for warning, 1.35 for degraded, 10.0 for fault or offline, 5.0 for maintenance, and 1.2 for idle. The maximum effective cycle time is the binding line time. Capacity per hour is calculated as 3,600 divided by that value. Slack is the nonnegative difference between the binding time and each station's effective time.

The penalty factors are design heuristics. They create consistent state-sensitive comparisons but do not originate from measured availability data. A real line should estimate effective cycle from logged production events or a validated cycle-time model and should distinguish temporary setup, planned downtime, faults, micro-stops, rework, and capacity sharing.

4.13 Scenario Worker and presets

The scenario worker runs an equal-horizon baseline and counterfactual. A result stores identifiers, status, seed, horizon, time step, overrides, both line outcomes, comparison values, and a sampled trajectory. The comparison includes output and scrap deltas, quality and energy changes, CNC health change, and bottleneck shift. A bounded number of snapshots is held in memory; scenario records use the configured repository adapter.

The codebase defines presets for CNC speed changes, a parallel CNC approximation, cadence derating, an unplanned CNC outage, preventive tool change, coolant failure, cadence boost, and welding electrode wear. A parallel CNC preset currently represents effective cycle-time scaling on the CNC station; it does not add a physically separate machine node to the process topology. That distinction should remain explicit in any interpretation of the preset.

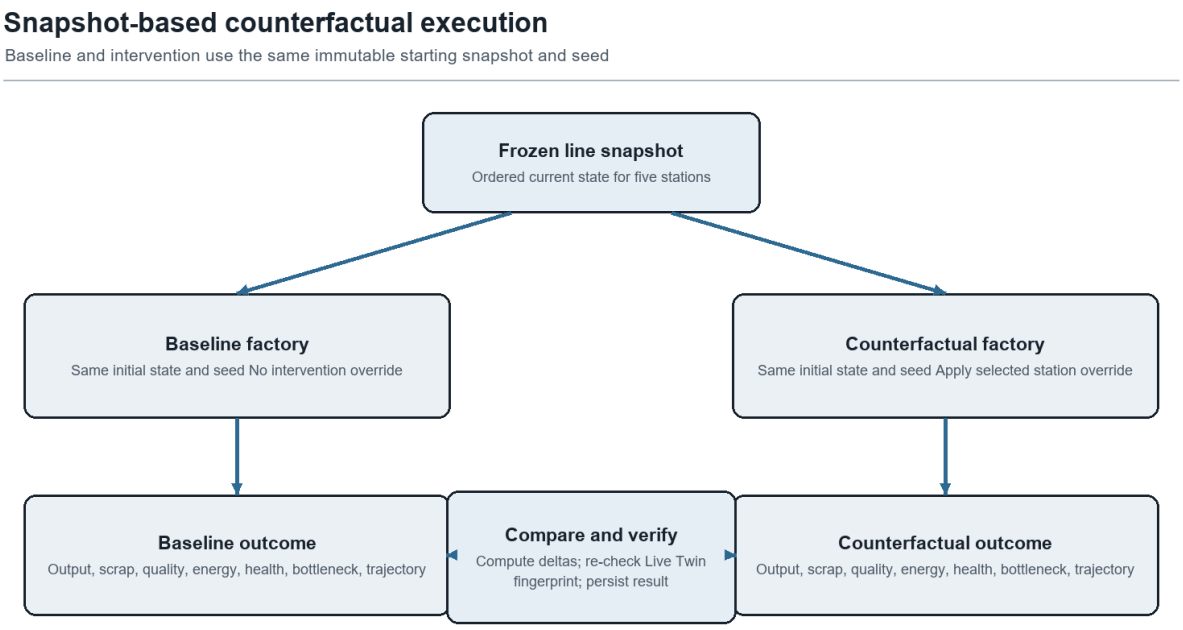


Figure 4.3: Frozen snapshot baseline and counterfactual execution

4.14 REST API design

The API uses unversioned compatibility routes for the original dashboard and versioned routes for twins, events, process, and scenarios. Health and readiness endpoints support orchestration. Requests are validated against typed response models where practical. Not-found conditions return 404 for unknown objects; malformed event ingestion returns 422; scenario failures are translated into a client error response. A scenario analysis route can optionally invoke a reasoning service, but the core scenario simulation is available without that service.

The API boundary is a practical integration surface, but CORS is broadly configured in the current FastAPI application and there is no authentication middleware in the examined source. Production rollout requires a stricter origin allowlist and identity-aware authorization. Write operations such as fault injection, reset, and scenario deletion require role-specific access rules before exposure on an operational network.

4.15 Frontend design

The React router organizes overview, 3D factory, production, machine registry, machine detail, twin studio, maintenance, analytics, events, and system pages. TypeScript interfaces mirror Python contracts. Polling hooks query readiness and twin/process endpoints. The header reports backend readiness and the selected persistence mode. The Process and Twin Studio pages visualize stations, WIP buffers, status, lineage, and scenario deltas.

An existing interface capture is included as a visual reference. It shows one simulator-backed dashboard state and should be read as a user-interface illustration. Values visible in a capture are not manufacturing measurements. The frontend build log provides compilation and bundling evidence, but does not establish usability, accessibility, or correctness of every dynamic screen under network interruption.

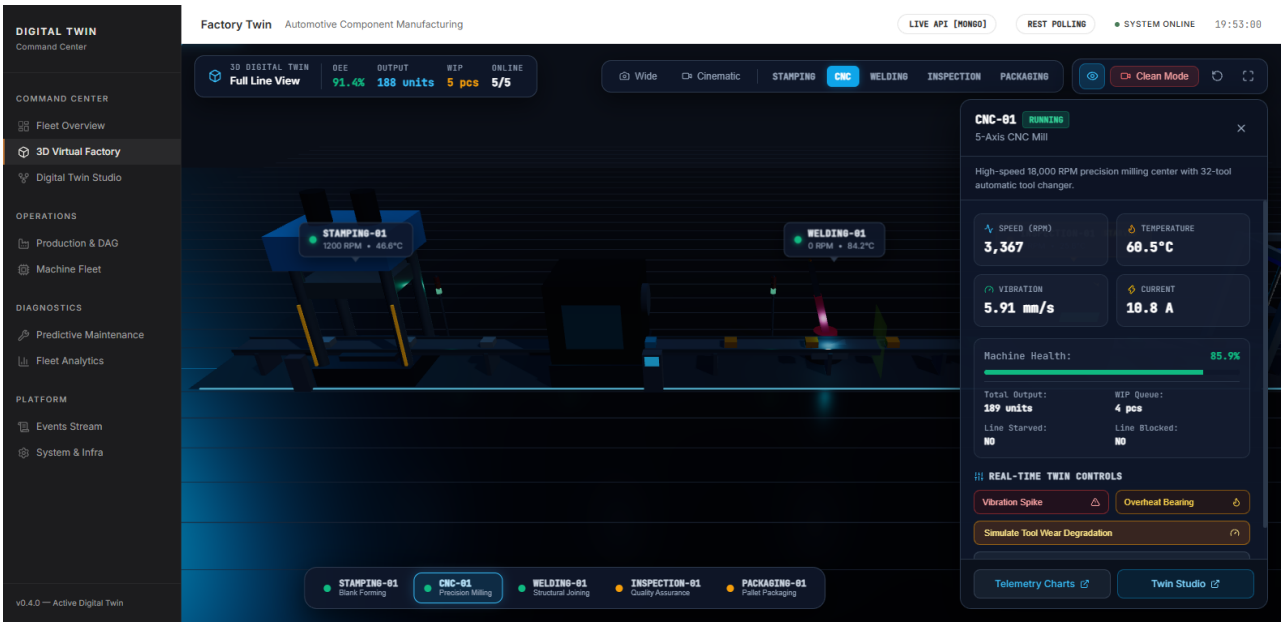


Figure 4.4: Prototype dashboard capture; displayed readings are simulator state, not physical factory measurements.

4.16 Deployment architecture

Docker Compose defines broker, database, backend, and frontend services on a shared network. The backend waits for database health and broker startup; health checks are assigned to services; MongoDB uses a named data volume. The front end is built as static assets and served through Nginx with API and MQTT endpoints configured by build arguments. Kubernetes files define corresponding workloads, services, configuration, storage, ingress, and health/readiness probes.

These files document a deployable topology, not a successful production deployment. Image tags, storage class, ingress controller, secrets, access policies, TLS, backup, and failure domains must match the intended cluster. The provided MongoDB standalone setup is a development and demonstration configuration, not a high-availability production database. Broker anonymous access must be disabled and replaced with authenticating clients and topic authorization.

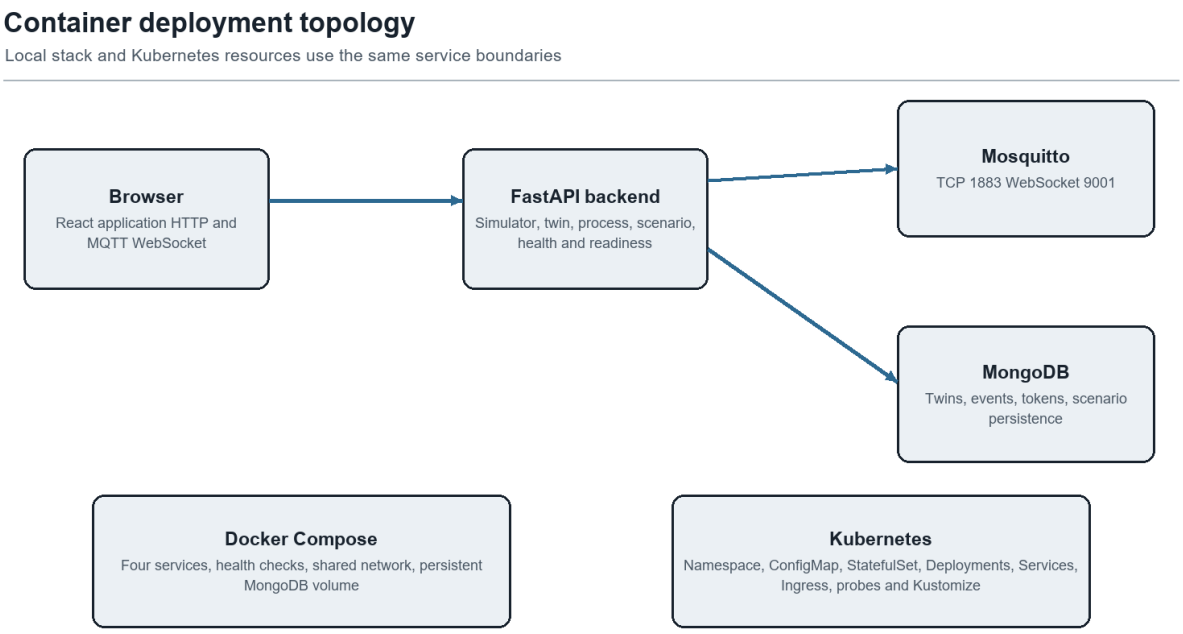


Figure 4.5: Container and Kubernetes deployment topology

4.17 User workflows

An operator opens the overview and checks readiness before trusting live views. The operator selects the production or machine page to inspect current simulated state, station status, buffer counts, and event history. To explore an intervention, the user opens the Twin Studio, selects a scenario and horizon, runs the comparison, and reviews baseline/counterfactual cards and station-level outcomes. The user can follow a production token through lineage and inspect quality flags. A maintainer checks system readiness and measured synchronization fields, while understanding that metrics are unavailable until an event with valid time data is consumed.

The workflow is intended for monitoring and analysis. It does not send control directly to a physical machine. Simulated command and fault controls are useful for exercising the interface and model state, but connecting them to equipment would require control-system ownership, operator authorization, safety analysis, state reconciliation, command acknowledgements, and fail-safe behavior.

4.18 Design trade-offs

The architecture prefers explicit module boundaries over a single all-purpose service. This makes the project easier to test and evolve but introduces more contracts and data transformations. MongoDB is flexible for nested twin documents and event history, but transactional guarantees and operational lifecycle need careful configuration. MQTT decouples publishers and consumers, but topic access and delivery semantics must be designed. Polling is simple to deploy, while WebSocket or MQTT streams can reduce refresh delay and add connection-management complexity.

Reduced-order physical models are more interpretable than opaque random telemetry, but have lower fidelity than calibrated multiphysics models. Frozen snapshots improve comparison validity, but scenario results remain conditional on state quality and model assumptions. The dashboard offers a wide operational surface in the prototype, while the absence of identity and usability evaluation limits production readiness. These trade-offs are recorded to guide future engineering decisions rather than treated as resolved universal choices.

CHAPTER 5 RESULTS AND DISCUSSION

5.1 Implementation status

The repository history records staged implementation of the system. The early phase introduced strict contracts and deterministic station models. Subsequent work added a MongoDB/in-memory repository interface and Live Twin reducer; process topology, snapshots, tokens, bottleneck and defect analysis; counterfactual scenario execution; a typed live-first dashboard; and container and Kubernetes packaging. The October 2026 entries record additional material-flow coupling, starvation and buffer blocking, token persistence, event routes, and a dashboard redesign.

The final material-flow log reports 48 backend test cases, two skipped, with the run described as successful. It also reports a frontend production build with zero errors. This is the evidence available in the project progress record. It is not a benchmark report, a field acceptance test, or independently repeated verification. A thesis submission should retain the source repository and test output if the examiner asks for detailed confirmation.

5.2 Functional outcomes

The implemented backend maintains a current twin record and recent event information per station. The process layer reports topology, process view, snapshots, bottleneck status, tokens, and defect traces. The scenario worker returns comparable baseline and counterfactual line outcomes and preserves Live Twin state. The dashboard exposes these structures through station cards, production queues, token inspection, scenario cards, analytics, events, and system views.

At the material-flow boundary, starvation and blocking are implemented in the causal model. A downstream station cannot produce without upstream supply, and a station may not run its upstream buffer beyond five units. The progress log reports a regression check for the line conservation invariant. These behaviors are stronger evidence than a UI-only flow animation because the invariant is enforced in production advance logic and tested in the backend model suite.

5.3 Model configuration results

The model's nominal cycle-time configuration makes CNC-01 the slowest station at 42 seconds. The derived idealized capacity proxy is 85.7 cycles per hour for CNC-01. The other stations have higher nominal rates before disruptions. This configured relationship explains the default bottleneck designation. It does not include line availability, product mix, scrap, downtime, maintenance, or physical measurement.

The 3D dashboard capture shows a particular simulator state, including machine status cards, telemetry values, WIP indicators, and navigation. The screenshot is useful as implementation evidence for visual integration. Since it records one application session and the backend is a simulator-backed prototype, it is not evidence of actual factory throughput, OEE, health, or production quality.

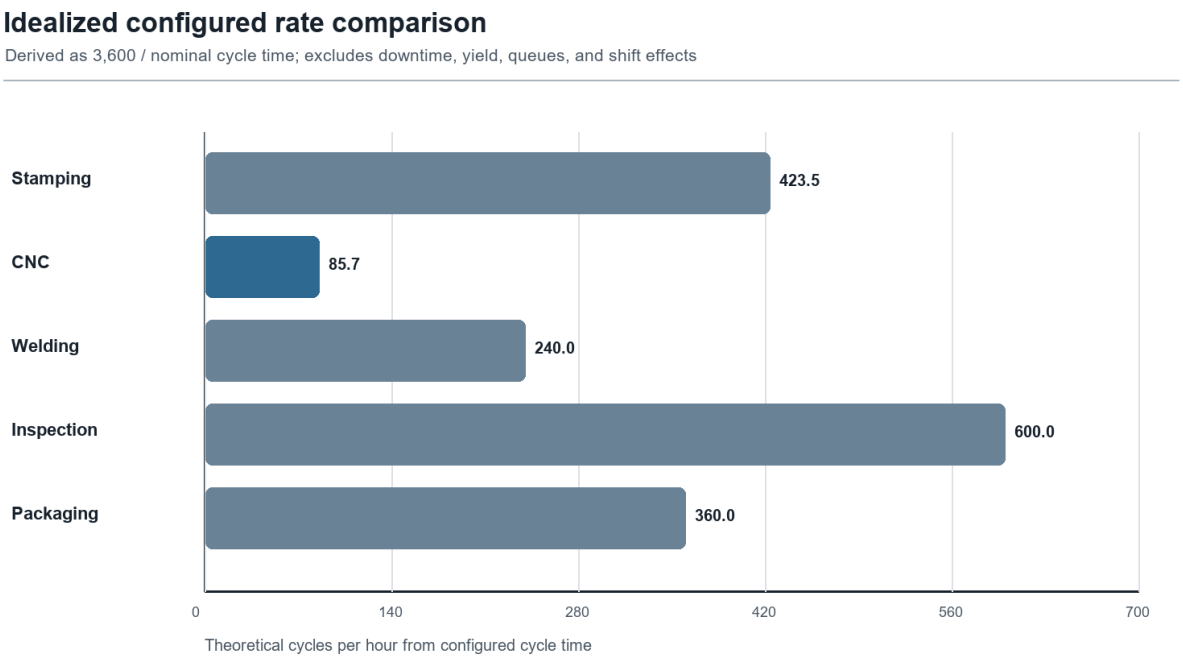


Figure 5.1: Idealized capacity derived from configured cycle times

5.4 Verification results in the repository log

The latest recorded verification summary on 2026-10-20 states that 48 backend tests ran successfully with two skipped, and that the production frontend build passed. Earlier phase logs give smaller cumulative counts as modules were added: 14 after initial causal contracts, 27 after live twin storage, 37 after process twin work, 45 after scenarios and dashboard work, and 47 after deployment work. These figures reflect successive logged snapshots; they are not independent test populations and should not be summed.

The two skipped tests are associated in the project log with optional Bedrock integration. Core simulation and twin behavior is intended to operate without cloud credentials. The exact individual pass/fail output is not embedded in this thesis; the status is attributed to the progress record. The chart below summarizes only the final recorded test count and skipped count.

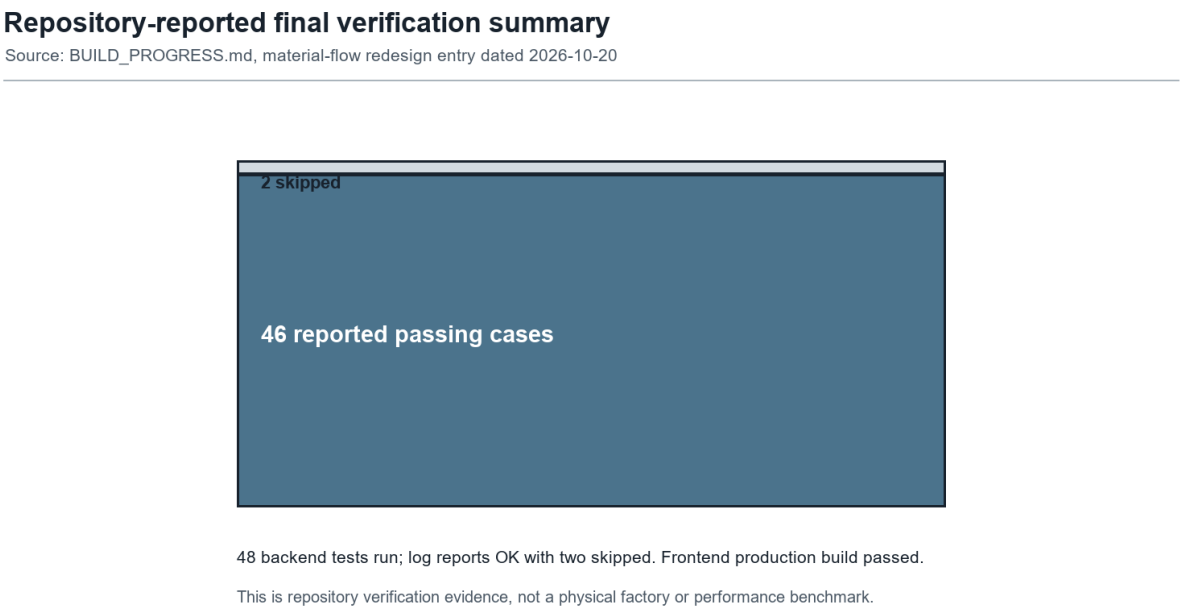


Figure 5.2: Repository-reported final verification summary

5.5 Test behavior coverage

The source test suite includes deterministic seed replay, fault direction, tool-change reset, invalid time step rejection, telemetry validation, duplicate event no-op, preservation of current state for late events, machine identity mismatch rejection, ordering, bounded history, MQTT adapter behavior, topology structure, snapshot capture, token progression and filtering, nominal and fault-driven bottleneck analysis, defect tracing, scenario immutability, counterfactual direction, repository operations, health/readiness routes, and API behavior.

These tests provide evidence about specified code paths and regression behavior. Unit tests cannot establish whether the causal coefficients represent a real machine. A test that verifies a coolant failure increases thermal burden validates direction within the programmed model, not the magnitude or timing of a real coolant fault. Similarly, a test for scenario immutability verifies the state fingerprint boundary exercised by the worker but does not prove that future side effects in new services are isolated.

5.6 Determinism and state consistency

Deterministic replay supports repeatable comparisons. Station-specific random streams reduce unintended cross-station coupling in pseudo-random variation. Event IDs derived from stable run and sequence fields make retries recognizable. The reducer tracks sequence state and stores late events without replacing the newest document. The repository adds an event identity uniqueness barrier. These mechanisms address distinct failure modes: random variance, duplicate delivery, stale update, and duplicate persistence.

The repository reports tests that exercise deterministic behavior, duplicate no-op, late-event handling, and strict scenario fork immutability. This makes the system's consistency policy inspectable. Clock synchronization remains an external assumption for delivery latency; the reducer requires timezone-aware timestamps, but does not validate source clock drift against a trusted time server.

5.7 Material-flow behavior

Before the material-flow coupling changes, station outputs were not necessarily constrained by upstream supply or downstream capacity. The current implementation adds a sequential production guard and bounded buffer. This supports a conservation relation across the line and allows the UI to show starvation and blocking as derived state.

The simple one-path model is appropriate for demonstrating flow control but cannot represent a full automotive process plan. In a real line, different parts may skip stations, split into branches, join at assembly, re-enter after rework, or consume several component lots. A more general process model would require explicit routing and token transition rules, discrete-event synchronization, buffer capacities by material type, and validation against actual production logs.

5.8 Scenario outcomes and interpretation

Scenario results compare a baseline with one or more station overrides from the same snapshot and seed. The implementation returns deltas for output, scrap, quality, energy, health, and bottleneck. A preventive tool-change scenario changes the tool state; a coolant failure scenario introduces a thermal fault; a cadence increase changes cycle-time multipliers and load; an electrode drift scenario explores quality propagation. The presets can reveal model behavior and help structure experiments.

The current project log does not include a saved scenario result dataset or measured result tables. Accordingly, no numeric scenario delta is reported as a result in this chapter. The scenario feature's evidence is functional: request definitions, isolated execution, comparison fields, and tests of directional behavior and immutability. A final empirical submission should export specific saved scenario results with the seed, snapshot ID, horizon, override, software revision, and trace.

5.9 Dashboard and operational observability

The interface provides an overview and station views; the production section exposes process sequence, WIP, token status, and defect trace; the system page queries readiness and timing fields; and the event page reads from the event log. The screenshot demonstrates the dark command-center interface and a machine-detail panel. These views connect code-level services to an operator-facing representation.

Operational observability is partly implemented through backend readiness and observed latency fields. The repository does not include an external metrics stack, a log aggregation deployment,

distributed tracing, alert rules, or retention policy for observability data. The fields should therefore be considered application instrumentation rather than a complete production observability program.

5.10 Deployment results and boundary

The codebase includes Dockerfiles, a complete Docker Compose service topology, broker configuration, and Kubernetes YAML for database, broker, backend, frontend, ingress, namespace, and kustomization. Readiness and liveness probes are represented. The project progress file says container and Kubernetes package implementation is complete, but does not include a deployment execution log.

The conclusion supported by this evidence is that deployment artifacts have been authored and the frontend build and backend tests are recorded as successful. It would be stronger than the evidence permits to state that the platform was deployed in a production cluster, sustained a particular load, or recovered from an outage. Such claims require a reproducible deployment and results.

5.11 Limitations of evaluation

There is no calibrated plant dataset, physical hardware connection record, measured MQTT-to-dashboard latency distribution, stress test, high-availability test, or operator usability study in the examined project files. The engineering logs explicitly reserve observability and experiments as further work. The energy equation uses a fixed supply assumption and a modelled current. The station models use reduced-order coefficients without a calibration dataset. The quality model is a simplified score and defect probability rule.

These limitations affect the interpretation, not the existence of the implementation. The thesis can report software architecture, source-level behavior, repository-recorded verification, and model-derived illustrative rates. It cannot claim validated predictive maintenance, real-time guarantees, improved OEE, lower scrap, energy savings, or deployment readiness for a live plant.

5.12 Discussion

The most important result is an integrated architecture that keeps separate concerns connected through explicit contracts. This allows the same telemetry definition to support simulation, MQTT, HTTP ingestion, storage, API serialization, and frontend types. It also reduces the risk that scenario execution changes current operating state. The project extends an individual machine twin into a process view with constrained material and traceable tokens.

The architecture has to advance from a prototype to a validated system through evidence. The next research step is not simply to add more dashboard tiles. It is to select a representative machine, collect trustworthy signals, calibrate a small set of equations, define a measurement protocol, compare prediction and observed behavior, measure software timing under defined load, and assess operator use. Security changes are also prerequisites before any external network or control command is introduced.

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This thesis has presented the design and implementation of a Digital Twin Orchestration Platform for an Industrial IoT manufacturing line. The prototype includes five station models, a versioned event contract, an idempotent and sequence-aware Live Twin reducer, MongoDB and in-memory repositories, a process graph, snapshot capture, constrained production tokens, bottleneck and defect analysis, isolated scenarios, versioned FastAPI routes, a React dashboard, and deployment manifests.

The design responds to the central engineering problem by joining machine state and process state without collapsing them into one service. Simulator observations become events; the Live Twin materializes current state; the Process Twin composes the line; and the Scenario Worker compares isolated futures from a shared snapshot. The bounded material-flow implementation adds starvation and buffer blocking, while token lineage preserves selected upstream quality indicators to inspection.

Repository logs report final backend verification of 48 tests with two skipped, plus a successful frontend production build. The evidence supports the statement that the implemented software paths were exercised according to the project log. It does not demonstrate physical model fidelity, production deployment, or improved manufacturing performance.

6.2 Objective attainment

The data contract objective is addressed by strict Pydantic models and mirrored TypeScript types. The deterministic simulation objective is addressed by station-specific reduced-order models with seeded random streams. The synchronization objective is addressed by a reducer that records receive time, computes timing, tracks sequence, and uses event identity. The line-flow objective is addressed by a canonical graph, bounded buffers, starvation checks, tokens, and defect trace. The counterfactual objective is addressed by same-seed baseline and override factories created from a frozen line snapshot, followed by an immutability fingerprint check. The deployment objective is addressed by Docker and Kubernetes files.

Objective attainment should be interpreted at the implementation level. An objective such as "provide deterministic replay" is supported by code and a recorded test name. An objective such as "improve line throughput" would require operational measurement and is not claimed. An objective such as "support real IIoT" remains partly complete because a real industrial adapter, authentication, clock-quality strategy, and safety boundary have not been commissioned.

6.3 Limitations

The five stations are model instances rather than documented real assets. Nominal cycle times and physics coefficients are configuration values. The line is serial and single-path. The quality logic is reduced. The broker example allows anonymous connections. Authentication and authorization are

absent in the visible application code. The deployment files are not accompanied by a recorded cluster test. The repository does not provide calibrated dataset traces, latency benchmarks, throughput trials, or operator study results.

These constraints mean the command center should be used as a prototype, research demonstrator, or integration starting point. It should not be used as a certified control plane or as evidence of predictive maintenance performance without additional work.

6.4 Future work: physical and semantic integration

The first extension is to connect a real machine or a hardware test bench through a dedicated adapter. The adapter should map controller tags to versioned telemetry fields, retain engineering units, preserve source timestamps, report signal quality, and separate measured values from defaults. OPC UA is one possible industrial interface; MQTT may remain useful for event distribution. The implementation should document information-model identifiers and map them to the project contract.

The next extension is calibration. For one station, collect normal and degraded sequences under approved conditions, estimate selected model parameters, reserve separate data for validation, and document uncertainty. Model fit should be assessed against more than average values: trends, fault transients, quality response, and operating-state transitions matter. Changes to parameters should be versioned so scenario outputs can be reproduced.

6.5 Future work: empirical system evaluation

A benchmark protocol should specify hardware, operating system, dependency versions, message rate, payload size, number of stations, broker settings, persistence mode, warm-up period, repeated runs, and clock source. It should report median and tail latencies, event loss and duplicate handling, database write rate, API response distribution, browser refresh delay, and recovery behavior. Results should be accompanied by raw traces and scripts.

Model evaluation should compare predicted signals with held-out measured observations and report errors with confidence intervals. Material-flow validation should compare token counts and actual work-in-process against production records. Scenario evaluation should test whether the model ranks interventions correctly, not merely whether it produces an internally consistent delta. Operator evaluation should check whether users can interpret fault, blocking, starvation, and scenario information without confusing simulated estimates with measurements.

6.6 Future work: security and reliability

Before network integration, disable anonymous MQTT access, require unique client credentials or certificates, encrypt transport, and apply topic-level authorization. Add user authentication and role-based access for commands and scenario deletion. Restrict CORS origins, validate request authorization,

store secrets outside manifests, and define audit retention. Threat modeling should consider malicious telemetry, replay, denial of service, compromised dashboard sessions, and cross-tenant data access.

Reliability work should test broker and database outages, stale data, service restart, duplicate messages, replica failover, storage backup, and restore. Readiness must represent actual dependencies instead of simulator running state alone. Production deployments should use persistent MongoDB storage and an appropriate replica set, resource requests and limits, disruption policies, network policies, TLS, external secrets, and monitoring.

6.7 Future work: process and user features

A production-grade process model should represent alternate routes, rework loops, operation setup, batch sizes, mixed product types, explicit arrival and departure events, shift calendars, and resource constraints. Tokens should be linked to product genealogy and inspection outcomes while respecting data retention. Bottleneck analysis should use empirical utilization and event times where available, with status penalties treated as fallback estimates.

The dashboard can add user-configurable alarms, evidence export, scenario comparison history, accessibility refinements, and guided operator workflows. Each visual should carry a clear data provenance label: measured, derived from live state, model-generated, or historical. Usability feedback should shape the interface before the dashboard is deployed on a production floor.

6.8 Final statement

The project demonstrates a coherent software architecture for a manufacturing digital twin command center and a repeatable basis for continued engineering. Its strongest results are explicit data and state boundaries, isolated what-if execution, material-flow constraints, and traceable implementation. The next stage is evidence-led integration: connect a bounded physical use case, calibrate and validate the model, quantify software performance, harden security, and evaluate the interface with intended users.

REFERENCES

[1] G. Shao and M. Helu, "Framework for a Digital Twin in Manufacturing: Scope and Requirements," Manufacturing Letters, vol. 24, pp. 105-107, 2020. doi: 10.1016/j.mfglet.2020.04.004.

[2] International Organization for Standardization, ISO 23247-2:2021, Automation systems and integration - Digital twin framework for manufacturing - Part 2: Reference architecture, 2021.

[3] E. Negri, L. Fumagalli, and M. Macchi, "A Review of the Roles of Digital Twin in CPS-based Production Systems," Procedia Manufacturing, vol. 11, pp. 939-948, 2017. doi: 10.1016/j.promfg.2017.07.198.

[4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital Twin in Industry: State-of-the-Art," IEEE Transactions on Industrial Informatics, vol. 15, no. 4, pp. 2405-2415, 2019. doi: 10.1109/TII.2018.2873186.

[5] OASIS, MQTT Version 5.0, OASIS Standard, 7 March 2019. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[6] OPC Foundation, OPC Unified Architecture Part 1: Overview and Concepts, OPC 10000-1, version 1.05.06. Available: https://reference.opcfoundation.org/specs/OPC-10000-1/full.

[7] G. Shao, D. Kibira, and S. P. Frechette, "Digital Twins for Advanced Manufacturing: The Standardized Approach," in Digital Twins, Simulation, and Metaverse, Springer Nature, 2024.

[8] G. Shao, M. Kibira, D. Kibira, and others, Use Case Scenarios for Digital Twin Implementation Based on ISO 23247, NIST Advanced Manufacturing Series 400-2, National Institute of Standards and Technology, 2021. doi: 10.6028/NIST.AMS.400-2.

APPENDIX A REQUIREMENTS AND TRACEABILITY

A.1 Functional requirement register

ID	Requirement	Implementation location	Verification reference
FR-01	Validate versioned telemetry and reject malformed values	domain/contracts.py	Contract and API tests
FR-02	Create deterministic event identity from run, machine, kind, and sequence	domain/contracts.py	Event identity test
FR-03	Materialize current machine state from telemetry	services/live_twin.py	Live Twin reducer tests
FR-04	Preserve state when an older sequence arrives	services/live_twin.py	Late-event preservation test
FR-05	Treat duplicate event identity as an idempotent no-op	repositories and LiveTwinService	Duplicate event test
FR-06	Record event generation and receipt timing	services/live_twin.py	Synchronization state tests
FR-07	Expose the five-station process graph	services/process_twin.py	Topology API test
FR-08	Capture an ordered line snapshot	services/process_twin.py	Snapshot tests
FR-09	Limit downstream starvation and upstream blocking	simulation/causal.py	Material conservation test
FR-10	Track token lineage and inspection disposition	services/process_twin.py	Token progression test
FR-11	Analyze effective bottleneck and station slack	services/process_twin.py	Bottleneck tests
FR-12	Trace defects to carried upstream flags	services/process_twin.py	Defect trace test
FR-13	Run baseline and counterfactual from same snapshot and seed	services/scenario_worker.py	Determinism tests
FR-14	Prevent scenario mutation of Live Twin state	services/scenario_worker.py	Fork immutability test
FR-15	Persist or retrieve scenario results	scenario repositories	Repository CRUD test
FR-16	Provide live-first process and twin views	frontend services and hooks	Frontend build evidence
FR-17	Expose liveness and readiness endpoints	main.py	Health/readiness test
FR-18	Package multi-service deployment resources	Docker and k8s files	Source inspection; cluster execution not recorded

A.2 Non-functional requirement register

ID	Requirement	Evidence or status
NFR-01	Reproducible seeded simulation	Deterministic replay test name in suite
NFR-02	Strict typed boundary data	Pydantic strict models
NFR-03	Bounded metrics and history	Field validators and bounded token/snapshot history
NFR-04	Storage abstraction	In-memory and Mongo repository implementations
NFR-05	Live Twin immutability during scenarios	Fingerprint check and recorded test
NFR-06	Container readiness checks	Compose and Kubernetes probes
NFR-07	Industrial security	Not complete; anonymous MQTT is enabled in example config
NFR-08	Model validation against physical measurements	Not available in repository evidence

ID	Requirement	Evidence or status
NFR-09	Performance and scale measurement	Not available in repository evidence

A.3 Acceptance interpretation

Acceptance should be evaluated at the level of each requirement. A source file proves implementation presence; a test shows an intended behavior is executable under its fixture; a recorded test summary gives evidence of a run; a build log supports frontend compilation; a deployment manifest expresses target topology. None of these individually proves physical integration, operational availability, throughput improvement, or cybersecurity. A future project review should attach the exact code revision and test output to each claim.

APPENDIX B DATA DICTIONARY AND TOPIC REFERENCE

B.1 Core event envelope

Field	Type	Role
schema_version	string	Contract revision identifier
event_id	string UUID	Stable event identity and duplicate barrier
run_id	string	Simulation or acquisition run grouping
machine_id	string	Station identity
kind	enum	telemetry, state, fault, production, command, or summary
sequence	integer	Per-machine monotonic ordering within a run
generated_at	UTC timestamp	Producer observation time
received_at	optional UTC timestamp	Consumer receipt time; filled by service
correlation_id	optional string	Cross-service tracing or request grouping
payload	object	Event-kind-specific typed payload

B.2 Telemetry signal dictionary

Signal	Unit or range	Use
temperature_c	degrees Celsius	Thermal state
vibration_mm_s	mm/s, nonnegative	Mechanical condition proxy
current_a	amperes	Electrical load proxy
voltage_v	optional volts	Station-specific electrical signal
force_kn	optional kN	Press or cutting force
hydraulic_pressure_bar	optional bar	Hydraulic signal
pneumatic_pressure_bar	optional bar	Pneumatic signal
target_rpm / actual_rpm	optional rpm	Commanded and observed/modelled speed
cycle_time_s	positive seconds	Current effective cycle signal
energy_kwh	nonnegative kWh	Accumulated modeled energy
production_count	nonnegative integer	Cumulative completed units
wip_count	nonnegative integer	Work-in-process signal
is_starved / is_blocked	boolean	Flow constraint indicator
health_score	0-100	Normalized model health
anomaly_score	0-1	Normalized model anomaly indicator
quality_score	0-1	Normalized process quality indicator
tool_wear_percent	optional 0-100	Degradation state
bearing_degradation_percent	optional 0-100	Bearing state

B.3 Documented MQTT topic taxonomy

Topic	Intended payload	Documented delivery
factory/v1/{machine_id}/telemetry	Sensor or model telemetry	QoS 0, non-retained
factory/v1/{machine_id}/state	State transitions	QoS 1, non-retained
factory/v1/{machine_id}/fault	Faults and alarms	QoS 1, retained
factory/v1/{machine_id}/production	Part completion	QoS 1, non-retained
factory/v1/{machine_id}/command	Operator or system command	QoS 1, non-retained
factory/v1/system/summary	Plant summary	QoS 0, non-retained

The implemented consumer subscribes to factory/v1/+/telemetry. The broader topic list describes the intended taxonomy and should not be mistaken for a guarantee that every event type has a complete production consumer or publisher.

APPENDIX C API REFERENCE

C.1 Health and compatibility routes

Method and route	Purpose
GET /health	Process liveness response
GET /ready	Simulation and persistence configuration readiness view
GET /machines	Current simulator machine telemetry
GET /machines/{machine_id}	One simulator machine record
GET /machines/{machine_id}/twin	Legacy machine twin state
GET /summary	Factory summary
GET /simulate/state	Simulator running state
POST /simulate/start or /simulate/stop	Start or stop simulation
POST /simulate/degrade_cnc or /simulate/reset_cnc	CNC demonstration controls
POST /machines/{machine_id}/inject-fault or /reset	Station simulation controls

C.2 Versioned twin and event routes

Method and route	Purpose
GET /api/v1/twins	List current twin documents
GET /api/v1/twins/{machine_id}	Retrieve one current twin
GET /api/v1/twins/{machine_id}/telemetry	Retrieve bounded telemetry history
GET /api/v1/events	Retrieve event history with optional machine filter
POST /api/v1/events	Controlled telemetry ingestion through shared reducer

C.3 Process and token routes

Method and route	Purpose
GET /api/v1/process/topology	Return process nodes and edges
GET /api/v1/process/snapshot	Capture an ordered immutable snapshot
GET /api/v1/process/view	Return combined process view
GET /api/v1/process/bottleneck	Return binding station, cycle, capacity, and slack
GET /api/v1/process/tokens	Query tokens with optional filters
GET /api/v1/process/tokens/{token_id}	Return token lineage
POST /api/v1/process/tokens	Emit a simulated token
GET /api/v1/process/defects/trace	Return model-based defect attribution

C.4 Scenario routes

Method and route	Purpose
POST /api/v1/scenarios/run	Run a caller-defined counterfactual
GET /api/v1/scenarios	List stored scenario results
GET /api/v1/scenarios/presets	List available presets
POST /api/v1/scenarios/presets/{preset_id}/run	Run one preset with horizon and seed
GET /api/v1/scenarios/{scenario_id}	Retrieve detailed scenario result
DELETE /api/v1/scenarios/{scenario_id}	Delete a stored scenario result
POST /api/v1/scenarios/analyze	Request a decision-oriented scenario summary
GET /api/v1/scenarios/strategies	List scenario strategy options

All mutating routes require security review before production exposure. The API source in this thesis is a catalog; request and response schemas are defined by the Python contract models and OpenAPI description generated by FastAPI.

APPENDIX D SCENARIO AND MODEL REFERENCE

D.1 Available scenario presets

Preset	Intervention represented	Caution
cnc_double_speed	CNC cycle-time multiplier 0.50 with elevated load	Increased wear is modelled; not a machine-control setting
cnc_parallel_cell	Effective CNC cycle scaling to approximate parallel capacity	Does not add a second node to topology
cnc_triple_speed_shift	CNC cycle-time multiplier 0.333	Intended to explore bottleneck migration
line_cadence_derate	CNC cycle-time multiplier 1.25 with lower load	Model hypothesis, not measured energy savings
cnc_unplanned_outage	CNC spindle-seizure fault override	Buffer response is simulated
preventive_tool_change	CNC tool change and wear reset	Does not model elapsed maintenance time in detail
coolant_pump_failure	CNC coolant fault and high-load override	Thermal response depends on reduced-order equations
line_cadence_boost	Stamping and CNC cycle-time multiplier 0.80	Evaluates model trade-off between cadence and wear
welding_electrode_drift	Welding wear/fault override	Quality propagation is modelled, not field-calibrated

D.2 Station model variables

Station	Modeled degradation or signals
Stamping	Die wear, force, vibration, current, temperature, quality, health
CNC	Tool wear, bearing degradation, coolant failure, force, RPM tracking, vibration, current, temperature, quality
Welding	Electrode wear, clamp degradation, voltage, current, temperature, quality
Inspection	Quality state and inspection-related fault behavior
Packaging	Conveyor wear, RPM, current, temperature, vibration, quality

APPENDIX E DEPLOYMENT AND OPERATOR GUIDE

E.1 Local development mode

The backend configuration accepts a persistence backend selection of memory or MongoDB. Memory mode is suitable for quick local inspection and resets with the process. MongoDB mode requires an available database endpoint and should be used when durable records are needed. The MQTT host, port, and enable flag are environment variables. Development should be isolated from any plant network.

The frontend uses configured API and MQTT WebSocket URLs. A user should check the header's backend readiness before interpreting live views. If the backend is offline, the live-first frontend policy should surface the connection failure rather than presenting mock state as current data. The simulator and UI can still be useful for development when using explicit local configuration.

E.2 Docker Compose topology

The Compose file defines four services: Mosquitto on TCP 1883 and WebSocket 9001, MongoDB on 27017 with a persistent named volume, FastAPI on 8000, and the frontend served on 3000. The backend is configured to use MongoDB and connect to the broker. The frontend receives API and WebSocket URLs as build arguments. Health checks are present for services.

Before a demonstration, verify container image builds, service readiness, MongoDB persistence after restart, broker connection, a telemetry event reaching the Live Twin, event history retrieval, token flow, and one scenario. Save logs with the software revision and configuration. Do not expose example ports to a public network without access controls.

E.3 Kubernetes topology

The manifest set contains a namespace, config map, MongoDB StatefulSet, Mosquitto deployment, backend deployment, frontend deployment, ingress, and Kustomize entry point. Backend liveness probes use /health and readiness probes use /ready. Database persistence, replica design, secret injection, ingress TLS, resource limits, network policy, service accounts, and observability must be configured for the target cluster.

E.4 Operator interpretation checklist

1. Confirm the status indicator reports backend readiness.
2. Confirm the selected persistence mode and MQTT connection state.
3. Read station status together with telemetry age and event sequence.
4. Distinguish WIP and buffer constraints from machine faults.
5. Inspect token lineage before interpreting a quality attribution.

6. Treat scenario output as a model comparison from a named snapshot.

7. Do not interpret simulator values as measurements from plant equipment.

8. Escalate physical safety or control actions to approved plant procedures.

E.5 Configuration and security checklist for future integration

Area	Required engineering action
Broker	Disable anonymous connections; configure TLS, client identities, and topic ACLs
API	Add authentication, role-based authorization, origin restrictions, and audit logs
Network	Segment OT and IT paths; restrict ingress/egress; document trust boundaries
Secrets	Inject credentials from a secret store rather than source-controlled manifests
Database	Configure replica set, backup, retention, access control, and restore test
Commands	Keep simulator controls separate from machine control; add human approval and safety analysis
Observability	Export metrics, logs, and traces with explicit retention and alert ownership
Model	Label every field as measured, calculated, or simulated

APPENDIX F VERIFICATION CATALOGUE

F.1 Test names in the source suite

The repository includes test cases with names covering the following behaviors: health and readiness; simulator generation and machine summary; unknown-machine responses; event ingestion and complete twin materialization; invalid event rejection; topology and process endpoints; scenario endpoints; bottleneck analysis; same-seed replay; CNC tool-wear direction; inspection response to upstream quality; tool-change reset; event identity and unobserved receive time; start and stop transitions; positive time-step validation; material-flow conservation; duplicate event idempotency; late-event preservation; machine-ID mismatch rejection; non-telemetry rejection; timezone-aware receipt time; station ordering; telemetry history bounds; repository factory selection; MQTT consumer handling; graph structure; snapshot precondition and frozen semantics; token progression and filtering; nominal and fault-driven bottleneck analysis; defect tracing under CNC degradation; process-view composition; scenario fork immutability; counterfactual health, coolant, and energy direction; preset metadata; scenario repository CRUD; legacy API shape; conservative output; and reset behavior.

F.2 Verification summary from dated project record

Date and milestone	Recorded backend test summary	Frontend evidence
2026-09-19 / initial model	14 tests passed	Not recorded
2026-09-20 / Live Twin	27 tests passed	Not recorded
2026-09-20 / Process Twin	37 tests passed	Not recorded
2026-09-20 / Scenario and UI	45 tests passed	TypeScript and Vite build passed
2026-10-20 / container remediation	47 tests run; 45 passed and two skipped	Production build passed
2026-10-20 / material-flow redesign	48 tests run; two skipped, run reported OK	Production build passed

These are dated entries from the source progress record. Earlier rows are intermediate snapshots, not cumulative counts to be added. The final entry is the relevant recorded summary for the current material-flow implementation.

F.3 Suggested field evaluation protocol

Step	Procedure	Output
1	Select one machine and define sensor points, engineering units, and clock source	Signed signal map
2	Capture representative normal and degraded data under approved conditions	Versioned, access-controlled dataset
3	Estimate model parameters on training interval	Parameter file and calibration script
4	Freeze a separate validation interval	Held-out dataset
5	Compare predicted and observed signals over aligned timestamps	Error metrics, plots, and uncertainty
6	Measure end-to-end event path at a fixed event rate and payload size	Latency percentiles, loss and duplicate counts
7	Repeat under database, broker, and network interruption	Recovery and data integrity report
8	Conduct operator walkthroughs with simulated data labels	Task completion and usability feedback

APPENDIX G COURSE OUTCOME MAPPING

The following outcomes and mapping are reproduced from the supplied institutional thesis format. They should be confirmed against the department's current submission rules before final print.

G.1 Course outcomes

Code	Outcome
CO1	Identify and define a complex engineering problem with clear project objectives and scope, considering societal and environmental aspects.
CO2	Apply engineering knowledge, modern tools, and relevant technologies to design, develop, and implement an effective solution or system.
CO3	Analyze and interpret data/results critically to evaluate the performance of the proposed solution using appropriate validation techniques.
CO4	Work effectively as an individual and as a team member, demonstrating professional ethics, communication skills, and project management abilities.
CO5	Prepare and deliver technical documentation and oral presentations with clarity and precision, adhering to engineering standards and practices.
CO6	Reflect on lifelong learning by identifying gaps in knowledge, exploring recent advancements, and proposing further improvements in the project.

G.2 CO to PO and PSO matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	2	3	2	2	1	3	2	1	2	2	2	2	2
CO2	3	2	3	2	3	2	1	2	1	2	2	3	3
CO3	2	3	2	3	3	2	1	2	2	2	2	3	3
CO4	1	1	1	1	1	2	3	3	3	3	2	1	1
CO5	1	1	1	1	1	1	2	2	3	2	2	1	2
CO6	2	2	2	2	3	2	2	1	2	2	3	3	3
Average	2	2	2	2	2	2	2	2	2	2	2	2	2

G.3 Outcome contribution

The project supports engineering knowledge through a software model and typed contracts; problem analysis through the separation of machine and process concerns; design through a layered platform; investigation through simulation and verification cases; tool use through Python, TypeScript, MQTT, MongoDB, containers, and API tooling; ethical engineering through explicit limits on control and data claims; teamwork through separable interfaces; communication through this thesis and system diagrams; project management through phased delivery; and lifelong learning through a defined plan for calibration, security, and measurement.

APPENDIX H EDITABLE SUBMISSION DETAILS

H.1 Student and guide details

Student names, roll numbers, enrollment numbers, email addresses, contact numbers, guide identity, industry guide identity, and designations are intentionally blank for manual completion. Insert the current approved details and verify spelling and institutional format.

H.2 Project photographs

Insert current photographs in the boxes below. Use only images approved for submission. Suggested evidence includes a project demonstration view, team photograph, and guide photograph. Do not use the dashboard screenshot as a substitute for a required student or guide photograph.

Project or team photograph	Insert approved photograph here
Student photograph 1	Insert photograph and name manually
Student photograph 2	Insert photograph and name manually
Student photograph 3	Insert photograph and name manually
Student photograph 4	Insert photograph and name manually
Student photograph 5	Insert photograph and name manually

H.3 Guide photograph

Guide photograph	Insert approved guide photograph here
Project demonstration photograph	Insert approved project photograph here

H.4 Certificate and publication evidence

Attach patent, copyright, paper acceptance, presentation, or publication certificates only if they exist and are approved for inclusion. No patent or publication evidence was present in the supplied source material.

H.5 Plagiarism report

Insert the institute-approved plagiarism or similarity report here after the required review. This thesis draft does not claim a similarity score.

H.6 Poster

Insert the final A3 project poster here after completing the institution's layout and print requirements.

APPENDIX I MEASUREMENT RECORD TEMPLATES

I.1 Event timing log

Run ID	Machine ID	Event ID	Generated UTC	Received UTC	Delivery latency ms	Processing latency ms	Notes
To be recorded							

I.2 Model calibration record

Model version	Station	Parameter	Unit	Estimate	Source interval	Validation metric	Reviewer
To be completed							

I.3 Scenario experiment record

Scenario ID	Snapshot ID	Seed	Horizon	Override	Baseline output	Counterfactual output	Quality delta	Energy delta
To be recorded								

I.4 Deployment observation record

Software revision	Environment	Services healthy	Test duration	Event rate	p50 latency	p95 latency	Recovery result
To be completed							

I.5 Evidence discipline

For each new experiment, retain the raw observation, timestamping method, software revision, configuration, seed, and analysis script. Clearly separate values read from the physical line, values calculated from current state, and values generated by a simulator. Report missing and skipped observations; do not fill gaps with assumed values. A derived capacity should always state its assumptions and formula.

APPENDIX J SELECTED SOURCE LISTINGS

The selected source listings are included to support traceability of the core implementation. They reproduce repository files and are not a replacement for the version-controlled project. Paths identify their location in the supplied project tree. Source listing page breaks are used for readability.

Source listing: dashboard/backend/domain/contracts.py

```
0001 """Versioned data contracts for the Digital Twin Orchestration Platform.
0002
0003 These models intentionally do not depend on FastAPI, MQTT, MongoDB, or the
0004 legacy dashboard models.  They are the source of truth for the event pipeline
0005 introduced in Phase 1 and will be reused by the Live Twin, Process Twin, and
0006 Scenario Worker phases.
0007 """
0008
0009 from __future__ import annotations
0010
0011 from datetime import datetime, timezone
0012 from enum import Enum
0013 from hashlib import sha256
0014 from typing import Any, Literal
0015 from uuid import UUID, uuid4, uuid5
0016
0017 from pydantic import BaseModel, ConfigDict, Field, field_validator
0018
0019
0020 SCHEMA_VERSION = "1.0"
0021 EVENT_NAMESPACE = UUID("ad74f6e5-25e4-5c3f-b8c5-fa9bc2bd161c")
0022
0023
0024 def utc_now() -> datetime:
0025     """Return an aware UTC timestamp.
0026
0027     Keeping this in one place makes it simple for later services to inject a
0028     test clock instead of generating untraceable timestamps.
0029     """
0030
0031     return datetime.now(timezone.utc)
0032
0033
0034 class StationType(str, Enum):
0035     STAMPING = "STAMPING"
0036     CNC = "CNC"
0037     WELDING = "WELDING"
0038     INSPECTION = "INSPECTION"
0039     PACKAGING = "PACKAGING"
0040
0041
0042 class OperationalState(str, Enum):
0043     IDLE = "IDLE"
0044     SETUP = "SETUP"
0045     LOADING = "LOADING"
0046     RUNNING = "RUNNING"
0047     TOOL_CHANGE = "TOOL_CHANGE"
0048     FAULT = "FAULT"
0049     MAINTENANCE = "MAINTENANCE"
0050     OFFLINE = "OFFLINE"
0051
0052
0053 class MachineStatus(str, Enum):
0054     RUNNING = "RUNNING"
0055     IDLE = "IDLE"
0056     WARNING = "WARNING"
0057     DEGRADED = "DEGRADED"
0058     FAULT = "FAULT"
0059     MAINTENANCE = "MAINTENANCE"
0060     OFFLINE = "OFFLINE"
0061
0062
0063 class EventKind(str, Enum):
0064     TELEMETRY = "telemetry"
```

```
    STATE = "state"
    FAULT = "fault"
    PRODUCTION = "production"
    COMMAND = "command"
    SUMMARY = "summary"


class FaultSeverity(str, Enum):
    INFO = "Info"
    WARNING = "Warning"
    CRITICAL = "Critical"


class CommandName(str, Enum):
    START = "start"
    STOP = "stop"
    RESET = "reset"
    BEGIN_MAINTENANCE = "begin_maintenance"
    COMPLETE_MAINTENANCE = "complete_maintenance"
    TOOL_CHANGE = "tool_change"
    INJECT_FAULT = "inject_fault"
    CLEAR_FAULT = "clear_fault"


class StrictModel(BaseModel):
    """Reject silent contract drift at system boundaries."""

    model_config = ConfigDict(extra="forbid", use_enum_values=False)


class CommandRequest(StrictModel):
    command: CommandName
    issued_at: datetime = Field(default_factory=utc_now)
    requested_by: str = Field(default="operator", min_length=1, max_length=120)
    fault_type: str | None = Field(default=None, max_length=120)
    severity: FaultSeverity | None = None
    parameters: dict[str, Any] = Field(default_factory=dict)

    @field_validator("issued_at")
    @classmethod
    def issued_at_must_be_aware(cls, value: datetime) -> datetime:
        if value.tzinfo is None or value.utcoffset() is None:
            raise ValueError("issued_at must include a UTC offset")
        return value.astimezone(timezone.utc)


class TelemetryPayload(StrictModel):
    """A causal station observation, before Live-Twin persistence.

    The report calls for target and actual RPM to be distinct for CNC.  Fields
    unused by other station types remain `None`; this keeps a stable platform
    envelope while avoiding fictional readings.
    """

    machine_id: str = Field(min_length=1, max_length=80)
    station: StationType
    status: MachineStatus
    operational_state: OperationalState
    simulated_at: datetime
    temperature_c: float = Field(ge=-50, le=500)
    vibration_mm_s: float = Field(ge=0, le=200)
    current_a: float = Field(ge=0, le=10000)
    voltage_v: float | None = Field(default=None, ge=0, le=10000)
    force_kn: float | None = Field(default=None, ge=0, le=100000)
```

```
    hydraulic_pressure_bar: float | None = Field(default=None, ge=0, le=100000)
    pneumatic_pressure_bar: float | None = Field(default=None, ge=0, le=100000)
    target_rpm: float | None = Field(default=None, ge=0, le=100000)
    actual_rpm: float | None = Field(default=None, ge=0, le=100000)
    cycle_time_s: float = Field(gt=0, le=36000)
    energy_kwh: float = Field(ge=0)
    production_count: int = Field(ge=0)
    wip_count: int = Field(default=0, ge=0)
    is_starved: bool = Field(default=False)
    is_blocked: bool = Field(default=False)
    health_score: float = Field(ge=0, le=100)
    anomaly_score: float = Field(ge=0, le=1)
    quality_score: float = Field(ge=0, le=1)
    tool_wear_percent: float | None = Field(default=None, ge=0, le=100)
    bearing_degradation_percent: float | None = Field(default=None, ge=0, le=100)
    fault_code: str | None = Field(default=None, max_length=120)
    source: Literal["simulator", "adapter"] = "simulator"

    @field_validator("simulated_at")
    @classmethod
    def simulated_at_must_be_aware(cls, value: datetime) -> datetime:
        if value.tzinfo is None or value.utcoffset() is None:
            raise ValueError("simulated_at must include a UTC offset")
        return value.astimezone(timezone.utc)


def event_id_for(run_id: str, machine_id: str, kind: EventKind, sequence: int) -> str:
    """Generate an idempotency key that is stable during deterministic replay."""

    if not run_id or not machine_id or sequence < 1:
        raise ValueError("run_id, machine_id, and positive sequence are required")
    return str(uuid5(EVENT_NAMESPACE, f"{run_id}:{machine_id}:{kind.value}:{sequence}"))


class EventEnvelope(StrictModel):
    """Transport-independent envelope used for MQTT and persistence.

    `received_at` is deliberately unset at generation.  The Live Twin fills it
    when an event is actually consumed, producing measurable—not invented—
    delivery and processing latency.
    """

    schema_version: str = SCHEMA_VERSION
    event_id: str = Field(min_length=36, max_length=36)
    run_id: str = Field(min_length=1, max_length=120)
    machine_id: str = Field(min_length=1, max_length=80)
    kind: EventKind
    sequence: int = Field(ge=1)
    generated_at: datetime
    received_at: datetime | None = None
    correlation_id: str | None = Field(default=None, max_length=120)
    payload: dict[str, Any]

    @field_validator("generated_at", "received_at")
    @classmethod
    def timestamps_must_be_aware(cls, value: datetime | None) -> datetime | None:
        if value is None:
            return value
        if value.tzinfo is None or value.utcoffset() is None:
            raise ValueError("timestamps must include a UTC offset")
        return value.astimezone(timezone.utc)

    @property
    def payload_fingerprint(self) -> str:
```

```
        """A useful diagnostic hash for idempotency collision investigation."""

        serialized = self.model_dump_json(exclude={"received_at"}, by_alias=True)
        return sha256(serialized.encode("utf-8")).hexdigest()


class IdentityState(StrictModel):
    machine_id: str = Field(min_length=1, max_length=80)
    station: StationType
    station_index: int = Field(ge=1, le=999)
    schema_version: str = SCHEMA_VERSION


class OperationalStateSnapshot(StrictModel):
    state: OperationalState
    status: MachineStatus
    updated_at: datetime


class HealthState(StrictModel):
    score: float = Field(ge=0, le=100)
    anomaly_score: float = Field(ge=0, le=1)
    tool_wear_percent: float | None = Field(default=None, ge=0, le=100)
    bearing_degradation_percent: float | None = Field(default=None, ge=0, le=100)


class ProductionState(StrictModel):
    cumulative_output: int = Field(ge=0)
    wip_count: int = Field(default=0, ge=0)
    is_starved: bool = Field(default=False)
    latest_quality_score: float = Field(ge=0, le=1)
    latest_token_id: str | None = None


class FaultState(StrictModel):
    active: bool = False
    code: str | None = None
    severity: FaultSeverity | None = None
    updated_at: datetime | None = None


class DesiredState(StrictModel):
    pending_command: CommandName | None = None
    command_requested_at: datetime | None = None


class SynchronizationState(StrictModel):
    """Observed timestamps and timing values maintained by a Live Twin.

    Values are optional until the first event is consumed.  A measurement is
    only set from observed clocks; the application never fills it with a random
    placeholder merely to make a dashboard tile look populated.
    """

    latest_sequence: int = Field(ge=0)
    last_generated_at: datetime | None = None
    last_received_at: datetime | None = None
    delivery_latency_ms: float | None = Field(default=None, ge=0)
    processing_latency_ms: float | None = Field(default=None, ge=0)
    source_age_ms: float | None = Field(default=None, ge=0)


class TwinDocument(StrictModel):
    """The one-document-per-machine shape intended for MongoDB `twins`."""
```

```
0257
0258     identity: IdentityState
0259     operational: OperationalStateSnapshot
0260     telemetry: TelemetryPayload
0261     health: HealthState
0262     production: ProductionState
0263     faults: FaultState = Field(default_factory=FaultState)
0264     desired: DesiredState = Field(default_factory=DesiredState)
0265     synchronization: SynchronizationState
0266     last_event_id: str = Field(min_length=36, max_length=36)
0267     revision: int = Field(default=1, ge=1)
0268     updated_at: datetime
0269
0270
0271 class ProductionToken(StrictModel):
0272     """Minimal material-flow contract for the Phase 3 Process Twin."""
0273
0274     token_id: str = Field(min_length=1, max_length=120)
0275     part_type: str = Field(default="automotive_component", min_length=1, max_length=120)
0276     quality: Literal["pending", "pass", "fail"] = "pending"
0277     source_station: StationType
0278     emitted_at: datetime
0279     carried_flags: dict[str, float | bool | str] = Field(default_factory=dict)
0280     lineage: list[str] = Field(default_factory=list)
0281
0282
0283 class FrozenLineSnapshot(StrictModel):
0284     """An immutable-at-creation view of all station Live Twins.
0285
0286     The Process Twin and Scenario Workers use this rather than separately
0287     fetching live machine documents, preventing a mixed-time line view.
0288     """
0289
0290     snapshot_id: str = Field(default_factory=lambda: str(uuid4()), min_length=36, max_length=36)
0291     captured_at: datetime = Field(default_factory=utc_now)
0292     topology_version: str = "five-station-v1"
0293     twins: list[TwinDocument] = Field(min_length=1)
0294
0295
0296 class ProcessMetrics(StrictModel):
0297     bottleneck_machine_id: str
0298     bottleneck_station: StationType
0299     bottleneck_cycle_time_s: float = Field(gt=0)
0300     line_capacity_per_hour: float = Field(ge=0)
0301     expected_line_quality: float = Field(ge=0, le=1)
0302     available_stations: int = Field(ge=0)
0303     total_stations: int = Field(ge=1)
0304     snapshot_max_source_age_ms: float | None = Field(default=None, ge=0)
0305     notes: list[str] = Field(default_factory=list)
0306
0307
0308 class ProcessTwinView(StrictModel):
0309     snapshot: FrozenLineSnapshot
0310     metrics: ProcessMetrics
0311
0312
0313 class DAGNode(StrictModel):
0314     machine_id: str = Field(min_length=1, max_length=80)
0315     station: StationType
0316     station_index: int = Field(ge=1, le=999)
0317     nominal_cycle_time_s: float = Field(gt=0)
0318     upstream_machine_ids: list[str] = Field(default_factory=list)
0319     downstream_machine_ids: list[str] = Field(default_factory=list)
0320
```

```
0321
0322 class DAGEdge(StrictModel):
0323     from_machine_id: str = Field(min_length=1, max_length=80)
0324     to_machine_id: str = Field(min_length=1, max_length=80)
0325     transition_type: str = "material_flow"
0326
0327
0328 class ProcessTopology(StrictModel):
0329     version: str = "five-station-v1"
0330     nodes: list[DAGNode]
0331     edges: list[DAGEdge]
0332     order: list[str]
0333
0334
0335 class StationBottleneckDetail(StrictModel):
0336     machine_id: str
0337     station: StationType
0338     cycle_time_s: float
0339     effective_cycle_time_s: float
0340     slack_time_s: float
0341     is_bottleneck: bool
0342     status: MachineStatus
0343
0344
0345 class BottleneckReport(StrictModel):
0346     binding_machine_id: str
0347     binding_station: StationType
0348     binding_cycle_time_s: float
0349     max_line_capacity_per_hour: float
0350     stations: list[StationBottleneckDetail]
0351     captured_at: datetime
0352
0353
0354 class StationDefectAttribution(StrictModel):
0355     machine_id: str
0356     station: StationType
0357     attribution_percent: float = Field(ge=0, le=100)
0358     sample_count: int = Field(ge=0)
0359     average_quality_score: float = Field(ge=0, le=1)
0360     reason: str
0361
0362
0363 class DefectAttributionReport(StrictModel):
0364     total_tokens_evaluated: int = Field(ge=0)
0365     defective_tokens_count: int = Field(ge=0)
0366     defect_rate_percent: float = Field(ge=0, le=100)
0367     primary_root_cause_station: StationType | None = None
0368     primary_root_cause_machine_id: str | None = None
0369     attributions: list[StationDefectAttribution] = Field(default_factory=list)
0370     summary_analysis: str
0371     evaluated_at: datetime
0372
0373
0374 # ---------------------------------------------------------------------------
0375 # Phase 4 — Scenario Worker Contracts (§4.3, §4.4)
0376 # ---------------------------------------------------------------------------
0377
0378 class ScenarioStatus(str, Enum):
0379     PENDING = "pending"
0380     RUNNING = "running"
0381     COMPLETED = "completed"
0382     FAILED = "failed"
0383
0384
```

```python
class StationOverride(StrictModel):
    """Counterfactual parameter override or command applied to a single station."""

    machine_id: str = Field(min_length=1, max_length=80)
    command: CommandName | None = None
    fault_type: str | None = Field(default=None, max_length=120)
    fault_severity: FaultSeverity | None = None
    tool_wear_percent: float | None = Field(default=None, ge=0, le=100)
    bearing_degradation_percent: float | None = Field(default=None, ge=0, le=100)
    load_factor: float | None = Field(default=None, ge=0.1, le=1.0)
    cycle_time_multiplier: float | None = Field(default=None, ge=0.2, le=5.0)
    target_rpm: float | None = Field(default=None, ge=0)


class ScenarioForkRequest(StrictModel):
    """Request payload for an isolated forward counterfactual simulation."""

    name: str = Field(default="Counterfactual Scenario", min_length=1, max_length=120)
    description: str | None = Field(default=None, max_length=500)
    snapshot_id: str | None = Field(default=None, min_length=36, max_length=36)
    seed: int = Field(default=20260920)
    horizon_seconds: float = Field(default=120.0, ge=5.0, le=3600.0)
    time_step_s: float = Field(default=1.0, ge=0.1, le=10.0)
    overrides: list[StationOverride] = Field(default_factory=list)


class StationOutcome(StrictModel):
    """Summary metrics for one station over the forward simulation horizon."""

    machine_id: str
    station: StationType
    final_state: OperationalState
    final_status: MachineStatus
    final_health_score: float = Field(ge=0, le=100)
    parts_produced: int = Field(ge=0)
    energy_kwh: float = Field(ge=0)
    avg_cycle_time_s: float = Field(ge=0)
    avg_quality_score: float = Field(ge=0, le=1)
    fault_occurred: bool = False


class LineOutcome(StrictModel):
    """Line-level outcome across the entire five-station DAG."""

    total_output: int = Field(ge=0)
    scrap_count: int = Field(ge=0)
    overall_quality: float = Field(ge=0, le=1)
    total_energy_kwh: float = Field(ge=0)
    bottleneck_machine_id: str
    binding_cycle_time_s: float = Field(gt=0)
    hourly_capacity: float = Field(ge=0)
    stations: list[StationOutcome] = Field(min_length=1)


class ScenarioDelta(StrictModel):
    """Comparative causal delta between Counterfactual and Baseline."""

    output_delta: int
    scrap_delta: int
    quality_delta: float
    energy_delta_kwh: float
    health_delta: float
    bottleneck_shifted: bool
    baseline_bottleneck: str
```

```
    counterfactual_bottleneck: str
    summary: str


class TrajectorySample(StrictModel):
    """Sampled forward time point for dashboard charts."""

    simulated_seconds: float
    baseline_output: int
    counterfactual_output: int
    baseline_quality: float
    counterfactual_quality: float
    baseline_cnc_health: float
    counterfactual_cnc_health: float




class DecisionStrategy(StrictModel):
    strategy_id: str
    name: str
    strategy_type: str  # fix_now, derate_finish, status_quo, custom
    action_label: str
    projected_output: int
    scrap_count: int
    downtime_minutes: float
    total_cost_usd: float
    scrap_cost_usd: float
    downtime_cost_usd: float
    tooling_cost_usd: float
    risk_level: str  # LOW, MEDIUM, HIGH, CRITICAL
    quota_met: bool
    is_recommended: bool
    recommendation_reason: str


class WorkerWorkOrder(StrictModel):
    work_order_id: str
    target_machine_id: str
    station_name: str
    priority: str  # URGENT, SCHEDULED, DEFERRED
    estimated_duration_min: int
    optimal_window_description: str
    required_tools_and_parts: list[str]
    safety_loto_steps: list[str]
    sop_procedure_steps: list[str]
    post_maintenance_verification: list[str]


class ManagerExecutiveBrief(StrictModel):
    recommended_strategy_name: str
    recommended_action: str
    financial_advantage_usd: float
    shift_quota_target: int
    shift_quota_feasible: bool
    risk_assessment: str
    key_tradeoff_summary: str
    ai_reasoning_narrative: str


class ScenarioDecisionAnalysis(StrictModel):
    scenario_id: str
    analyzed_at: datetime = Field(default_factory=utc_now)
    target_machine_id: str
```

```
    current_health_percent: float
    active_anomaly: str
    manager_brief: ManagerExecutiveBrief
    worker_order: WorkerWorkOrder
    strategies: list[DecisionStrategy]
    llm_prompt_template: str | None = None

class ScenarioRunResult(StrictModel):
    """Persisted scenario execution record in MongoDB `scenarios`."""

    scenario_id: str = Field(default_factory=lambda: str(uuid4()), min_length=36, max_length=36)
    name: str
    description: str | None = None
    status: ScenarioStatus = ScenarioStatus.COMPLETED
    snapshot_id: str = Field(min_length=36, max_length=36)
    seed: int
    horizon_seconds: float
    time_step_s: float
    created_at: datetime = Field(default_factory=utc_now)
    completed_at: datetime = Field(default_factory=utc_now)
    overrides: list[StationOverride] = Field(default_factory=list)
    baseline: LineOutcome
    counterfactual: LineOutcome
    comparison: ScenarioDelta
    trajectory: list[TrajectorySample] = Field(default_factory=list)
    decision_analysis: ScenarioDecisionAnalysis | None = None


```

Source listing: dashboard/backend/simulation/causal.py

```
0001 """Causal, deterministic reduced-order models for the five-station line.
0002
0003 This module is intentionally pure Python.  It does not open sockets, access a
0004 database, or know about FastAPI.  That separation lets the same models run in a
0005 Live-Twin container, an isolated Scenario Worker, or a reproducible unit test.
0006
0007 The values are *calibration-ready parameters*, not claims of physical
0008 validation.  Phase 7 will fit and record their values against the public data
0009 sets specified in the research report.
0010 """
0011
0012 from __future__ import annotations
0013
0014 from dataclasses import dataclass
0015 from datetime import datetime, timedelta, timezone
0016 import math
0017 import random
0018 from typing import Iterable
0019
0020 from domain.contracts import (
0021     CommandName,
0022     CommandRequest,
0023     EventEnvelope,
0024     EventKind,
0025     FaultSeverity,
0026     FrozenLineSnapshot,
0027     MachineStatus,
0028     OperationalState,
0029     StationOverride,
0030     StationType,
0031     TelemetryPayload,
0032     TwinDocument,
0033     event_id_for,
0034 )
0035
0036
0037 SIMULATION_EPOCH = datetime(2026, 1, 1, tzinfo=timezone.utc)
0038
0039
0040 def clamp(value: float, lower: float, upper: float) -> float:
0041     return max(lower, min(upper, value))
0042
0043
0044 @dataclass(frozen=True)
0045 class StationConfig:
0046     machine_id: str
0047     station: StationType
0048     station_index: int
0049     cycle_time_s: float
0050     nominal_rpm: float | None = None
0051
0052
0053 @dataclass(frozen=True)
0054 class SimulationTick:
0055     """Everything generated in one deterministic factory time step."""
0056
0057     simulated_at: datetime
0058     telemetry: tuple[TelemetryPayload, ...]
0059     events: tuple[EventEnvelope, ...]
0060
0061
0062 DEFAULT_STATIONS: tuple[StationConfig, ...] = (
0063         StationConfig("STAMPING-01", StationType.STAMPING, 1, 8.5, nominal_rpm=1200.0),
0064     StationConfig("CNC-01", StationType.CNC, 2, 42.0, nominal_rpm=3400.0),
```

```
    StationConfig("WELDING-01", StationType.WELDING, 3, 15.0),
    StationConfig("INSPECTION-01", StationType.INSPECTION, 4, 6.0),
    StationConfig("PACKAGING-01", StationType.PACKAGING, 5, 10.0, nominal_rpm=800.0),
)


class CausalStation:
    """Base class for all station models.

    A station owns state evolution, operational state, command transitions, and
    an independent pseudo-random stream so that one station's seed replay does
    not perturb the others.
    """

    def __init__(self, config: StationConfig, *, seed: int) -> None:
        self.config = config
        self._rng = random.Random(seed)
        self.operational_state = OperationalState.RUNNING
        self._fault_type: str | None = None
        self._fault_severity: FaultSeverity | None = None
        self._fault_elapsed_s = 0.0
        self._load_factor = 0.55
        self._cycle_elapsed_s = 0.0
        self.production_count = 0
        self.energy_kwh = 0.0
        self._last_status = MachineStatus.RUNNING
        self.cycle_time_multiplier = 1.0
        self._upstream_station: CausalStation | None = None
        self._downstream_station: CausalStation | None = None
        self._max_buffer: int = 5

    def set_upstream_station(self, upstream: CausalStation | None) -> None:
        self._upstream_station = upstream

    def set_downstream_station(self, downstream: CausalStation | None, *, max_buffer: int = 5) -> None:
        self._downstream_station = downstream
        self._max_buffer = max_buffer

    @property
    def machine_id(self) -> str:
        return self.config.machine_id

    @property
    def station(self) -> StationType:
        return self.config.station

    @property
    def fault_type(self) -> str | None:
        return self._fault_type

    @property
    def fault_active(self) -> bool:
        return self._fault_type is not None

    def apply_command(self, command: CommandRequest) -> None:
        """Apply a validated operator/scenario command to the local state.

        The future Live Twin will persist command intent before calling this;
        this pure model only performs the state transition.
        """

        name = command.command
        if name == CommandName.START:
            if self.operational_state in {OperationalState.IDLE, OperationalState.OFFLINE}:
```

```
0129                 self.operational_state = OperationalState.RUNNING
0130         elif name == CommandName.STOP:
0131             self.operational_state = OperationalState.IDLE
0132         elif name == CommandName.RESET:
0133             self.reset()
0134         elif name == CommandName.BEGIN_MAINTENANCE:
0135             self.operational_state = OperationalState.MAINTENANCE
0136         elif name == CommandName.COMPLETE_MAINTENANCE:
0137             self._complete_maintenance()
0138             self.operational_state = OperationalState.RUNNING
0139         elif name == CommandName.TOOL_CHANGE:
0140             self._tool_change()
0141             self.operational_state = OperationalState.RUNNING
0142         elif name == CommandName.INJECT_FAULT:
0143             self._fault_type = command.fault_type or "injected_fault"
0144             self._fault_severity = command.severity or FaultSeverity.WARNING
0145             self._fault_elapsed_s = 0.0
0146         elif name == CommandName.CLEAR_FAULT:
0147             self._clear_fault()
0148 
0149     def reset(self) -> None:
0150         """Return generic state to nominal; subclasses reset degradation too."""
0151 
0152         self.operational_state = OperationalState.RUNNING
0153         self._fault_type = None
0154         self._fault_severity = None
0155         self._fault_elapsed_s = 0.0
0156         self._load_factor = 0.55
0157         self._cycle_elapsed_s = 0.0
0158         self.production_count = 0
0159         self.energy_kwh = 0.0
0160         self._last_status = MachineStatus.RUNNING
0161         self.cycle_time_multiplier = 1.0
0162         self._reset_degradation()
0163 
0164     def rehydrate_from_twin(self, twin: TwinDocument) -> None:
0165         """Restore internal station state from a FrozenLineSnapshot TwinDocument."""
0166         self.operational_state = twin.operational.state
0167         self._last_status = twin.operational.status
0168         self.production_count = twin.production.cumulative_output
0169         self.energy_kwh = twin.telemetry.energy_kwh
0170         self._fault_type = twin.faults.code if twin.faults.active else (
0171             twin.telemetry.fault_code or ("injected_fault" if twin.operational.state ==
     OperationalState.FAULT else None)
0172         )
0173         self._fault_severity = twin.faults.severity
0174         if self._fault_type is not None:
0175             if self._fault_severity is None:
0176                 if twin.operational.status == MachineStatus.FAULT or twin.operational.state ==
     OperationalState.FAULT:
0177                     self._fault_severity = FaultSeverity.CRITICAL
0178                 elif twin.operational.status == MachineStatus.DEGRADED:
0179                     self._fault_severity = FaultSeverity.WARNING
0180                 else:
0181                     self._fault_severity = FaultSeverity.INFO
0182             severity_scale = 1.0 if self._fault_severity == FaultSeverity.CRITICAL else 0.62
0183             if self._fault_severity == FaultSeverity.INFO:
0184                 severity_scale = 0.25
0185             if twin.operational.status == MachineStatus.FAULT or twin.operational.state ==
     OperationalState.FAULT:
0186                 target_fault = 1.0
0187             elif twin.operational.status == MachineStatus.DEGRADED:
0188                 target_fault = 0.50
0189             elif twin.operational.status == MachineStatus.WARNING:
```

```
                target_fault = 0.20
            else:
                target_fault = 0.05
            self._fault_elapsed_s = (target_fault * 60.0) / max(0.01, severity_scale)
        else:
            self._fault_elapsed_s = 0.0
        self._rehydrate_station_degradation(twin)

    def _rehydrate_station_degradation(self, twin: TwinDocument) -> None:
        pass

    def apply_override(self, override: StationOverride) -> None:
        """Apply a counterfactual override to this station for scenario forking."""
        if override.command is not None:
            self.apply_command(
                CommandRequest(
                    command=override.command,
                    fault_type=override.fault_type,
                    severity=override.fault_severity,
                )
            )
        elif override.fault_type is not None:
            self.apply_command(
                CommandRequest(
                    command=CommandName.INJECT_FAULT,
                    fault_type=override.fault_type,
                    severity=override.fault_severity or FaultSeverity.WARNING,
                )
            )
        if override.load_factor is not None:
            self._load_factor = clamp(override.load_factor, 0.1, 1.0)
        if override.cycle_time_multiplier is not None:
            self.cycle_time_multiplier = clamp(override.cycle_time_multiplier, 0.2, 5.0)
        self._apply_station_override(override)

    def _apply_station_override(self, override: StationOverride) -> None:
        pass

    @property
    def _is_actively_producing(self) -> bool:
        """True when the station is in a state that performs physical work.

        FAULT machines are still physically operating under degraded conditions
        (generating heat, forces, consuming power, wearing tools) — they just
        aren't counted as producing good parts by _advance_production.
        """
        return self.operational_state in {OperationalState.RUNNING, OperationalState.FAULT}

    def _calculate_idle_values(self, dt_s: float) -> dict[str, float | None]:
        """Return near-zero ambient readings for a stopped/idle machine.

        Stopped machines still have ambient temperature and standby current
        but produce zero cutting force, zero RPM, and no tool wear.
        """
        fault = self._fault_factor()
        # Ambient temperature with very slow drift toward room temp
        ambient_temp = 25.0 + self._rng.gauss(0.0, 0.08)
        # Standby current (electronics/PLC only)
        standby_current = 0.12 + self._rng.gauss(0.0, 0.01)
        # Residual vibration from building/adjacent machines
        residual_vibration = 0.05 + self._rng.gauss(0.0, 0.01)

        return {
            "fault_factor": fault,
```

```
            "temperature_c": ambient_temp,
            "vibration_mm_s": max(0.0, residual_vibration),
            "current_a": max(0.0, standby_current),
            "voltage_v": None,
            "force_kn": 0.0,
            "hydraulic_pressure_bar": None,
            "pneumatic_pressure_bar": None,
            "target_rpm": 0.0,
            "actual_rpm": 0.0,
            "cycle_time_s": self.config.cycle_time_s * self.cycle_time_multiplier,
            "health_score": self._idle_health_score(),
            "anomaly_score": clamp(0.01 + 0.55 * fault, 0.0, 1.0),
            "quality_score": 1.0,  # Not producing, so no quality to degrade
            "tool_wear_percent": self._current_tool_wear_percent(),
            "bearing_degradation_percent": self._current_bearing_degradation_percent(),
        }

    def _idle_health_score(self) -> float:
        """Health score when idle — subclasses can override for station-specific health."""
        return 100.0

    def _current_tool_wear_percent(self) -> float | None:
        """Return current tool wear; subclasses override."""
        return None

    def _current_bearing_degradation_percent(self) -> float | None:
        """Return current bearing degradation; subclasses override."""
        return None

    def advance(self, *, dt_s: float, simulated_at: datetime) -> TelemetryPayload:
        """Advance one station by deterministic simulation time.

        `simulated_at` comes from the factory clock rather than wall time.  It
        allows a scenario to replay the exact same trajectory from a snapshot.
        """

        if dt_s <= 0:
            raise ValueError("dt_s must be positive")
        if simulated_at.tzinfo is None or simulated_at.utcoffset() is None:
            raise ValueError("simulated_at must be timezone-aware")

        self._advance_operational_state(dt_s)

        if self._is_actively_producing:
            # Full physics: load evolution, causal values, production, energy
            self._advance_load(dt_s)
            values = self._calculate_values(dt_s)
            self._advance_production(dt_s, values["cycle_time_s"])
            self.energy_kwh += self._energy_increment_kwh(values["current_a"], dt_s)
        else:
            # Idle/stopped: ambient readings only, no wear, no production, standby energy
            values = self._calculate_idle_values(dt_s)
            standby_current = values["current_a"] or 0.12
            self.energy_kwh += self._energy_increment_kwh(standby_current, dt_s)

        status = self._status_for_fault_factor(values["fault_factor"])
        self._last_status = status

        wip_count = max(0, self._upstream_station.production_count - self.production_count) if
    self._upstream_station else 0
        is_starved = bool(self._upstream_station and self.production_count >=
    self._upstream_station.production_count)
        is_blocked = bool(self._downstream_station and (self.production_count -
    self._downstream_station.production_count >= self._max_buffer))
```

```
0315
0316        return TelemetryPayload(
0317            machine_id=self.machine_id,
0318            station=self.station,
0319            status=status,
0320            operational_state=self.operational_state,
0321            simulated_at=simulated_at.astimezone(timezone.utc),
0322            temperature_c=round(values["temperature_c"], 3),
0323            vibration_mm_s=round(values["vibration_mm_s"], 3),
0324            current_a=round(values["current_a"], 3),
0325            voltage_v=self._round_or_none(values.get("voltage_v")),
0326            force_kn=self._round_or_none(values.get("force_kn")),
0327            hydraulic_pressure_bar=self._round_or_none(values.get("hydraulic_pressure_bar")),
0328            pneumatic_pressure_bar=self._round_or_none(values.get("pneumatic_pressure_bar")),
0329            target_rpm=self._round_or_none(values.get("target_rpm")),
0330            actual_rpm=self._round_or_none(values.get("actual_rpm")),
0331            cycle_time_s=round(values["cycle_time_s"], 3),
0332            energy_kwh=round(self.energy_kwh, 6),
0333            production_count=self.production_count,
0334            wip_count=wip_count,
0335            is_starved=is_starved,
0336            is_blocked=is_blocked,
0337            health_score=round(values["health_score"], 3),
0338            anomaly_score=round(values["anomaly_score"], 4),
0339            quality_score=round(values["quality_score"], 5),
0340            tool_wear_percent=self._round_or_none(values.get("tool_wear_percent")),
0341            bearing_degradation_percent=self._round_or_none(values.get("bearing_degradation_percent")),
0342            fault_code=self._fault_type,
0343        )
0344
0345    def snapshot(self) -> dict[str, object]:
0346        """Return only serialisable state needed for later scenario forks."""
0347
0348        return {
0349            "machine_id": self.machine_id,
0350            "station": self.station.value,
0351            "operational_state": self.operational_state.value,
0352            "fault_type": self._fault_type,
0353            "fault_severity": self._fault_severity.value if self._fault_severity else None,
0354            "fault_elapsed_s": self._fault_elapsed_s,
0355            "load_factor": self._load_factor,
0356            "cycle_elapsed_s": self._cycle_elapsed_s,
0357            "production_count": self.production_count,
0358            "energy_kwh": self.energy_kwh,
0359            "degradation": self._degradation_snapshot(),
0360        }
0361
0362    def _advance_operational_state(self, dt_s: float) -> None:
0363        if self.fault_active:
0364            self._fault_elapsed_s += dt_s
0365
0366    def _advance_load(self, dt_s: float) -> None:
0367        # Mean-reverting operational load plus a bounded sensor-independent
0368        # disturbance. This is deliberately a residual, not the whole model.
0369        mean_reversion = (0.55 - self._load_factor) * min(1.0, dt_s / 15.0)
0370        disturbance = self._rng.gauss(0.0, 0.025 * math.sqrt(dt_s))
0371        self._load_factor = clamp(self._load_factor + mean_reversion + disturbance, 0.25, 0.95)
0372
0373    def _fault_factor(self) -> float:
0374        if not self.fault_active:
0375            return 0.0
0376        severity_scale = 1.0 if self._fault_severity == FaultSeverity.CRITICAL else 0.62
0377        if self._fault_severity == FaultSeverity.INFO:
0378            severity_scale = 0.25
```

```
0379         return clamp((self._fault_elapsed_s / 60.0) * severity_scale, 0.0, 1.0)
0380
0381     def _status_for_fault_factor(self, fault_factor: float) -> MachineStatus:
0382         if self.operational_state == OperationalState.OFFLINE:
0383             return MachineStatus.OFFLINE
0384         if self.operational_state == OperationalState.MAINTENANCE:
0385             return MachineStatus.MAINTENANCE
0386         if self.operational_state == OperationalState.IDLE:
0387             return MachineStatus.IDLE
0388         if fault_factor >= 0.9:
0389             self.operational_state = OperationalState.FAULT
0390             return MachineStatus.FAULT
0391         if fault_factor >= 0.42:
0392             return MachineStatus.DEGRADED
0393         if fault_factor >= 0.12:
0394             return MachineStatus.WARNING
0395         return MachineStatus.RUNNING
0396
0397     def _advance_production(self, dt_s: float, cycle_time_s: float) -> None:
0398         if self.operational_state in {
0399             OperationalState.IDLE,
0400             OperationalState.MAINTENANCE,
0401             OperationalState.FAULT,
0402             OperationalState.OFFLINE,
0403         }:
0404             return
0405
0406         # Upstream material constraint (starvation):
0407         # A downstream station cannot process or finish a part if its upstream station
0408         # has not yet completed a part for it to work on.
0409         if self._upstream_station is not None:
0410             if self.production_count >= self._upstream_station.production_count:
0411                 self._cycle_elapsed_s = 0.0
0412                 return
0413
0414         # Downstream buffer constraint (blocking):
0415         # If the inter-station buffer is full, hold until downstream consumes a part.
0416         if self._downstream_station is not None:
0417             if self.production_count - self._downstream_station.production_count >= self._max_buffer:
0418                 return
0419
0420         self._cycle_elapsed_s += dt_s
0421         while self._cycle_elapsed_s >= cycle_time_s:
0422             if self._upstream_station is not None and self.production_count >=
     self._upstream_station.production_count:
0423                 self._cycle_elapsed_s = 0.0
0424                 break
0425             if self._downstream_station is not None and (self.production_count -
     self._downstream_station.production_count >= self._max_buffer):
0426                 break
0427             self.production_count += 1
0428             self._cycle_elapsed_s -= cycle_time_s
0429             if self._upstream_station is not None and self.production_count >=
     self._upstream_station.production_count:
0430                 self._cycle_elapsed_s = 0.0
0431                 break
0432
0433     @staticmethod
0434     def _energy_increment_kwh(current_a: float, dt_s: float) -> float:
0435         # 415V three-phase, 0.8 power factor.  It intentionally derives energy
0436         # from the modelled current instead of inventing a separate KPI series.
0437         return max(0.0, current_a) * 415.0 * math.sqrt(3) * 0.8 * dt_s / 3_600_000.0
0438
0439     @staticmethod
```

```
0440     def _round_or_none(value: float | None) -> float | None:
0441         return round(value, 3) if value is not None else None
0442
0443     def _clear_fault(self) -> None:
0444         self._fault_type = None
0445         self._fault_severity = None
0446         self._fault_elapsed_s = 0.0
0447         if self.operational_state == OperationalState.FAULT:
0448             self.operational_state = OperationalState.RUNNING
0449
0450     def _complete_maintenance(self) -> None:
0451         self._clear_fault()
0452         self._reset_degradation()
0453
0454     def _tool_change(self) -> None:
0455         """Subclasses override if their station has a tool-change operation."""
0456
0457     def _reset_degradation(self) -> None:
0458         """Subclasses override to reset station-specific internal variables."""
0459
0460     def _degradation_snapshot(self) -> dict[str, float]:
0461         return {}
0462
0463     def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
0464         raise NotImplementedError
0465
0466
0467 class StampingStation(CausalStation):
0468     """Die wear -> press force -> current/vibration/quality causal chain."""
0469
0470     def __init__(self, config: StationConfig, *, seed: int) -> None:
0471         super().__init__(config, seed=seed)
0472         self.die_wear = 0.04
0473
0474     def _rehydrate_station_degradation(self, twin: TwinDocument) -> None:
0475         if twin.health.tool_wear_percent is not None:
0476             self.die_wear = clamp(twin.health.tool_wear_percent / 100.0, 0.0, 1.0)
0477         else:
0478             self.die_wear = clamp((100.0 - twin.health.score) / 68.0, 0.0, 1.0)
0479
0480     def _apply_station_override(self, override: StationOverride) -> None:
0481         if override.tool_wear_percent is not None:
0482             self.die_wear = clamp(override.tool_wear_percent / 100.0, 0.0, 1.0)
0483
0484     def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
0485         fault = self._fault_factor()
0486         accelerated_wear = 1.0 + (4.0 * fault if self._fault_type in {"die_wear", "feed_jam"} else 0.0)
0487         self.die_wear = clamp(
0488             self.die_wear + 0.000018 * dt_s * (0.65 + self._load_factor) * accelerated_wear,
0489             0.0,
0490             1.0,
0491         )
0492         required_force = 110.0 * (0.72 + 0.56 * self._load_factor) * (1.0 + 0.9 * self.die_wear)
0493         current = 5.0 + 0.055 * required_force + self._rng.gauss(0.0, 0.16)
0494         hydraulic_pressure = 85.0 + 0.45 * required_force + self._rng.gauss(0.0, 0.4)
0495         vibration = 1.15 + 0.0065 * required_force + 3.2 * self.die_wear + self._rng.gauss(0.0, 0.07)
0496         temperature = 43.0 + 0.16 * current + 7.0 * self.die_wear + self._rng.gauss(0.0, 0.18)
0497         cycle_time = self.config.cycle_time_s * (1.0 + 0.22 * fault + 0.07 * self.die_wear) *
     self.cycle_time_multiplier
0498         quality = clamp(0.997 - 0.52 * self.die_wear - 0.35 * fault, 0.0, 1.0)
0499         health = clamp(100.0 - 68.0 * self.die_wear - 42.0 * fault, 0.0, 100.0)
0500         return {
0501             "fault_factor": fault,
0502             "temperature_c": temperature,
```

```
            "vibration_mm_s": max(0.0, vibration),
            "current_a": max(0.0, current),
            "force_kn": required_force,
            "hydraulic_pressure_bar": hydraulic_pressure,
            "target_rpm": self.config.nominal_rpm,
            "actual_rpm": max(0.0, (self.config.nominal_rpm or 0.0) * (1.0 - 0.08 * fault)),
            "cycle_time_s": cycle_time,
            "health_score": health,
            "anomaly_score": clamp(0.02 + 0.62 * self.die_wear + 0.65 * fault, 0.0, 1.0),
            "quality_score": quality,
            "tool_wear_percent": self.die_wear * 100.0,
            "bearing_degradation_percent": None,
        }

    def _idle_health_score(self) -> float:
        return clamp(100.0 - 68.0 * self.die_wear, 0.0, 100.0)

    def _current_tool_wear_percent(self) -> float | None:
        return self.die_wear * 100.0

    def _reset_degradation(self) -> None:
        self.die_wear = 0.04

    def _degradation_snapshot(self) -> dict[str, float]:
        return {"die_wear_percent": round(self.die_wear * 100.0, 5)}


class CncStation(CausalStation):
    """Flagship reduced-order CNC model based on the report's causal graph.

    The internal variables are deliberately separate: tool wear affects cutting
    force and quality, while bearing degradation creates a distinct vibration
    and RPM-tracking signal.  Phase 7 will replace the provisional coefficients
    with recorded calibration artifacts from PHM 2010 and NASA IMS analysis.
    """

    def __init__(self, config: StationConfig, *, seed: int) -> None:
        super().__init__(config, seed=seed)
        self.tool_wear = 0.10
        self.bearing_degradation = 0.03
        self.coolant_failure = False
        self.custom_rpm: float | None = None

    def _rehydrate_station_degradation(self, twin: TwinDocument) -> None:
        if twin.health.tool_wear_percent is not None:
            self.tool_wear = clamp(twin.health.tool_wear_percent / 100.0, 0.0, 1.0)
        if twin.health.bearing_degradation_percent is not None:
            self.bearing_degradation = clamp(twin.health.bearing_degradation_percent / 100.0, 0.0, 1.0)
        if twin.faults.active and twin.faults.code == "coolant_failure":
            self.coolant_failure = True

    def _apply_station_override(self, override: StationOverride) -> None:
        if override.tool_wear_percent is not None:
            self.tool_wear = clamp(override.tool_wear_percent / 100.0, 0.0, 1.0)
        if override.bearing_degradation_percent is not None:
            self.bearing_degradation = clamp(override.bearing_degradation_percent / 100.0, 0.0, 1.0)
        if override.target_rpm is not None:
            self.custom_rpm = override.target_rpm

    def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
        fault = self._fault_factor()
        if self._fault_type == "coolant_failure":
            self.coolant_failure = True
        tool_acceleration = 1.0 + (5.0 * fault if self._fault_type in {"tool_wear", "injected_fault"}
```

```
    else 0.0)
0567        bearing_acceleration = 1.0 + (4.0 * fault if self._fault_type == "bearing_fault" else 0.0)
0568        self.tool_wear = clamp(
0569            self.tool_wear + 0.000012 * dt_s * (0.5 + self._load_factor) * tool_acceleration,
0570            0.0,
0571            1.0,
0572        )
0573        self.bearing_degradation = clamp(
0574            self.bearing_degradation + 0.000004 * dt_s * (0.4 + self._load_factor) *
    bearing_acceleration,
0575            0.0,
0576            1.0,
0577        )
0578
0579        cutting_force = 7.2 * (0.75 + self._load_factor) * (1.0 + 0.95 * self.tool_wear)
0580        current = 3.9 + 0.60 * cutting_force + 1.8 * self.bearing_degradation + self._rng.gauss(0.0,
    0.12)
0581        coolant_penalty = 14.0 * (0.55 + fault) if self.coolant_failure else 0.0
0582        temperature = (
0583            40.0
0584            + 1.65 * current
0585            + 10.0 * self.tool_wear
0586            + 5.5 * self.bearing_degradation
0587            + coolant_penalty
0588            + self._rng.gauss(0.0, 0.22)
0589        )
0590        target_rpm = self.custom_rpm if self.custom_rpm is not None else (self.config.nominal_rpm or
    3400.0)
0591        rpm_tracking_error = 35.0 * self._load_factor + 210.0 * self.bearing_degradation + 360.0 *
    fault
0592        actual_rpm = max(200.0, target_rpm - rpm_tracking_error + self._rng.gauss(0.0, 5.0))
0593        vibration = (
0594            1.5
0595            + 0.25 * cutting_force
0596            + 5.0 * self.tool_wear
0597            + 4.0 * self.bearing_degradation
0598            + 1.5 * fault
0599            + self._rng.gauss(0.0, 0.08)
0600        )
0601        cycle_time = self.config.cycle_time_s * (
0602            1.0 + 0.14 * self.tool_wear + 0.08 * self.bearing_degradation + 0.22 * fault
0603        ) * self.cycle_time_multiplier
0604        quality = clamp(
0605            0.998 - 0.58 * (self.tool_wear ** 1.45) - 0.10 * self.bearing_degradation - 0.24 * fault,
0606            0.0,
0607            1.0,
0608        )
0609        health = clamp(
0610            100.0 - 62.0 * self.tool_wear - 26.0 * self.bearing_degradation - 20.0 * fault,
0611            0.0,
0612            100.0,
0613        )
0614        anomaly = clamp(
0615            0.015
0616            + 0.46 * self.tool_wear
0617            + 0.25 * self.bearing_degradation
0618            + 0.55 * fault
0619            + (0.10 if self.coolant_failure else 0.0),
0620            0.0,
0621            1.0,
0622        )
0623        return {
0624            "fault_factor": fault,
0625            "temperature_c": temperature,
```

```
            "vibration_mm_s": max(0.0, vibration),
            "current_a": max(0.0, current),
            "force_kn": cutting_force,
            "target_rpm": target_rpm,
            "actual_rpm": actual_rpm,
            "cycle_time_s": cycle_time,
            "health_score": health,
            "anomaly_score": anomaly,
            "quality_score": quality,
            "tool_wear_percent": self.tool_wear * 100.0,
            "bearing_degradation_percent": self.bearing_degradation * 100.0,
        }

    def _tool_change(self) -> None:
        self.tool_wear = 0.0
        self.coolant_failure = False
        if self._fault_type in {"tool_wear", "coolant_failure"}:
            self._clear_fault()

    def _idle_health_score(self) -> float:
        return clamp(100.0 - 62.0 * self.tool_wear - 26.0 * self.bearing_degradation, 0.0, 100.0)

    def _current_tool_wear_percent(self) -> float | None:
        return self.tool_wear * 100.0

    def _current_bearing_degradation_percent(self) -> float | None:
        return self.bearing_degradation * 100.0

    def _reset_degradation(self) -> None:
        self.tool_wear = 0.10
        self.bearing_degradation = 0.03
        self.coolant_failure = False
        self.custom_rpm = None

    def _degradation_snapshot(self) -> dict[str, float]:
        return {
            "tool_wear_percent": round(self.tool_wear * 100.0, 5),
            "bearing_degradation_percent": round(self.bearing_degradation * 100.0, 5),
            "coolant_failure": float(self.coolant_failure),
        }


class WeldingStation(CausalStation):
    """Electrode wear -> voltage/current drift -> weld quality."""

    def __init__(self, config: StationConfig, *, seed: int) -> None:
        super().__init__(config, seed=seed)
        self.electrode_wear = 0.05
        self.clamp_degradation = 0.03

    def _rehydrate_station_degradation(self, twin: TwinDocument) -> None:
        if twin.health.tool_wear_percent is not None:
            self.electrode_wear = clamp(twin.health.tool_wear_percent / 100.0, 0.0, 1.0)
        else:
            # Fallback: inverse health, but only if no fault is active to avoid confounding
            if not twin.faults.active:
                self.electrode_wear = clamp((100.0 - twin.health.score) / 58.0, 0.0, 1.0)

    def _apply_station_override(self, override: StationOverride) -> None:
        if override.tool_wear_percent is not None:
            self.electrode_wear = clamp(override.tool_wear_percent / 100.0, 0.0, 1.0)

    def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
        fault = self._fault_factor()
```

```
0690         self.electrode_wear = clamp(
0691             self.electrode_wear + 0.000014 * dt_s * (0.5 + self._load_factor) * (1.0 + 4.0 * fault),
0692             0.0,
0693             1.0,
0694         )
0695         self.clamp_degradation = clamp(
0696             self.clamp_degradation + 0.000006 * dt_s * (1.0 + 2.5 * fault),
0697             0.0,
0698             1.0,
0699         )
0700         current = 42.0 + 9.5 * self._load_factor + 16.0 * self.electrode_wear + self._rng.gauss(0.0,
     0.32)
0701         voltage = 20.0 + 3.4 * self.electrode_wear + 2.6 * fault + self._rng.gauss(0.0, 0.08)
0702         pneumatic = 6.3 - 1.5 * self.clamp_degradation - 1.2 * fault + self._rng.gauss(0.0, 0.04)
0703         temperature = 68.0 + 0.30 * current + 6.0 * self.electrode_wear + self._rng.gauss(0.0, 0.25)
0704         vibration = 1.2 + 1.7 * self.clamp_degradation + 1.2 * fault + self._rng.gauss(0.0, 0.06)
0705         cycle_time = self.config.cycle_time_s * (1.0 + 0.1 * self.clamp_degradation + 0.23 * fault) *
     self.cycle_time_multiplier
0706         quality = clamp(
0707             0.997 - 0.48 * self.electrode_wear - 0.28 * self.clamp_degradation - 0.30 * fault,
0708             0.0,
0709             1.0,
0710         )
0711         health = clamp(100.0 - 58.0 * self.electrode_wear - 20.0 * self.clamp_degradation - 30.0 *
     fault, 0.0, 100.0)
0712         return {
0713             "fault_factor": fault,
0714             "temperature_c": temperature,
0715             "vibration_mm_s": max(0.0, vibration),
0716             "current_a": max(0.0, current),
0717             "voltage_v": max(0.0, voltage),
0718             "pneumatic_pressure_bar": max(0.0, pneumatic),
0719             "cycle_time_s": cycle_time,
0720             "health_score": health,
0721             "anomaly_score": clamp(0.02 + 0.55 * self.electrode_wear + 0.2 * self.clamp_degradation +
     0.55 * fault, 0.0, 1.0),
0722             "quality_score": quality,
0723             "tool_wear_percent": self.electrode_wear * 100.0,
0724             "bearing_degradation_percent": None,
0725         }
0726
0727     def _idle_health_score(self) -> float:
0728         return clamp(100.0 - 58.0 * self.electrode_wear - 20.0 * self.clamp_degradation, 0.0, 100.0)
0729
0730     def _current_tool_wear_percent(self) -> float | None:
0731         return self.electrode_wear * 100.0
0732
0733     def _reset_degradation(self) -> None:
0734         self.electrode_wear = 0.05
0735         self.clamp_degradation = 0.03
0736
0737     def _degradation_snapshot(self) -> dict[str, float]:
0738         return {
0739             "electrode_wear_percent": round(self.electrode_wear * 100.0, 5),
0740             "clamp_degradation_percent": round(self.clamp_degradation * 100.0, 5),
0741         }
0742
0743
0744 class InspectionStation(CausalStation):
0745     """Upstream CNC/weld quality -> bounded defect probability -> result.
0746
0747     The Process Twin will own historic token correlation in Phase 3.  The
0748     station model still exposes an explicit upstream-quality input so the causal
0749     dependency is present rather than invented through independent noise.
```

```
0750     """
0751
0752     def __init__(self, config: StationConfig, *, seed: int) -> None:
0753         super().__init__(config, seed=seed)
0754         self._upstream_cnc_quality = 0.995
0755         self._upstream_weld_quality = 0.995
0756         self.last_defect_probability = 0.01
0757
0758     def set_upstream_quality(self, *, cnc_quality: float, weld_quality: float) -> None:
0759         self._upstream_cnc_quality = clamp(cnc_quality, 0.0, 1.0)
0760         self._upstream_weld_quality = clamp(weld_quality, 0.0, 1.0)
0761
0762     def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
0763         fault = self._fault_factor()
0764         upstream_defect_risk = 0.62 * (1.0 - self._upstream_cnc_quality) + 0.38 * (1.0 -
     self._upstream_weld_quality)
0765         self.last_defect_probability = clamp(0.003 + upstream_defect_risk + 0.30 * fault, 0.0, 0.98)
0766         quality = clamp(1.0 - self.last_defect_probability, 0.0, 1.0)
0767         current = 1.8 + 0.9 * self._load_factor + 1.3 * fault + self._rng.gauss(0.0, 0.04)
0768         temperature = 25.0 + 1.5 * self._load_factor + 5.0 * fault + self._rng.gauss(0.0, 0.08)
0769         vibration = 0.18 + 0.15 * self._load_factor + 0.55 * fault + self._rng.gauss(0.0, 0.02)
0770         cycle_time = self.config.cycle_time_s * (1.0 + 0.45 * fault) * self.cycle_time_multiplier
0771         health = clamp(100.0 - 48.0 * fault, 0.0, 100.0)
0772         return {
0773             "fault_factor": fault,
0774             "temperature_c": temperature,
0775             "vibration_mm_s": max(0.0, vibration),
0776             "current_a": max(0.0, current),
0777             "cycle_time_s": cycle_time,
0778             "health_score": health,
0779             "anomaly_score": clamp(0.01 + 0.75 * fault + 0.2 * upstream_defect_risk, 0.0, 1.0),
0780             "quality_score": quality,
0781             "tool_wear_percent": None,
0782             "bearing_degradation_percent": None,
0783         }
0784
0785
0786 class PackagingStation(CausalStation):
0787     """Minimal downstream conveyor/counting station."""
0788
0789     def __init__(self, config: StationConfig, *, seed: int) -> None:
0790         super().__init__(config, seed=seed)
0791         self.conveyor_wear = 0.02
0792
0793     def _rehydrate_station_degradation(self, twin: TwinDocument) -> None:
0794         if twin.health.tool_wear_percent is not None:
0795             self.conveyor_wear = clamp(twin.health.tool_wear_percent / 100.0, 0.0, 1.0)
0796         else:
0797             # Fallback: inverse health, but only if no fault is active to avoid confounding
0798             if not twin.faults.active:
0799                 self.conveyor_wear = clamp((100.0 - twin.health.score) / 40.0, 0.0, 1.0)
0800
0801     def _apply_station_override(self, override: StationOverride) -> None:
0802         if override.tool_wear_percent is not None:
0803             self.conveyor_wear = clamp(override.tool_wear_percent / 100.0, 0.0, 1.0)
0804
0805     def _calculate_values(self, dt_s: float) -> dict[str, float | None]:
0806         fault = self._fault_factor()
0807         self.conveyor_wear = clamp(
0808             self.conveyor_wear + 0.000003 * dt_s * (0.5 + self._load_factor) * (1.0 + 3.0 * fault),
0809             0.0,
0810             1.0,
0811         )
0812         current = 2.5 + 2.4 * self._load_factor + 3.2 * fault + self._rng.gauss(0.0, 0.07)
```

```
        temperature = 28.0 + 1.25 * current + 4.0 * self.conveyor_wear + self._rng.gauss(0.0, 0.12)
        vibration = 0.6 + 0.6 * self.conveyor_wear + 1.6 * fault + self._rng.gauss(0.0, 0.04)
        target_rpm = self.config.nominal_rpm or 800.0
        actual_rpm = max(0.0, target_rpm * (1.0 - 0.28 * fault - 0.08 * self.conveyor_wear) +
    self._rng.gauss(0.0, 3.0))
        cycle_time = self.config.cycle_time_s * (1.0 + 0.13 * self.conveyor_wear + 0.32 * fault) *
    self.cycle_time_multiplier
        quality = clamp(0.999 - 0.20 * fault, 0.0, 1.0)
        health = clamp(100.0 - 40.0 * self.conveyor_wear - 45.0 * fault, 0.0, 100.0)
        return {
            "fault_factor": fault,
            "temperature_c": temperature,
            "vibration_mm_s": max(0.0, vibration),
            "current_a": max(0.0, current),
            "target_rpm": target_rpm,
            "actual_rpm": actual_rpm,
            "cycle_time_s": cycle_time,
            "health_score": health,
            "anomaly_score": clamp(0.01 + 0.45 * self.conveyor_wear + 0.7 * fault, 0.0, 1.0),
            "quality_score": quality,
            "tool_wear_percent": self.conveyor_wear * 100.0,
            "bearing_degradation_percent": None,
        }

    def _idle_health_score(self) -> float:
        return clamp(100.0 - 40.0 * self.conveyor_wear, 0.0, 100.0)

    def _current_tool_wear_percent(self) -> float | None:
        return self.conveyor_wear * 100.0

    def _reset_degradation(self) -> None:
        self.conveyor_wear = 0.02

    def _degradation_snapshot(self) -> dict[str, float]:
        return {"conveyor_wear_percent": round(self.conveyor_wear * 100.0, 5)}


def build_station(config: StationConfig, *, seed: int) -> CausalStation:
    factories = {
        StationType.STAMPING: StampingStation,
        StationType.CNC: CncStation,
        StationType.WELDING: WeldingStation,
        StationType.INSPECTION: InspectionStation,
        StationType.PACKAGING: PackagingStation,
    }
    return factories[config.station](config, seed=seed)


class CausalFactory:
    """Deterministic in-process factory runner used before service extraction."""

    def __init__(
        self,
        *,
        run_id: str = "development-run",
        seed: int = 20260919,
        start_at: datetime = SIMULATION_EPOCH,
        stations: Iterable[StationConfig] = DEFAULT_STATIONS,
    ) -> None:
        if not run_id:
            raise ValueError("run_id is required")
        if start_at.tzinfo is None or start_at.utcoffset() is None:
            raise ValueError("start_at must be timezone-aware")
        self.run_id = run_id
```

```
        self.seed = seed
        self.current_time = start_at.astimezone(timezone.utc)
        self.models: dict[str, CausalStation] = {}
        self._sequences: dict[str, int] = {}
        for index, config in enumerate(stations):
            machine_seed = seed + (index + 1) * 10_007
            self.models[config.machine_id] = build_station(config, seed=machine_seed)
            self._sequences[config.machine_id] = 0

        # Link sequential station pipeline for material flow conservation
        ordered_stations = sorted(self.models.values(), key=lambda s: s.config.station_index)
        for i in range(len(ordered_stations)):
            if i > 0:
                ordered_stations[i].set_upstream_station(ordered_stations[i - 1])
            if i < len(ordered_stations) - 1:
                ordered_stations[i].set_downstream_station(ordered_stations[i + 1], max_buffer=5)

    def apply_command(self, machine_id: str, command: CommandRequest) -> None:
        try:
            self.models[machine_id].apply_command(command)
        except KeyError as error:
            raise KeyError(f"Unknown machine {machine_id}") from error

    def tick(self, *, dt_s: float = 1.0) -> SimulationTick:
        if dt_s <= 0:
            raise ValueError("dt_s must be positive")
        self.current_time += timedelta(seconds=dt_s)
        ordered_stations = sorted(self.models.values(), key=lambda s: s.config.station_index)
        telemetry_by_machine: dict[str, TelemetryPayload] = {}
        inspection_model: InspectionStation | None = None
        for station in ordered_stations:
            if isinstance(station, InspectionStation):
                inspection_model = station
                continue
            telemetry_by_machine[station.machine_id] = station.advance(
                dt_s=dt_s, simulated_at=self.current_time
            )

        # Preserve the direct physical quality dependency now. The durable
        # production-token/DAG representation is deliberately Phase 3.
        if inspection_model is not None:
            cnc = telemetry_by_machine.get("CNC-01")
            weld = telemetry_by_machine.get("WELDING-01")
            if cnc is not None and weld is not None:
                inspection_model.set_upstream_quality(
                    cnc_quality=cnc.quality_score,
                    weld_quality=weld.quality_score,
                )
            telemetry_by_machine[inspection_model.machine_id] = inspection_model.advance(
                dt_s=dt_s, simulated_at=self.current_time
            )

        telemetry = sorted(
            telemetry_by_machine.values(),
            key=lambda sample: self.models[sample.machine_id].config.station_index,
        )

        events: list[EventEnvelope] = []
        for sample in telemetry:
            self._sequences[sample.machine_id] += 1
            sequence = self._sequences[sample.machine_id]
            events.append(
                EventEnvelope(
                    event_id=event_id_for(self.run_id, sample.machine_id, EventKind.TELEMETRY,
```

```
    sequence),
                    run_id=self.run_id,
                    machine_id=sample.machine_id,
                    kind=EventKind.TELEMETRY,
                    sequence=sequence,
                    generated_at=sample.simulated_at,
                    payload=sample.model_dump(mode="json"),
                )
            )
        return SimulationTick(self.current_time, tuple(telemetry), tuple(events))

    def snapshot(self) -> dict[str, object]:
        """A serialisable frozen factory snapshot for Phase 4 Scenario Workers."""

        return {
            "run_id": self.run_id,
            "seed": self.seed,
            "captured_at": self.current_time.isoformat(),
            "stations": {machine_id: model.snapshot() for machine_id, model in self.models.items()},
        }

    @classmethod
    def from_frozen_line_snapshot(
        cls,
        snapshot: FrozenLineSnapshot,
        *,
        seed: int = 20260920,
        run_id: str = "scenario-run",
    ) -> CausalFactory:
        """Construct an isolated, ephemeral CausalFactory rehydrated from a FrozenLineSnapshot."""
        factory = cls(run_id=run_id, seed=seed, start_at=snapshot.captured_at)
        for twin in snapshot.twins:
            if twin.identity.machine_id in factory.models:
                factory.models[twin.identity.machine_id].rehydrate_from_twin(twin)
        return factory


def build_default_factory(*, run_id: str = "development-run", seed: int = 20260919) -> CausalFactory:
    return CausalFactory(run_id=run_id, seed=seed)

```

Source listing: dashboard/backend/services/live_twin.py

```
0001 """Live Twin event reducer with idempotency and observed synchronization timing."""
0002
0003 from __future__ import annotations
0004
0005 from dataclasses import dataclass
0006 from datetime import datetime, timezone
0007 from time import perf_counter_ns
0008
0009 from domain.contracts import (
0010     DesiredState,
0011     EventEnvelope,
0012     EventKind,
0013     FaultSeverity,
0014     FaultState,
0015     HealthState,
0016     IdentityState,
0017     MachineStatus,
0018     OperationalState,
0019     OperationalStateSnapshot,
0020     ProductionState,
0021     SynchronizationState,
0022     TelemetryPayload,
0023     TwinDocument,
0024     utc_now,
0025 )
0026 from repositories.twin_repository import TwinRepository
0027
0028
0029 STATION_INDEX = {
0030     "STAMPING": 1,
0031     "CNC": 2,
0032     "WELDING": 3,
0033     "INSPECTION": 4,
0034     "PACKAGING": 5,
0035 }
0036
0037
0038 @dataclass(frozen=True)
0039 class LiveTwinConsumeResult:
0040     """Result of one consumed transport event."""
0041
0042     document: TwinDocument
0043     event_persisted: bool
0044     state_updated: bool
0045     out_of_order: bool
0046
0047
0048 class LiveTwinService:
0049     """Reduce versioned telemetry envelopes into persistent Live Twin state.
0050
0051     This class is transport-independent. MQTT consumers and the internal HTTP
0052     ingestion endpoint both call `consume`, which makes their behavior and
0053     tests identical.
0054     """
0055
0056     def __init__(self, repository: TwinRepository) -> None:
0057         self._repository = repository
0058
0059     @property
0060     def repository(self) -> TwinRepository:
0061         return self._repository
0062
0063     def consume(
0064         self,
```

```
        event: EventEnvelope,
        *,
        received_at: datetime | None = None,
    ) -> LiveTwinConsumeResult:
        if event.kind != EventKind.TELEMETRY:
            raise ValueError(f"Live Twin currently consumes telemetry events, got {event.kind.value}")
        raw_received = received_at or utc_now()
        if raw_received.tzinfo is None or raw_received.utcoffset() is None:
            raise ValueError("received_at must be timezone-aware")
        received = raw_received.astimezone(timezone.utc)
        materialized_event = event.model_copy(update={"received_at": received})
        started_ns = perf_counter_ns()
        telemetry = TelemetryPayload.model_validate(materialized_event.payload)
        if telemetry.machine_id != materialized_event.machine_id:
            raise ValueError("event.machine_id must match payload.machine_id")

        existing = self._repository.get_twin(materialized_event.machine_id)
        out_of_order = bool(
            existing and materialized_event.sequence <= existing.synchronization.latest_sequence
        )
        if out_of_order:
            # Preserve current state while retaining the late event in history.
            assert existing is not None
            candidate = existing
            state_updated = False
        else:
            processing_latency_ms = (perf_counter_ns() - started_ns) / 1_000_000.0
            candidate = self._reduce_telemetry(
                existing=existing,
                event=materialized_event,
                telemetry=telemetry,
                received_at=received,
                processing_latency_ms=processing_latency_ms,
            )
            state_updated = True

        write = self._repository.save_event_and_twin(materialized_event, candidate)
        # A duplicate event was not newly persisted and therefore did not alter
        # state even if its sequence looks newer in an isolated retry.
        return LiveTwinConsumeResult(
            document=write.document,
            event_persisted=write.applied,
            state_updated=state_updated and write.applied,
            out_of_order=out_of_order,
        )

    def get_twin(self, machine_id: str) -> TwinDocument | None:
        return self._repository.get_twin(machine_id)

    def list_twins(self) -> list[TwinDocument]:
        return self._repository.list_twins()

    def telemetry_history(self, machine_id: str, *, limit: int = 100) -> list[EventEnvelope]:
        return self._repository.list_telemetry(machine_id, limit=limit)

    def list_events(self, *, limit: int = 100, machine_id: str | None = None) -> list[EventEnvelope]:
        return self._repository.list_events(limit=limit, machine_id=machine_id)

    @staticmethod
    def _reduce_telemetry(
        *,
        existing: TwinDocument | None,
        event: EventEnvelope,
        telemetry: TelemetryPayload,
```

```
0129         received_at: datetime,
0130         processing_latency_ms: float,
0131     ) -> TwinDocument:
0132         delivery_latency_ms = max(
0133             0.0,
0134             (received_at - event.generated_at).total_seconds() * 1_000.0,
0135         )
0136         station_index = STATION_INDEX[telemetry.station.value]
0137         same_fault = existing and existing.faults.code == telemetry.fault_code
0138         inferred_severity = existing.faults.severity if same_fault and existing else None
0139         if inferred_severity is None and telemetry.fault_code is not None:
0140             if telemetry.status == MachineStatus.FAULT or telemetry.operational_state ==
     OperationalState.FAULT:
0141                 inferred_severity = FaultSeverity.CRITICAL
0142             elif telemetry.status == MachineStatus.DEGRADED:
0143                 inferred_severity = FaultSeverity.WARNING
0144             elif telemetry.status == MachineStatus.WARNING:
0145                 inferred_severity = FaultSeverity.INFO
0146         fault_state = FaultState(
0147             active=telemetry.fault_code is not None,
0148             code=telemetry.fault_code,
0149             severity=inferred_severity,
0150             updated_at=received_at if telemetry.fault_code is not None else None,
0151         )
0152         return TwinDocument(
0153             identity=IdentityState(
0154                 machine_id=telemetry.machine_id,
0155                 station=telemetry.station,
0156                 station_index=station_index,
0157             ),
0158             operational=OperationalStateSnapshot(
0159                 state=telemetry.operational_state,
0160                 status=telemetry.status,
0161                 updated_at=received_at,
0162             ),
0163             telemetry=telemetry,
0164             health=HealthState(
0165                 score=telemetry.health_score,
0166                 anomaly_score=telemetry.anomaly_score,
0167                 tool_wear_percent=telemetry.tool_wear_percent,
0168                 bearing_degradation_percent=telemetry.bearing_degradation_percent,
0169             ),
0170             production=ProductionState(
0171                 cumulative_output=telemetry.production_count,
0172                 wip_count=getattr(telemetry, "wip_count", 0),
0173                 is_starved=getattr(telemetry, "is_starved", False),
0174                 latest_quality_score=telemetry.quality_score,
0175                 latest_token_id=existing.production.latest_token_id if existing else None,
0176             ),
0177             faults=fault_state,
0178             desired=existing.desired if existing else DesiredState(),
0179             synchronization=SynchronizationState(
0180                 latest_sequence=event.sequence,
0181                 last_generated_at=event.generated_at,
0182                 last_received_at=received_at,
0183                 delivery_latency_ms=round(delivery_latency_ms, 6),
0184                 processing_latency_ms=round(max(0.0, processing_latency_ms), 6),
0185                 source_age_ms=round(delivery_latency_ms, 6),
0186             ),
0187             last_event_id=event.event_id,
0188             revision=(existing.revision + 1) if existing else 1,
0189             updated_at=received_at,
0190         )
```

Source listing: dashboard/backend/services/process_twin.py

```
0001 """Process-level Digital Twin service composing independent station twins.
0002
0003 Implements Sections 4.2, 4.4, and 9 of the research report:
0004 - DAG-based process topology with frozen-snapshot semantics.
0005 - Lightweight production token flow with station lineage and carried flags.
0006 - Line-level computations: bottleneck attribution and upstream defect tracing.
0007 """
0008
0009 from __future__ import annotations
0010
0011 from collections import deque
0012 from contextlib import suppress
0013 from repositories.twin_repository import TwinRepository
0014 from datetime import datetime, timezone
0015 import math
0016 from threading import RLock
0017 from typing import Literal
0018 from uuid import uuid4
0019
0020 from domain.contracts import (
0021     BottleneckReport,
0022     DAGEdge,
0023     DAGNode,
0024     DefectAttributionReport,
0025     FrozenLineSnapshot,
0026     MachineStatus,
0027     ProcessMetrics,
0028     ProcessTopology,
0029     ProcessTwinView,
0030     ProductionToken,
0031     StationBottleneckDetail,
0032     StationDefectAttribution,
0033     StationType,
0034     TwinDocument,
0035     utc_now,
0036 )
0037 from services.live_twin import LiveTwinService
0038
0039
0040 CANONICAL_TOPOLOGY_ORDER: tuple[str, ...] = (
0041     "STAMPING-01",
0042     "CNC-01",
0043     "WELDING-01",
0044     "INSPECTION-01",
0045     "PACKAGING-01",
0046 )
0047
0048 CANONICAL_NODES: tuple[DAGNode, ...] = (
0049     DAGNode(
0050         machine_id="STAMPING-01",
0051         station=StationType.STAMPING,
0052         station_index=1,
0053         nominal_cycle_time_s=8.5,
0054         upstream_machine_ids=[],
0055         downstream_machine_ids=["CNC-01"],
0056     ),
0057     DAGNode(
0058         machine_id="CNC-01",
0059         station=StationType.CNC,
0060         station_index=2,
0061         nominal_cycle_time_s=42.0,
0062         upstream_machine_ids=["STAMPING-01"],
0063         downstream_machine_ids=["WELDING-01"],
0064     ),
```

```
0065     DAGNode(
0066         machine_id="WELDING-01",
0067         station=StationType.WELDING,
0068         station_index=3,
0069         nominal_cycle_time_s=15.0,
0070         upstream_machine_ids=["CNC-01"],
0071         downstream_machine_ids=["INSPECTION-01"],
0072     ),
0073     DAGNode(
0074         machine_id="INSPECTION-01",
0075         station=StationType.INSPECTION,
0076         station_index=4,
0077         nominal_cycle_time_s=6.0,
0078         upstream_machine_ids=["WELDING-01"],
0079         downstream_machine_ids=["PACKAGING-01"],
0080     ),
0081     DAGNode(
0082         machine_id="PACKAGING-01",
0083         station=StationType.PACKAGING,
0084         station_index=5,
0085         nominal_cycle_time_s=10.0,
0086         upstream_machine_ids=["INSPECTION-01"],
0087         downstream_machine_ids=[],
0088     ),
0089 )
0090
0091 CANONICAL_EDGES: tuple[DAGEdge, ...] = (
0092     DAGEdge(from_machine_id="STAMPING-01", to_machine_id="CNC-01", transition_type="material_flow"),
0093     DAGEdge(from_machine_id="CNC-01", to_machine_id="WELDING-01", transition_type="material_flow"),
0094     DAGEdge(from_machine_id="WELDING-01", to_machine_id="INSPECTION-01",
     transition_type="material_flow"),
0095     DAGEdge(from_machine_id="INSPECTION-01", to_machine_id="PACKAGING-01",
     transition_type="material_flow"),
0096 )
0097
0098
0099 class ProcessTwinService:
0100     """Manages the DAG topology, frozen snapshot capture, tokens, and line analytics."""
0101
0102     def __init__(self, live_twin_service: LiveTwinService, *, repository: TwinRepository | None = None,
      max_token_history: int = 1000) -> None:
0103         self._live_twin_service = live_twin_service
0104         self._repository: TwinRepository | None = repository or getattr(live_twin_service,
     "repository", None)
0105         self._lock = RLock()
0106         self._topology = ProcessTopology(
0107             version="five-station-v1",
0108             nodes=list(CANONICAL_NODES),
0109             edges=list(CANONICAL_EDGES),
0110             order=list(CANONICAL_TOPOLOGY_ORDER),
0111         )
0112         self._max_token_history = max_token_history
0113         self._tokens: deque[ProductionToken] = deque(maxlen=max_token_history)
0114         self._tokens_by_id: dict[str, ProductionToken] = {}
0115         self._dispatched_count = 0
0116         self._scrapped_count = 0
0117         # Token emission rate control (Bug 7): only emit a token when enough
0118         # simulated time has elapsed to match the physical bottleneck rate.
0119         self._cycle_accumulator: float = 0.0
0120         self._bottleneck_cycle_time_s: float = 42.0  # CNC-01 is the default bottleneck
0121
0122     @property
0123     def topology(self) -> ProcessTopology:
0124         return self._topology
```

```
0125
0126    # ------------------------------------------------------------------
0127    # Frozen Snapshot Semantics (§4.2)
0128    # ------------------------------------------------------------------
0129
0130    def accumulate_time(self, dt_s: float) -> ProductionToken | None:
0131        """Accumulate simulation time and emit a token when enough has passed.
0132
0133        The emission rate is governed by the physical line bottleneck cycle time
0134        (default 42.0s for CNC-01).  This prevents emitting 42x too many tokens
0135        when called once per simulation tick (1s).
0136        """
0137        self._cycle_accumulator += dt_s
0138        if self._cycle_accumulator >= self._bottleneck_cycle_time_s:
0139            self._cycle_accumulator -= self._bottleneck_cycle_time_s
0140            return self.emit_token_through_pipeline()
0141        return None
0142
0143    def capture_snapshot(self) -> FrozenLineSnapshot:
0144        """Capture an atomic, immutable snapshot of all 5 station Live Twins.
0145
0146        Reading all station twins at one instant avoids distributed-snapshot
0147        inconsistency where different stations are observed at different times.
0148        """
0149        twins = self._live_twin_service.list_twins()
0150        if not twins:
0151            raise ValueError("Cannot capture FrozenLineSnapshot: no Live Twins exist in repository")
0152
0153        # Guarantee ordering by station_index
0154        ordered_twins = sorted(twins, key=lambda t: t.identity.station_index)
0155
0156        return FrozenLineSnapshot(
0157            snapshot_id=str(uuid4()),
0158            captured_at=utc_now(),
0159            topology_version=self._topology.version,
0160            twins=ordered_twins,
0161        )
0162
0163    # ------------------------------------------------------------------
0164    # Production Token Abstraction & Material Flow (§9.1)
0165    # ------------------------------------------------------------------
0166
0167    def emit_token_through_pipeline(
0168        self,
0169        *,
0170        token_id: str | None = None,
0171        part_type: str = "automotive_component",
0172        twins_map: dict[str, TwinDocument] | None = None,
0173        emitted_at: datetime | None = None,
0174    ) -> ProductionToken:
0175        """Simulate the progression of one discrete part along the 5-station DAG.
0176
0177        Accumulates quality scores and flags at each station, determines
0178        inspection pass/fail, and finalizes at packaging dispatch or scrap.
0179        """
0180        now = emitted_at or utc_now()
0181        tid = token_id or f"tok-{uuid4().hex[:12]}"
0182
0183        # If twins_map is not passed, use current Live Twin state
0184        if twins_map is None:
0185            twins = self._live_twin_service.list_twins()
0186            twins_map = {t.identity.machine_id: t for t in twins}
0187
0188        stamping = twins_map.get("STAMPING-01")
```

```
0189        cnc = twins_map.get("CNC-01")
0190        welding = twins_map.get("WELDING-01")
0191        inspection = twins_map.get("INSPECTION-01")
0192        packaging = twins_map.get("PACKAGING-01")
0193
0194        lineage: list[str] = []
0195        carried_flags: dict[str, float | bool | str] = {}
0196
0197        # 1. Stamping Press
0198        lineage.append("STAMPING-01")
0199        stamping_q = stamping.telemetry.quality_score if stamping else 0.99
0200        carried_flags["stamping_quality"] = round(stamping_q, 4)
0201        if stamping and stamping.telemetry.force_kn is not None:
0202            carried_flags["stamping_force_kn"] = round(stamping.telemetry.force_kn, 2)
0203
0204        # 2. CNC Machining Center
0205        lineage.append("CNC-01")
0206        cnc_q = cnc.telemetry.quality_score if cnc else 0.98
0207        carried_flags["machining_quality"] = round(cnc_q, 4)
0208        if cnc and cnc.telemetry.tool_wear_percent is not None:
0209            carried_flags["cnc_tool_wear_percent"] = round(cnc.telemetry.tool_wear_percent, 2)
0210        if cnc and cnc.telemetry.bearing_degradation_percent is not None:
0211            carried_flags["cnc_bearing_wear_percent"] =
    round(cnc.telemetry.bearing_degradation_percent, 2)
0212
0213        # 3. Welding/Assembly Station
0214        lineage.append("WELDING-01")
0215        weld_q = welding.telemetry.quality_score if welding else 0.99
0216        carried_flags["weld_quality"] = round(weld_q, 4)
0217        if welding and welding.telemetry.voltage_v is not None:
0218            carried_flags["welding_voltage_v"] = round(welding.telemetry.voltage_v, 2)
0219
0220        # 4. Quality Inspection Station
0221        lineage.append("INSPECTION-01")
0222        # Defect probability from upstream quality: f(machining_quality, weld_quality,
    stamping_quality)
0223        # Bounded causal function per §8.4
0224        defect_prob = (
0225            0.005  # Base defect rate (<1% Bosch benchmark)
0226            + 0.55 * max(0.0, 0.95 - cnc_q) ** 1.5
0227            + 0.35 * max(0.0, 0.95 - weld_q) ** 1.5
0228            + 0.10 * max(0.0, 0.95 - stamping_q) ** 1.5
0229        )
0230        defect_prob = max(0.0, min(1.0, defect_prob))
0231        carried_flags["defect_probability"] = round(defect_prob, 5)
0232
0233        # Inspection verdict: if inspection twin reports fault/low quality or defect_prob > 0.15
0234        is_defective = defect_prob >= 0.15 or (inspection and inspection.telemetry.quality_score <
    0.85)
0235        quality_status: Literal["pending", "pass", "fail"] = "fail" if is_defective else "pass"
0236        carried_flags["inspection_verdict"] = "REJECTED" if is_defective else "ACCEPTED"
0237
0238        # 5. Packaging/Dispatch Station
0239        if not is_defective:
0240            lineage.append("PACKAGING-01")
0241            carried_flags["disposition"] = "DISPATCHED"
0242            carried_flags["dispatched"] = True
0243        else:
0244            carried_flags["disposition"] = "SCRAPPED"
0245            carried_flags["dispatched"] = False
0246
0247        token = ProductionToken(
0248            token_id=tid,
0249            part_type=part_type,
```

```
            quality=quality_status,
            source_station=StationType.STAMPING,
            emitted_at=now,
            carried_flags=carried_flags,
            lineage=lineage,
        )

        with self._lock:
            # Evict oldest token from dict if deque is at capacity (Bug 3 fix)
            if len(self._tokens) == self._max_token_history:
                evicted = self._tokens[0]  # Oldest item about to be pushed off
                self._tokens_by_id.pop(evicted.token_id, None)
            self._tokens.append(token)
            self._tokens_by_id[token.token_id] = token
            if is_defective:
                self._scrapped_count += 1
            else:
                self._dispatched_count += 1

        if self._repository is not None:
            with suppress(Exception):
                self._repository.save_token(token)

        return token

    def list_tokens(
        self,
        *,
        limit: int = 50,
        quality: str | None = None,
        station: str | None = None,
    ) -> list[ProductionToken]:
        """Return newest-first production tokens matching optional filters."""
        if limit < 1 or limit > 10_000:
            raise ValueError("limit must be between 1 and 10000")

        if self._repository is not None:
            try:
                repo_tokens = self._repository.list_tokens(limit=limit, quality=quality,
    station=station)
                if repo_tokens:
                    return repo_tokens
            except Exception:
                pass

        with self._lock:
            results: list[ProductionToken] = []
            for token in reversed(self._tokens):
                if quality and token.quality != quality.lower():
                    continue
                if station and station.upper() not in token.lineage:
                    continue
                results.append(token)
                if len(results) >= limit:
                    break
            return results

    def get_token(self, token_id: str) -> ProductionToken | None:
        with self._lock:
            local = self._tokens_by_id.get(token_id)
            if local is not None:
                return local
        if self._repository is not None:
            try:
```

```
0313                return self._repository.get_token(token_id)
0314            except Exception:
0315                pass
0316        return None
0317
0318    # ------------------------------------------------------------------
0319    # Bottleneck Attribution (§9.2)
0320    # ------------------------------------------------------------------
0321
0322    def analyze_bottleneck(self, snapshot: FrozenLineSnapshot | None = None) -> BottleneckReport:
0323        """Compute the binding line bottleneck by comparing effective station cycle times.
0324
0325        Answers: Which station is currently the binding constraint on line throughput?
0326        Accounts for nominal cycle time and operational state (e.g. running, degraded, faulted).
0327        """
0328        active_snapshot = snapshot or self.capture_snapshot()
0329
0330        station_details: list[StationBottleneckDetail] = []
0331        max_effective_time = 0.0
0332        binding_twin: TwinDocument | None = None
0333
0334        for twin in active_snapshot.twins:
0335            nominal = next(
0336                (node.nominal_cycle_time_s for node in self._topology.nodes if node.machine_id ==
     twin.identity.machine_id),
0337                twin.telemetry.cycle_time_s,
0338            )
0339            reported_time = twin.telemetry.cycle_time_s or nominal
0340
0341            # Operational penalty factor
0342            status = twin.operational.status
0343            if status == MachineStatus.RUNNING:
0344                factor = 1.0
0345            elif status == MachineStatus.WARNING:
0346                factor = 1.15
0347            elif status == MachineStatus.DEGRADED:
0348                factor = 1.35
0349            elif status in (MachineStatus.FAULT, MachineStatus.OFFLINE):
0350                factor = 10.0  # Line halted or severely obstructed
0351            elif status == MachineStatus.MAINTENANCE:
0352                factor = 5.0
0353            else:  # IDLE
0354                factor = 1.2
0355
0356            effective_time = round(reported_time * factor, 2)
0357            if effective_time > max_effective_time:
0358                max_effective_time = effective_time
0359                binding_twin = twin
0360
0361            station_details.append(
0362                StationBottleneckDetail(
0363                    machine_id=twin.identity.machine_id,
0364                    station=twin.identity.station,
0365                    cycle_time_s=round(reported_time, 2),
0366                    effective_cycle_time_s=effective_time,
0367                    slack_time_s=0.0,  # Computed below once max is known
0368                    is_bottleneck=False,
0369                    status=status,
0370                )
0371            )
0372
0373        if binding_twin is None and active_snapshot.twins:
0374            binding_twin = active_snapshot.twins[0]
0375            max_effective_time = binding_twin.telemetry.cycle_time_s
```

```
0376
0377        # Compute slacks relative to binding bottleneck
0378        updated_details: list[StationBottleneckDetail] = []
0379        for detail in station_details:
0380            slack = round(max(0.0, max_effective_time - detail.effective_cycle_time_s), 2)
0381            is_bt = (detail.machine_id == (binding_twin.identity.machine_id if binding_twin else ""))
0382            updated_details.append(
0383                detail.model_copy(update={"slack_time_s": slack, "is_bottleneck": is_bt})
0384            )
0385
0386        hourly_capacity = round(3600.0 / max_effective_time, 2) if max_effective_time > 0 else 0.0
0387
0388        return BottleneckReport(
0389            binding_machine_id=binding_twin.identity.machine_id if binding_twin else "UNKNOWN",
0390            binding_station=binding_twin.identity.station if binding_twin else StationType.CNC,
0391            binding_cycle_time_s=max_effective_time,
0392            max_line_capacity_per_hour=hourly_capacity,
0393            stations=updated_details,
0394            captured_at=active_snapshot.captured_at,
0395        )
0396
0397    # ------------------------------------------------------------------
0398    # Propagated Defect Tracing (§9.2)
0399    # ------------------------------------------------------------------
0400
0401    def trace_defects(self, *, limit: int = 100) -> DefectAttributionReport:
0402        """Trace inspection defect occurrences back to upstream station degradations.
0403
0404        Answers: Given defective tokens, which upstream machine contributed most
0405        to the failure (CNC machining quality degradation vs Weld quality drift vs Stamping)?
0406        """
0407        tokens = self.list_tokens(limit=limit)
0408        total_count = len(tokens)
0409        defective_tokens = [t for t in tokens if t.quality == "fail"]
0410        defective_count = len(defective_tokens)
0411
0412        evaluated_at = utc_now()
0413
0414        if defective_count == 0:
0415            return DefectAttributionReport(
0416                total_tokens_evaluated=total_count,
0417                defective_tokens_count=0,
0418                defect_rate_percent=0.0,
0419                primary_root_cause_station=None,
0420                primary_root_cause_machine_id=None,
0421                attributions=[
0422                    StationDefectAttribution(
0423                        machine_id="CNC-01",
0424                        station=StationType.CNC,
0425                        attribution_percent=0.0,
0426                        sample_count=0,
0427                        average_quality_score=1.0,
0428                        reason="No defects detected in recent token window.",
0429                    ),
0430                    StationDefectAttribution(
0431                        machine_id="WELDING-01",
0432                        station=StationType.WELDING,
0433                        attribution_percent=0.0,
0434                        sample_count=0,
0435                        average_quality_score=1.0,
0436                        reason="No defects detected in recent token window.",
0437                    ),
0438                    StationDefectAttribution(
0439                        machine_id="STAMPING-01",
```

```
                        station=StationType.STAMPING,
                        attribution_percent=0.0,
                        sample_count=0,
                        average_quality_score=1.0,
                        reason="No defects detected in recent token window.",
                    ),
                ],
                summary_analysis="Production line operating nominally. Zero defect events in current evaluation window.",
                evaluated_at=evaluated_at,
            )

        # Tally upstream degradation penalties across defective tokens
        cnc_penalty_sum = 0.0
        weld_penalty_sum = 0.0
        stamping_penalty_sum = 0.0

        cnc_scores: list[float] = []
        weld_scores: list[float] = []
        stamping_scores: list[float] = []

        for token in defective_tokens:
            flags = token.carried_flags
            cq = float(flags.get("machining_quality", 0.98))
            wq = float(flags.get("weld_quality", 0.99))
            sq = float(flags.get("stamping_quality", 0.99))

            cnc_scores.append(cq)
            weld_scores.append(wq)
            stamping_scores.append(sq)

            # Penalize deviation from nominal (0.98+)
            cnc_penalty_sum += max(0.0, 0.98 - cq)
            weld_penalty_sum += max(0.0, 0.98 - wq)
            stamping_penalty_sum += max(0.0, 0.98 - sq)

        total_penalty = cnc_penalty_sum + weld_penalty_sum + stamping_penalty_sum
        if total_penalty <= 1e-6:
            # Random unmodeled residual defects
            cnc_attr = 50.0
            weld_attr = 35.0
            stamping_attr = 15.0
        else:
            cnc_attr = round((cnc_penalty_sum / total_penalty) * 100.0, 1)
            weld_attr = round((weld_penalty_sum / total_penalty) * 100.0, 1)
            stamping_attr = round((stamping_penalty_sum / total_penalty) * 100.0, 1)

        attributions: list[StationDefectAttribution] = [
            StationDefectAttribution(
                machine_id="CNC-01",
                station=StationType.CNC,
                attribution_percent=cnc_attr,
                sample_count=defective_count,
                average_quality_score=round(sum(cnc_scores) / len(cnc_scores), 4) if cnc_scores else 1.0,
                reason=f"Machining out-of-tolerance / tool wear degradation (avg quality: {round(sum(cnc_scores)/len(cnc_scores), 2) if cnc_scores else 1.0})",
            ),
            StationDefectAttribution(
                machine_id="WELDING-01",
                station=StationType.WELDING,
                attribution_percent=weld_attr,
                sample_count=defective_count,
                average_quality_score=round(sum(weld_scores) / len(weld_scores), 4) if weld_scores else
```

```
     1.0,
                reason=f"Welding electrode wear / voltage drift (avg quality:
{round(sum(weld_scores)/len(weld_scores), 2) if weld_scores else 1.0})",
            ),
            StationDefectAttribution(
                machine_id="STAMPING-01",
                station=StationType.STAMPING,
                attribution_percent=stamping_attr,
                sample_count=defective_count,
                average_quality_score=round(sum(stamping_scores) / len(stamping_scores), 4) if
stamping_scores else 1.0,
                reason=f"Stamping press force variance (avg quality:
{round(sum(stamping_scores)/len(stamping_scores), 2) if stamping_scores else 1.0})",
            ),
        ]

        # Rank by attribution percentage
        attributions.sort(key=lambda a: a.attribution_percent, reverse=True)
        primary = attributions[0]
        defect_rate = round((defective_count / total_count) * 100.0, 2) if total_count > 0 else 0.0

        summary = (
            f"Inspection rejected {defective_count} of {total_count} parts ({defect_rate}% defect
rate). "
            f"Primary root cause traced to {primary.machine_id} ({primary.station.value}) contributing
"
            f"{primary.attribution_percent}% of quality degradation."
        )

        return DefectAttributionReport(
            total_tokens_evaluated=total_count,
            defective_tokens_count=defective_count,
            defect_rate_percent=defect_rate,
            primary_root_cause_station=primary.station,
            primary_root_cause_machine_id=primary.machine_id,
            attributions=attributions,
            summary_analysis=summary,
            evaluated_at=evaluated_at,
        )

    # ------------------------------------------------------------------
    # Full Process Twin View (§9.2)
    # ------------------------------------------------------------------

    def get_process_view(self) -> ProcessTwinView:
        """Compose an atomic FrozenLineSnapshot with line-level ProcessMetrics."""
        snapshot = self.capture_snapshot()
        bottleneck = self.analyze_bottleneck(snapshot)

        total_stations = len(snapshot.twins)
        available_stations = sum(
            1 for twin in snapshot.twins
            if twin.operational.status not in (MachineStatus.FAULT, MachineStatus.OFFLINE)
        )

        # Expected overall line quality: product of station quality scores
        quality_product = 1.0
        max_source_age_ms = 0.0
        for twin in snapshot.twins:
            quality_product *= twin.telemetry.quality_score
            age = twin.synchronization.source_age_ms or 0.0
            if age > max_source_age_ms:
                max_source_age_ms = age

```

```
0559         notes = [
0560             f"Line bottleneck is {bottleneck.binding_machine_id} ({bottleneck.binding_station.value})
     with effective cycle time {bottleneck.binding_cycle_time_s}s.",
0561             f"Theoretical max line throughput: {bottleneck.max_line_capacity_per_hour} parts/hour.",
0562             f"Station availability: {available_stations}/{total_stations} stations operational.",
0563         ]
0564
0565         metrics = ProcessMetrics(
0566             bottleneck_machine_id=bottleneck.binding_machine_id,
0567             bottleneck_station=bottleneck.binding_station,
0568             bottleneck_cycle_time_s=bottleneck.binding_cycle_time_s,
0569             line_capacity_per_hour=bottleneck.max_line_capacity_per_hour,
0570             expected_line_quality=round(quality_product, 4),
0571             available_stations=available_stations,
0572             total_stations=total_stations,
0573             snapshot_max_source_age_ms=round(max_source_age_ms, 2) if max_source_age_ms > 0 else None,
0574             notes=notes,
0575         )
0576
0577         return ProcessTwinView(snapshot=snapshot, metrics=metrics)
```

Source listing: dashboard/backend/services/scenario_worker.py

```
0001 """Scenario Worker service for isolated forward simulations from frozen snapshots.
0002
0003 Implements Sections 4.3, 4.4, and Table 5 of the research report:
0004 - Snapshot forking: rehydrates a detached CausalFactory from an immutable FrozenLineSnapshot.
0005 - Deterministic forward simulation: runs identical Baseline and Counterfactual trajectories
0006   under the exact same seed to isolate intervention effects.
0007 - Comparative delta analytics: computes output, scrap, quality, energy, and bottleneck shifts.
0008 - Strict fork immutability: verifies that Live Twin documents and running simulator state
0009   are never mutated during scenario execution.
0010 """
0011
0012 from __future__ import annotations
0013
0014 from datetime import datetime, timezone
0015 import math
0016 from typing import Any
0017 from uuid import uuid4
0018
0019 from domain.contracts import (
0020     CommandName,
0021     FaultSeverity,
0022     FrozenLineSnapshot,
0023     LineOutcome,
0024     OperationalState,
0025     ScenarioDelta,
0026     ScenarioForkRequest,
0027     ScenarioRunResult,
0028     ScenarioStatus,
0029     StationOutcome,
0030     StationOverride,
0031     StationType,
0032     TrajectorySample,
0033     utc_now,
0034 )
0035 from repositories.scenario_repository import ScenarioRepository
0036 from services.live_twin import LiveTwinService
0037 from services.process_twin import ProcessTwinService
0038 from simulation.causal import CausalFactory, clamp
0039 from services.scenario_reasoning import ScenarioReasoningService
0040
0041
0042 # ---------------------------------------------------------------------------
0043 # Pre-packaged Research Presets (§4.3)
0044 # ---------------------------------------------------------------------------
0045
0046 PRESET_DEFINITIONS: dict[str, dict[str, Any]] = {
0047     "cnc_double_speed": {
0048         "name": "Double CNC Speed (2.0x Cadence / 21s)",
0049         "description": "Run CNC-01 at 200% feed/spindle speed. Cuts CNC cycle time from 42s to 21s,
     doubling throughput at the expense of accelerated tool wear.",
0050         "overrides": [
0051             StationOverride(
0052                 machine_id="CNC-01",
0053                 cycle_time_multiplier=0.50,
0054                 load_factor=0.90,
0055             )
0056         ],
0057     },
0058     "cnc_parallel_cell": {
0059         "name": "Add 2nd CNC in Parallel (Dual-CNC Cell)",
0060         "description": "Simulate adding a duplicate CNC milling station in parallel. Divides cycle time
      across 2 machines (effective 21s) without tool wear or thermal penalty.",
0061         "overrides": [
0062             StationOverride(
```

```
                machine_id="CNC-01",
                cycle_time_multiplier=0.50,
                load_factor=0.55,
            )
        ],
    },
    "cnc_triple_speed_shift": {
        "name": "Triple CNC Speed (14s) - Bottleneck Shift",
        "description": "Run CNC at 300% speed (14s). Drops CNC below Welding (15s), causing the line
bottleneck to shift to WELDING-01.",
        "overrides": [
            StationOverride(
                machine_id="CNC-01",
                cycle_time_multiplier=0.333,
                load_factor=0.98,
            )
        ],
    },
    "line_cadence_derate": {
        "name": "Paced Derate (-20% Speed for Tool Preservation)",
        "description": "Slow CNC cadence to 52.5s. Cools spindle temperature, dampens chatter, and
extends tool life to complete the shift.",
        "overrides": [
            StationOverride(
                machine_id="CNC-01",
                cycle_time_multiplier=1.25,
                load_factor=0.45,
            )
        ],
    },
    "cnc_unplanned_outage": {
        "name": "15-Minute Unplanned CNC Outage (Line Ripple)",
        "description": "Simulate sudden CNC spindle seizure. Evaluates buffer exhaustion: Stamping
blocks in 42s and Welding starves in 75s.",
        "overrides": [
            StationOverride(
                machine_id="CNC-01",
                fault_type="spindle_seizure",
                fault_severity=FaultSeverity.CRITICAL,
                cycle_time_multiplier=5.0,
            )
        ],
    },
    "preventive_tool_change": {
        "name": "Preventive CNC Tool Change",
        "description": "Simulate immediate tool replacement at CNC-01 to prevent impending failure and
restore cutting quality.",
        "overrides": [
            StationOverride(
                machine_id="CNC-01",
                command=CommandName.TOOL_CHANGE,
                tool_wear_percent=0.0,
            )
        ],
    },
    "coolant_pump_failure": {
        "name": "Coolant Pump Failure",
        "description": "Simulate unexpected coolant circulation breakdown at CNC-01 causing rapid
thermal runaway and surface defects.",
        "overrides": [
            StationOverride(
                machine_id="CNC-01",
                fault_type="coolant_failure",
                fault_severity=FaultSeverity.CRITICAL,
```

```
                load_factor=0.85,
            )
        ],
    },
    "line_cadence_boost": {
        "name": "High-Cadence Line Acceleration",
        "description": "Accelerate Stamping and CNC cycle times by 20% to evaluate line throughput ceiling vs increased energy and wear.",
        "overrides": [
            StationOverride(
                machine_id="STAMPING-01",
                cycle_time_multiplier=0.80,
                load_factor=0.80,
            ),
            StationOverride(
                machine_id="CNC-01",
                cycle_time_multiplier=0.80,
                load_factor=0.85,
            ),
        ],
    },
    "welding_electrode_drift": {
        "name": "Welding Electrode Degradation",
        "description": "Simulate heavy electrode wear at WELDING-01 to trace downstream defect propagation at Inspection.",
        "overrides": [
            StationOverride(
                machine_id="WELDING-01",
                tool_wear_percent=85.0,
                fault_type="electrode_wear",
                fault_severity=FaultSeverity.WARNING,
            )
        ],
    },
}



class ScenarioWorkerService:
    """Orchestrates isolated what-if simulations forked from frozen snapshots."""

    MAX_SNAPSHOT_CACHE = 20

    def __init__(
        self,
        process_twin_service: ProcessTwinService,
        live_twin_service: LiveTwinService,
        scenario_repository: ScenarioRepository,
    ) -> None:
        self._process_twin_service = process_twin_service
        self._live_twin_service = live_twin_service
        self._scenario_repository = scenario_repository
        self._snapshot_cache: dict[str, FrozenLineSnapshot] = {}

    def get_presets(self) -> list[dict[str, Any]]:
        """Return list of available research preset configurations."""
        return [
            {
                "preset_id": key,
                "name": val["name"],
                "description": val["description"],
                "overrides": [o.model_dump(mode="json") for o in val["overrides"]],
            }
            for key, val in PRESET_DEFINITIONS.items()
```

```
        ]

    def run_preset(
        self,
        preset_id: str,
        *,
        horizon_seconds: float = 120.0,
        seed: int = 20260920,
    ) -> ScenarioRunResult:
        if preset_id not in PRESET_DEFINITIONS:
            raise KeyError(f"Unknown scenario preset: {preset_id}")
        preset = PRESET_DEFINITIONS[preset_id]
        request = ScenarioForkRequest(
            name=preset["name"],
            description=preset["description"],
            horizon_seconds=horizon_seconds,
            seed=seed,
            overrides=list(preset["overrides"]),
        )
        return self.run_scenario(request)

    def run_scenario(self, request: ScenarioForkRequest) -> ScenarioRunResult:
        """Run an isolated forward counterfactual simulation.

        Guarantees that Live Twin state is never mutated (§4.3).
        """
        # Step 1: Obtain or resolve the FrozenLineSnapshot
        snapshot = self._resolve_snapshot(request.snapshot_id)

        # Step 2: Record Live Twin state fingerprint for strict immutability verification
        pre_fingerprint = self._compute_live_twins_fingerprint()

        created_at = utc_now()

        # Step 3: Run Baseline forward simulation (no overrides)
        baseline_factory = CausalFactory.from_frozen_line_snapshot(
            snapshot,
            seed=request.seed,
            run_id=f"base-{uuid4().hex[:8]}",
        )
        baseline_ticks = self._simulate_forward(
            baseline_factory,
            horizon_seconds=request.horizon_seconds,
            time_step_s=request.time_step_s,
        )
        baseline_outcome = self._calculate_line_outcome(baseline_factory, baseline_ticks, snapshot)

        # Step 4: Run Counterfactual forward simulation (exact same snapshot & seed + overrides)
        counterfactual_factory = CausalFactory.from_frozen_line_snapshot(
            snapshot,
            seed=request.seed,
            run_id=f"cf-{uuid4().hex[:8]}",
        )
        # Apply requested overrides to decoupled counterfactual factory
        for override in request.overrides:
            if override.machine_id in counterfactual_factory.models:
                counterfactual_factory.models[override.machine_id].apply_override(override)

        counterfactual_ticks = self._simulate_forward(
            counterfactual_factory,
            horizon_seconds=request.horizon_seconds,
            time_step_s=request.time_step_s,
        )
        counterfactual_outcome = self._calculate_line_outcome(
```

```
            counterfactual_factory, counterfactual_ticks, snapshot
        )

        # Step 5: Compute comparative causal delta
        delta = self._compute_delta(request.name, baseline_outcome, counterfactual_outcome)

        # Step 6: Generate trajectory samples for visualization
        trajectory = self._generate_trajectory(
            baseline_ticks, counterfactual_ticks, time_step_s=request.time_step_s
        )

        # Step 7: Verify Live Twin immutability
        post_fingerprint = self._compute_live_twins_fingerprint()
        if pre_fingerprint != post_fingerprint:
            raise RuntimeError("CRITICAL: Scenario Worker mutated Live Twin state! Fork immutability
    violated.")

        # Step 8: Build and persist the ScenarioRunResult
        result = ScenarioRunResult(
            scenario_id=str(uuid4()),
            name=request.name,
            description=request.description,
            status=ScenarioStatus.COMPLETED,
            snapshot_id=snapshot.snapshot_id,
            seed=request.seed,
            horizon_seconds=request.horizon_seconds,
            time_step_s=request.time_step_s,
            created_at=created_at,
            completed_at=utc_now(),
            overrides=request.overrides,
            baseline=baseline_outcome,
            counterfactual=counterfactual_outcome,
            comparison=delta,
            trajectory=trajectory,
        )

        try:
            reasoning_service = ScenarioReasoningService()
            result.decision_analysis = reasoning_service.analyze_scenario(result, snapshot=snapshot)
        except Exception:
            pass

        self._scenario_repository.save(result)
        return result

    def get_scenario(self, scenario_id: str) -> ScenarioRunResult | None:
        return self._scenario_repository.get(scenario_id)

    def list_scenarios(self, *, limit: int = 50) -> list[ScenarioRunResult]:
        return self._scenario_repository.list_all(limit=limit)

    def delete_scenario(self, scenario_id: str) -> bool:
        return self._scenario_repository.delete(scenario_id)

    # ------------------------------------------------------------------
    # Internal Simulation & Analysis Mechanics
    # ------------------------------------------------------------------

    def _resolve_snapshot(self, snapshot_id: str | None) -> FrozenLineSnapshot:
        if snapshot_id is not None:
            cached = self._snapshot_cache.get(snapshot_id)
            if cached is not None:
                return cached
        snapshot = self._process_twin_service.capture_snapshot()
```

```
        # Evict oldest snapshots when cache exceeds retention limit (Bug 5)
        if len(self._snapshot_cache) >= self.MAX_SNAPSHOT_CACHE:
            oldest_key = next(iter(self._snapshot_cache))
            del self._snapshot_cache[oldest_key]
        self._snapshot_cache[snapshot.snapshot_id] = snapshot
        return snapshot

    def _compute_live_twins_fingerprint(self) -> tuple[tuple[str, int, int, float], ...]:
        """Produce an immutable hashable fingerprint of all Live Twins in repository."""
        twins = self._live_twin_service.list_twins()
        return tuple(
            (
                t.identity.machine_id,
                t.synchronization.latest_sequence,
                t.production.cumulative_output,
                round(t.telemetry.energy_kwh, 4),
            )
            for t in sorted(twins, key=lambda x: x.identity.station_index)
        )

    def _simulate_forward(
        self,
        factory: CausalFactory,
        *,
        horizon_seconds: float,
        time_step_s: float,
    ) -> list[dict[str, Any]]:
        """Advance the ephemeral factory forward by horizon_seconds."""
        steps = int(math.ceil(horizon_seconds / time_step_s))
        tick_history: list[dict[str, Any]] = []

        for _ in range(steps):
            tick = factory.tick(dt_s=time_step_s)
            tick_dict = {
                sample.machine_id: sample for sample in tick.telemetry
            }
            tick_history.append(tick_dict)

        return tick_history

    def _calculate_line_outcome(
        self,
        factory: CausalFactory,
        ticks: list[dict[str, Any]],
        initial_snapshot: FrozenLineSnapshot,
    ) -> LineOutcome:
        initial_production = {
            twin.identity.machine_id: twin.production.cumulative_output
            for twin in initial_snapshot.twins
        }
        initial_energy = {
            twin.identity.machine_id: twin.telemetry.energy_kwh
            for twin in initial_snapshot.twins
        }

        station_outcomes: list[StationOutcome] = []
        cycle_times: dict[str, float] = {}

        for machine_id, model in factory.models.items():
            station_samples = [t[machine_id] for t in ticks if machine_id in t]
            produced = model.production_count - initial_production.get(machine_id, 0)
            energy = max(0.0, model.energy_kwh - initial_energy.get(machine_id, 0.0))

            avg_cycle = (
```

```
                sum(s.cycle_time_s for s in station_samples) / len(station_samples)
                if station_samples
                else model.config.cycle_time_s
            )
            avg_quality = (
                sum(s.quality_score for s in station_samples) / len(station_samples)
                if station_samples
                else 1.0
            )
            final_sample = station_samples[-1] if station_samples else None
            final_health = final_sample.health_score if final_sample else 100.0
            final_status = final_sample.status if final_sample else model._last_status
            fault_occurred = any(s.fault_code is not None for s in station_samples)

            cycle_times[machine_id] = avg_cycle

            station_outcomes.append(
                StationOutcome(
                    machine_id=machine_id,
                    station=model.station,
                    final_state=model.operational_state,
                    final_status=final_status,
                    final_health_score=round(final_health, 2),
                    parts_produced=max(0, produced),
                    energy_kwh=round(energy, 4),
                    avg_cycle_time_s=round(avg_cycle, 2),
                    avg_quality_score=round(avg_quality, 4),
                    fault_occurred=fault_occurred,
                )
            )

        # Line output is determined by PACKAGING-01
        packaging_produced = next(
            (so.parts_produced for so in station_outcomes if so.station == StationType.PACKAGING),
            0,
        )

        # Inspection quality determines scrap
        inspection_quality = next(
            (so.avg_quality_score for so in station_outcomes if so.station == StationType.INSPECTION),
            0.99,
        )
        scrap_count = int(math.floor(packaging_produced * max(0.0, 1.0 - inspection_quality)))
        net_output = max(0, packaging_produced - scrap_count)

        total_energy = sum(so.energy_kwh for so in station_outcomes)
        overall_quality = sum(so.avg_quality_score for so in station_outcomes) / len(station_outcomes)

        # Identify bottleneck station using status-aware effective cycle time
        # (Bug 6: aligned with ProcessTwinService.analyze_bottleneck logic)
        from domain.contracts import MachineStatus
        effective_cycle_times: dict[str, float] = {}
        for so in station_outcomes:
            raw_ct = cycle_times.get(so.machine_id, so.avg_cycle_time_s)
            # Apply operational state penalty factor matching ProcessTwinService
            status = so.final_status
            if status == MachineStatus.RUNNING:
                factor = 1.0
            elif status == MachineStatus.WARNING:
                factor = 1.15
            elif status == MachineStatus.DEGRADED:
                factor = 1.35
            elif status in (MachineStatus.FAULT, MachineStatus.OFFLINE):
                factor = 10.0
```

```
            elif status == MachineStatus.MAINTENANCE:
                factor = 5.0
            else:  # IDLE
                factor = 1.2
            effective_cycle_times[so.machine_id] = raw_ct * factor

        bottleneck_id = max(effective_cycle_times, key=effective_cycle_times.get)
        binding_cycle = effective_cycle_times[bottleneck_id]
        hourly_capacity = round(3600.0 / binding_cycle, 1) if binding_cycle > 0 else 0.0

        return LineOutcome(
            total_output=net_output,
            scrap_count=scrap_count,
            overall_quality=round(overall_quality, 4),
            total_energy_kwh=round(total_energy, 4),
            bottleneck_machine_id=bottleneck_id,
            binding_cycle_time_s=round(binding_cycle, 2),
            hourly_capacity=hourly_capacity,
            stations=sorted(station_outcomes, key=lambda s:
    factory.models[s.machine_id].config.station_index),
        )

    def _compute_delta(
        self,
        name: str,
        baseline: LineOutcome,
        counterfactual: LineOutcome,
    ) -> ScenarioDelta:
        output_delta = counterfactual.total_output - baseline.total_output
        scrap_delta = counterfactual.scrap_count - baseline.scrap_count
        quality_delta = round(counterfactual.overall_quality - baseline.overall_quality, 4)
        energy_delta = round(counterfactual.total_energy_kwh - baseline.total_energy_kwh, 4)

        base_cnc = next((s for s in baseline.stations if s.machine_id == "CNC-01"), None)
        cf_cnc = next((s for s in counterfactual.stations if s.machine_id == "CNC-01"), None)
        health_delta = (
            round(cf_cnc.final_health_score - base_cnc.final_health_score, 2)
            if base_cnc and cf_cnc
            else 0.0
        )

        bottleneck_shifted = baseline.bottleneck_machine_id != counterfactual.bottleneck_machine_id

        summary_parts = []
        if output_delta > 0:
            summary_parts.append(f"+{output_delta} units output")
        elif output_delta < 0:
            summary_parts.append(f"{output_delta} units output")

        if quality_delta != 0:
            summary_parts.append(f"{quality_delta:+.2%} quality shift")

        if health_delta > 0:
            summary_parts.append(f"+{health_delta:.1f}% CNC health preserved")
        elif health_delta < 0:
            summary_parts.append(f"{health_delta:.1f}% CNC health degraded")

        if bottleneck_shifted:
            summary_parts.append(f"bottleneck shifted from {baseline.bottleneck_machine_id} to
    {counterfactual.bottleneck_machine_id}")

        summary = f"{name}: " + (", ".join(summary_parts) if summary_parts else "Nominal trajectory
    identical to baseline.")

```

```
        return ScenarioDelta(
            output_delta=output_delta,
            scrap_delta=scrap_delta,
            quality_delta=quality_delta,
            energy_delta_kwh=energy_delta,
            health_delta=health_delta,
            bottleneck_shifted=bottleneck_shifted,
            baseline_bottleneck=baseline.bottleneck_machine_id,
            counterfactual_bottleneck=counterfactual.bottleneck_machine_id,
            summary=summary,
        )

    def _generate_trajectory(
        self,
        base_ticks: list[dict[str, Any]],
        cf_ticks: list[dict[str, Any]],
        *,
        time_step_s: float,
        target_points: int = 15,
    ) -> list[TrajectorySample]:
        total_steps = len(base_ticks)
        if total_steps == 0:
            return []

        sample_stride = max(1, total_steps // target_points)
        trajectory: list[TrajectorySample] = []

        for step in range(0, total_steps, sample_stride):
            bt = base_ticks[step]
            ct = cf_ticks[step]
            sim_time = round(step * time_step_s, 1)

            b_pkg = bt.get("PACKAGING-01")
            c_pkg = ct.get("PACKAGING-01")
            b_cnc = bt.get("CNC-01")
            c_cnc = ct.get("CNC-01")
            b_insp = bt.get("INSPECTION-01")
            c_insp = ct.get("INSPECTION-01")

            trajectory.append(
                TrajectorySample(
                    simulated_seconds=sim_time,
                    baseline_output=b_pkg.production_count if b_pkg else 0,
                    counterfactual_output=c_pkg.production_count if c_pkg else 0,
                    baseline_quality=round(b_insp.quality_score if b_insp else 0.99, 4),
                    counterfactual_quality=round(c_insp.quality_score if c_insp else 0.99, 4),
                    baseline_cnc_health=round(b_cnc.health_score if b_cnc else 100.0, 2),
                    counterfactual_cnc_health=round(c_cnc.health_score if c_cnc else 100.0, 2),
                )
            )

        return trajectory
```

Source listing: dashboard/backend/main.py

```
0001 """FastAPI dashboard gateway for the Digital Twin Orchestration Platform.
0002
0003 Phase 1 keeps the existing REST responses available while the simulator emits
0004 versioned causal-model telemetry. Persistent Live Twin APIs arrive in Phase 2.
0005 """
0006
0007 from __future__ import annotations
0008
0009 from contextlib import asynccontextmanager, suppress
0010 from fastapi import FastAPI, HTTPException, Query
0011 from fastapi.middleware.cors import CORSMiddleware
0012
0013 from models import MachineTelemetry, FactorySummary, MachineTwinState, SimulationState
0014 from simulator import SimulatorEngine
0015 from config import Settings
0016 from domain.contracts import (
0017     BottleneckReport,
0018     DefectAttributionReport,
0019     EventEnvelope,
0020     FrozenLineSnapshot,
0021     ProcessTopology,
0022     ProcessTwinView,
0023     ProductionToken,
0024     ScenarioForkRequest,
0025     ScenarioRunResult,
0026     ScenarioDecisionAnalysis,
0027     TwinDocument,
0028 )
0029 from repositories.factory import create_scenario_repository, create_twin_repository
0030 from services import (
0031     BedrockService,
0032     LiveTwinMqttConsumer,
0033     LiveTwinService,
0034     ProcessTwinService,
0035     ScenarioWorkerService,
0036     ScenarioReasoningService,
0037 )
0038
0039
0040 # ---------------------------------------------------------------------------
0041 # Simulator and Services
0042 # ---------------------------------------------------------------------------
0043 settings = Settings.from_environment()
0044 twin_repository = create_twin_repository(settings)
0045 scenario_repository = create_scenario_repository(settings)
0046 live_twin_service = LiveTwinService(twin_repository)
0047 process_twin_service = ProcessTwinService(live_twin_service, repository=twin_repository)
0048 scenario_worker_service = ScenarioWorkerService(
0049     process_twin_service=process_twin_service,
0050     live_twin_service=live_twin_service,
0051     scenario_repository=scenario_repository,
0052 )
0053 bedrock_service = None
0054 if BedrockService is not None:
0055     try:
0056         bedrock_service = BedrockService(
0057             profile_name=settings.aws_profile,
0058             region_name=settings.aws_region,
0059             fast_model_id=settings.bedrock_fast_model,
0060             reasoning_model_id=settings.bedrock_reasoning_model,
0061         )
0062     except Exception:
0063         bedrock_service = None
0064 live_twin_consumer = LiveTwinMqttConsumer(
```

```
0065     live_twin_service,
0066     host=settings.mqtt_host,
0067     port=settings.mqtt_port,
0068 )
0069
0070
0071 def _on_simulation_tick(events: tuple[EventEnvelope, ...]) -> None:
0072     # In-process bridge for development environments without a running MQTT broker
0073     if not live_twin_consumer.connected:
0074         for event in events:
0075             with suppress(Exception):
0076                 live_twin_service.consume(event)
0077     # Accumulate simulation time for rate-limited token emission through DAG
0078     with suppress(Exception):
0079         process_twin_service.accumulate_time(1.0)
0080
0081
0082 engine = SimulatorEngine(on_tick=_on_simulation_tick)
0083
0084
0085 @asynccontextmanager
0086 async def lifespan(app: FastAPI):
0087     """Start simulation and the separate Live Twin MQTT consumer."""
0088     if settings.mqtt_live_twin_enabled:
0089         live_twin_consumer.start()
0090     engine.start()
0091     yield
0092     engine.stop()
0093     live_twin_consumer.stop()
0094     scenario_repository.close()
0095     twin_repository.close()
0096
0097
0098 app = FastAPI(
0099     title="Digital Twin Orchestration Platform",
0100     version="0.4.0",
0101     lifespan=lifespan,
0102 )
0103
0104
0105 # Allow React dev server
0106 app.add_middleware(
0107     CORSMiddleware,
0108     allow_origins=["*"],
0109     allow_methods=["*"],
0110     allow_headers=["*"],
0111 )
0112
0113
0114 # ---------------------------------------------------------------------------
0115 # Service probes (used by Docker Compose and Kubernetes in Phase 6)
0116 # ---------------------------------------------------------------------------
0117
0118 @app.get("/health")
0119 def health_check():
0120     """Liveness probe: the API process can answer requests."""
0121     return {"status": "ok", "service": "dashboard-api"}
0122
0123
0124 @app.get("/ready")
0125 def readiness_check():
0126     """Readiness probe for the current in-process simulation dependency."""
0127     state = engine.get_simulation_state()
0128     return {
```

```
0129         "status": "ready" if state.running else "starting",
0130         "simulation_running": state.running,
0131         "simulation_tick": state.tick,
0132         "persistence_backend": settings.persistence_backend,
0133         "live_twin_mqtt_connected": live_twin_consumer.connected,
0134     }
0135
0136
0137 # ---------------------------------------------------------------------------
0138 # Machine Telemetry
0139 # ---------------------------------------------------------------------------
0140
0141 @app.get("/machines", response_model=list[MachineTelemetry])
0142 def get_all_machines():
0143     """Return latest telemetry for all machines."""
0144     return engine.get_all_telemetry()
0145
0146
0147 @app.get("/machines/{machine_id}", response_model=MachineTelemetry)
0148 def get_machine(machine_id: str):
0149     """Return latest telemetry for a specific machine."""
0150     data = engine.get_telemetry(machine_id)
0151     if not data:
0152         raise HTTPException(404, f"Machine {machine_id} not found")
0153     return data
0154
0155
0156 @app.get("/machines/{machine_id}/twin", response_model=MachineTwinState)
0157 def get_machine_twin(machine_id: str):
0158     """Return Digital Twin synchronization state for a machine."""
0159     data = engine.get_twin_state(machine_id)
0160     if not data:
0161         raise HTTPException(404, f"Machine {machine_id} not found")
0162     return data
0163
0164
0165 # ---------------------------------------------------------------------------
0166 # Versioned Live Twin API (introduced in Phase 2)
0167 # ---------------------------------------------------------------------------
0168
0169 @app.get("/api/v1/twins", response_model=list[TwinDocument])
0170 def list_live_twins():
0171     """List the persistent current state of each Live Twin."""
0172     return live_twin_service.list_twins()
0173
0174
0175 @app.get("/api/v1/twins/{machine_id}", response_model=TwinDocument)
0176 def get_live_twin(machine_id: str):
0177     """Return a single persistent Live Twin document."""
0178     twin = live_twin_service.get_twin(machine_id)
0179     if twin is None:
0180         raise HTTPException(404, f"Live Twin {machine_id} not found")
0181     return twin
0182
0183
0184 @app.get("/api/v1/twins/{machine_id}/telemetry", response_model=list[EventEnvelope])
0185 def get_live_twin_telemetry(
0186     machine_id: str,
0187     limit: int = Query(default=100, ge=1, le=10_000),
0188 ):
0189     """Return newest-first immutable telemetry envelopes for a Live Twin."""
0190     if live_twin_service.get_twin(machine_id) is None:
0191         raise HTTPException(404, f"Live Twin {machine_id} not found")
0192     return live_twin_service.telemetry_history(machine_id, limit=limit)
```

```
0193
0194
0195 @app.get("/api/v1/events", response_model=list[EventEnvelope])
0196 def list_live_twin_events(
0197     limit: int = Query(default=100, ge=1, le=1000),
0198     machine_id: str | None = Query(default=None),
0199 ):
0200     """Retrieve newest-first persistent plant event envelopes."""
0201     return live_twin_service.list_events(limit=limit, machine_id=machine_id)
0202
0203
0204 @app.post("/api/v1/events", response_model=TwinDocument, status_code=202)
0205 def ingest_live_twin_event(event: EventEnvelope):
0206     """Internal/test ingestion route using the exact MQTT reducer path.
0207
0208     Production station simulators publish the same envelope to MQTT. This
0209     endpoint is intentionally useful for deterministic integration tests and
0210     controlled experiments where event replay is needed.
0211     """
0212     try:
0213         return live_twin_service.consume(event).document
0214     except ValueError as error:
0215         raise HTTPException(422, str(error)) from error
0216
0217
0218 # ---------------------------------------------------------------------------
0219 # Versioned Process Twin API (introduced in Phase 3)
0220 # ---------------------------------------------------------------------------
0221
0222 @app.get("/api/v1/process/topology", response_model=ProcessTopology)
0223 def get_process_topology():
0224     """Return the DAG process topology (nodes, material flow edges, sequence)."""
0225     return process_twin_service.topology
0226
0227
0228 @app.get("/api/v1/process/snapshot", response_model=FrozenLineSnapshot)
0229 def get_frozen_line_snapshot():
0230     """Capture an atomic, immutable snapshot across all 5 station Live Twins."""
0231     try:
0232         return process_twin_service.capture_snapshot()
0233     except ValueError as error:
0234         raise HTTPException(404, str(error)) from error
0235
0236
0237 @app.get("/api/v1/process/view", response_model=ProcessTwinView)
0238 def get_process_twin_view():
0239     """Return composite Process Twin view: frozen snapshot + computed line metrics."""
0240     try:
0241         return process_twin_service.get_process_view()
0242     except ValueError as error:
0243         raise HTTPException(404, str(error)) from error
0244
0245
0246 @app.get("/api/v1/process/bottleneck", response_model=BottleneckReport)
0247 def get_process_bottleneck():
0248     """Identify the binding bottleneck station and max line throughput capacity."""
0249     try:
0250         return process_twin_service.analyze_bottleneck()
0251     except ValueError as error:
0252         raise HTTPException(404, str(error)) from error
0253
0254
0255 @app.get("/api/v1/process/tokens", response_model=list[ProductionToken])
0256 def list_production_tokens(
```

```
    limit: int = Query(default=50, ge=1, le=10_000),
    quality: str | None = Query(default=None),
    station: str | None = Query(default=None),
):
    """List newest-first production tokens with station lineage and carried quality flags."""
    return process_twin_service.list_tokens(limit=limit, quality=quality, station=station)


@app.get("/api/v1/process/tokens/{token_id}", response_model=ProductionToken)
def get_production_token(token_id: str):
    """Retrieve a specific production token by ID with full lineage history."""
    token = process_twin_service.get_token(token_id)
    if token is None:
        raise HTTPException(404, f"Production token {token_id} not found")
    return token


@app.post("/api/v1/process/tokens", response_model=ProductionToken, status_code=201)
def emit_production_token():
    """Trigger emission of a production token through the 5-station pipeline."""
    return process_twin_service.emit_token_through_pipeline()


@app.get("/api/v1/process/defects/trace", response_model=DefectAttributionReport)
def trace_process_defects(
    limit: int = Query(default=100, ge=1, le=10_000),
):
    """Trace recent inspection defects back to upstream machine degradation root causes."""
    return process_twin_service.trace_defects(limit=limit)


# ---------------------------------------------------------------------------
# Phase 4 — Scenario Worker API (§4.3, §4.4)
# ---------------------------------------------------------------------------

@app.post("/api/v1/scenarios/run", response_model=ScenarioRunResult)
def run_scenario(request: ScenarioForkRequest):
    """Run an isolated forward counterfactual simulation from a frozen snapshot."""
    try:
        return scenario_worker_service.run_scenario(request)
    except Exception as error:
        raise HTTPException(400, str(error)) from error


@app.get("/api/v1/scenarios", response_model=list[ScenarioRunResult])
def list_scenarios(limit: int = Query(default=50, ge=1, le=1000)):
    """List historical scenario runs ordered newest first."""
    return scenario_worker_service.list_scenarios(limit=limit)


@app.get("/api/v1/scenarios/presets")
def list_scenario_presets():
    """List pre-packaged research scenario presets."""
    return scenario_worker_service.get_presets()


@app.post("/api/v1/scenarios/presets/{preset_id}/run", response_model=ScenarioRunResult)
def run_scenario_preset(
    preset_id: str,
    horizon_seconds: float = Query(default=120.0, ge=5.0, le=3600.0),
    seed: int = Query(default=20260920),
):
    """Trigger execution of a pre-configured research what-if scenario preset."""
    try:
```

```
0321         return scenario_worker_service.run_preset(
0322             preset_id,
0323             horizon_seconds=horizon_seconds,
0324             seed=seed,
0325         )
0326     except KeyError as error:
0327         raise HTTPException(404, str(error)) from error
0328     except Exception as error:
0329         raise HTTPException(400, str(error)) from error
0330
0331
0332 @app.get("/api/v1/scenarios/{scenario_id}", response_model=ScenarioRunResult)
0333 def get_scenario(scenario_id: str):
0334     """Retrieve detailed execution results and delta for a specific scenario."""
0335     result = scenario_worker_service.get_scenario(scenario_id)
0336     if result is None:
0337         raise HTTPException(404, f"Scenario {scenario_id} not found")
0338     return result
0339
0340
0341 @app.delete("/api/v1/scenarios/{scenario_id}")
0342 def delete_scenario(scenario_id: str):
0343     """Delete a scenario record."""
0344     deleted = scenario_worker_service.delete_scenario(scenario_id)
0345     if not deleted:
0346         raise HTTPException(404, f"Scenario {scenario_id} not found")
0347     return {"status": "deleted", "scenario_id": scenario_id}
0348
0349 @app.post("/api/v1/scenarios/analyze", response_model=ScenarioDecisionAnalysis)
0350 def analyze_scenario_decisions(payload: dict = None):
0351     """Generate dual Manager + Worker decision analysis from live snapshot and multi-strategy
     evaluation."""
0352     target_machine_id = payload.get("target_machine_id", "CNC-01") if payload else "CNC-01"
0353     quota = payload.get("shift_quota_target", 120) if payload else 120
0354     preset_id = payload.get("preset_id", "preventive_tool_change") if payload else
     "preventive_tool_change"
0355     horizon = payload.get("horizon_seconds", 120.0) if payload else 120.0
0356
0357     snapshot = process_twin_service.capture_snapshot()
0358     run_result = scenario_worker_service.run_preset(preset_id, horizon_seconds=horizon)
0359     reasoning_service = ScenarioReasoningService()
0360     return reasoning_service.analyze_scenario(run_result, snapshot=snapshot, shift_quota_target=quota)
0361
0362
0363 @app.get("/api/v1/scenarios/strategies")
0364 def get_decision_strategies(machine_id: str = "CNC-01", quota: int = 120):
0365     """Retrieve the multi-strategy comparative tradeoff matrix for a machine station."""
0366     snapshot = process_twin_service.capture_snapshot()
0367     run_result = scenario_worker_service.run_preset("preventive_tool_change", horizon_seconds=120)
0368     reasoning_service = ScenarioReasoningService()
0369     analysis = reasoning_service.analyze_scenario(run_result, snapshot=snapshot,
     shift_quota_target=quota)
0370     return {
0371         "target_machine_id": machine_id,
0372         "manager_brief": analysis.manager_brief,
0373         "worker_order": analysis.worker_order,
0374         "strategies": analysis.strategies,
0375     }
0376
0377
0378
0379 # ------------------------------------------------------------------------------
0380 # Factory Summary
0381 # ------------------------------------------------------------------------------
```

```
0382
0383 @app.get("/summary", response_model=FactorySummary)
0384 def get_summary():
0385     """Return factory-level KPIs."""
0386     return engine.get_summary()
0387
0388
0389 # ---------------------------------------------------------------------------
0390 # Simulation Control
0391 # ---------------------------------------------------------------------------
0392
0393 @app.get("/simulate/state", response_model=SimulationState)
0394 def get_simulation_state():
0395     """Return current simulation engine state."""
0396     return engine.get_simulation_state()
0397
0398
0399 @app.post("/simulate/start")
0400 def start_simulation():
0401     engine.start()
0402     return {"status": "started"}
0403
0404
0405 @app.post("/simulate/stop")
0406 def stop_simulation():
0407     engine.stop()
0408     return {"status": "stopped"}
0409
0410
0411 @app.post("/simulate/degrade_cnc")
0412 def degrade_cnc():
0413     """Trigger CNC-01 degradation scenario."""
0414     engine.degrade_cnc()
0415     return {"status": "cnc_degradation_started"}
0416
0417
0418 @app.post("/simulate/reset_cnc")
0419 def reset_cnc():
0420     """Simulate maintenance — reset CNC-01 to healthy state."""
0421     engine.reset_cnc()
0422     return {"status": "cnc_reset"}
0423
0424
0425 @app.post("/machines/{machine_id}/inject-fault")
0426 def inject_machine_fault(machine_id: str, payload: dict = None):
0427     """Inject a specific fault scenario into any machine station."""
0428     fault_type = payload.get("fault_type", "default") if payload else "default"
0429     severity = payload.get("severity", "Warning") if payload else "Warning"
0430     success = engine.inject_fault(machine_id, fault_type, severity)
0431     if not success:
0432         raise HTTPException(404, f"Machine {machine_id} not found")
0433     return {"status": "fault_injected", "machine_id": machine_id, "fault_type": fault_type, "severity":
      severity}
0434
0435
0436 @app.post("/machines/{machine_id}/reset")
0437 def reset_machine(machine_id: str):
0438     """Simulate maintenance calibration on a specific machine."""
0439     success = engine.reset_machine(machine_id)
0440     if not success:
0441         raise HTTPException(404, f"Machine {machine_id} not found")
0442     return {"status": "machine_reset", "machine_id": machine_id}
0443
0444
```

```
@app.post("/simulate/reset")
def reset_all():
    """Reset entire simulation to initial state."""
    engine.reset_all()
    return {"status": "reset"}


# ---------------------------------------------------------------------------
# AWS Bedrock AI Diagnostics & Reasoning Endpoints
# ---------------------------------------------------------------------------
@app.post("/api/ai/ask")
def ask_ai(payload: dict):
    """Interactive AI query endpoint supporting fast (Luna) and reasoning (Astra/Claude) models."""
    if bedrock_service is None:
        raise HTTPException(503, "AWS Bedrock integration is disabled (AWS has been removed).")
    prompt = payload.get("prompt")
    if not prompt:
        raise HTTPException(400, "Missing 'prompt' in request body")
    model_id = payload.get("model_id")
    system_prompt = payload.get(
        "system_prompt",
        "You are an expert AI industrial engineer for the factory digital twin platform.",
    )
    try:
        reply = bedrock_service.generate(
            prompt=prompt,
            system_prompt=system_prompt,
            model_id=model_id,
        )
        return {
            "prompt": prompt,
            "response": reply,
            "model_id": model_id or bedrock_service.fast_model_id,
        }
    except Exception as exc:
        raise HTTPException(502, f"Bedrock invocation failed: {exc}")


@app.post("/api/ai/diagnose-bottlenecks")
def diagnose_bottlenecks():
    """Deep causal diagnosis of active factory bottlenecks using reasoning model."""
    if bedrock_service is None:
        raise HTTPException(503, "AWS Bedrock integration is disabled (AWS has been removed).")
    try:
        bottleneck_data = process_twin_service.analyze_bottleneck().model_dump()
        machines = [m.model_dump() for m in engine.get_all_telemetry()]
        analysis = bedrock_service.diagnose_bottleneck(bottleneck_data, machines)
        return {
            "model_id": bedrock_service.reasoning_model_id,
            "analysis": analysis,
            "bottlenecks": bottleneck_data,
        }
    except Exception as exc:
        raise HTTPException(502, f"AI bottleneck diagnosis failed: {exc}")


@app.post("/api/ai/scenario-insights/{scenario_id}")
def get_scenario_ai_insights(scenario_id: str):
    """Generate deep engineering insights for a finished scenario fork."""
    if bedrock_service is None:
        raise HTTPException(503, "AWS Bedrock integration is disabled (AWS has been removed).")
    scenario = scenario_repository.get_scenario(scenario_id)
    if not scenario:
        raise HTTPException(404, f"Scenario {scenario_id} not found")
```

```
    try:
        insights = bedrock_service.explain_scenario(scenario.model_dump())
        return {
            "scenario_id": scenario_id,
            "model_id": bedrock_service.reasoning_model_id,
            "insights": insights,
        }
    except Exception as exc:
        raise HTTPException(502, f"AI scenario analysis failed: {exc}")

```

Source listing: dashboard/backend/tests/test_causal_models.py

```
"""Fast, deterministic tests for Phase 1 simulation contracts."""

from __future__ import annotations

from datetime import timedelta
import unittest

from domain.contracts import (
    CommandName,
    CommandRequest,
    EventKind,
    FaultSeverity,
    MachineStatus,
    event_id_for,
)
from simulation.causal import build_default_factory


def sample_for(tick, machine_id: str):
    return next(sample for sample in tick.telemetry if sample.machine_id == machine_id)


class CausalFactoryTests(unittest.TestCase):
    def test_same_seed_replays_identically(self) -> None:
        first = build_default_factory(run_id="repeatable", seed=91)
        second = build_default_factory(run_id="repeatable", seed=91)

        first_ticks = [first.tick(dt_s=5.0) for _ in range(4)]
        second_ticks = [second.tick(dt_s=5.0) for _ in range(4)]

        self.assertEqual(
            [tick.events for tick in first_ticks],
            [tick.events for tick in second_ticks],
        )
        self.assertEqual(first.snapshot(), second.snapshot())

    def test_cnc_tool_wear_fault_has_expected_causal_direction(self) -> None:
        healthy_factory = build_default_factory(run_id="healthy", seed=700)
        degraded_factory = build_default_factory(run_id="degraded", seed=700)
        degraded_factory.apply_command(
            "CNC-01",
            CommandRequest(
                command=CommandName.INJECT_FAULT,
                fault_type="tool_wear",
                severity=FaultSeverity.CRITICAL,
            ),
        )

        healthy = sample_for(healthy_factory.tick(dt_s=60.0), "CNC-01")
        degraded = sample_for(degraded_factory.tick(dt_s=60.0), "CNC-01")

        self.assertIsNotNone(healthy.tool_wear_percent)
        self.assertIsNotNone(degraded.tool_wear_percent)
        self.assertGreater(degraded.tool_wear_percent, healthy.tool_wear_percent)
        self.assertGreater(degraded.current_a, healthy.current_a)
        self.assertGreater(degraded.vibration_mm_s, healthy.vibration_mm_s)
        self.assertGreater(degraded.temperature_c, healthy.temperature_c)
        self.assertLess(degraded.quality_score, healthy.quality_score)
        self.assertLess(degraded.health_score, healthy.health_score)
        self.assertLess(degraded.actual_rpm, healthy.actual_rpm)

    def test_inspection_quality_responds_to_current_upstream_quality(self) -> None:
        healthy_factory = build_default_factory(run_id="inspection-healthy", seed=999)
        faulted_factory = build_default_factory(run_id="inspection-faulted", seed=999)
```

```
        faulted_factory.apply_command(
            "CNC-01",
            CommandRequest(
                command=CommandName.INJECT_FAULT,
                fault_type="tool_wear",
                severity=FaultSeverity.CRITICAL,
            ),
        )

        healthy = sample_for(healthy_factory.tick(dt_s=60.0), "INSPECTION-01")
        faulted = sample_for(faulted_factory.tick(dt_s=60.0), "INSPECTION-01")

        self.assertLess(faulted.quality_score, healthy.quality_score)
        self.assertGreater(faulted.anomaly_score, healthy.anomaly_score)

    def test_tool_change_resets_cnc_tool_wear_and_fault(self) -> None:
        factory = build_default_factory(run_id="maintenance", seed=111)
        factory.apply_command(
            "CNC-01",
            CommandRequest(
                command=CommandName.INJECT_FAULT,
                fault_type="tool_wear",
                severity=FaultSeverity.CRITICAL,
            ),
        )
        before = sample_for(factory.tick(dt_s=30.0), "CNC-01")
        factory.apply_command("CNC-01", CommandRequest(command=CommandName.TOOL_CHANGE))
        after = sample_for(factory.tick(dt_s=1.0), "CNC-01")

        self.assertGreater(before.tool_wear_percent, after.tool_wear_percent)
        # One post-maintenance simulated second adds only normal baseline wear.
        self.assertLess(after.tool_wear_percent, 0.01)
        self.assertIsNone(after.fault_code)

    def test_event_has_stable_idempotency_identity_and_unobserved_receive_time(self) -> None:
        factory = build_default_factory(run_id="event-contract", seed=1)
        tick = factory.tick()
        event = next(item for item in tick.events if item.machine_id == "CNC-01")

        self.assertEqual(event.kind, EventKind.TELEMETRY)
        self.assertEqual(event.sequence, 1)
        self.assertIsNone(event.received_at)
        self.assertEqual(
            event.event_id,
            event_id_for("event-contract", "CNC-01", EventKind.TELEMETRY, 1),
        )
        self.assertEqual(event.payload["machine_id"], "CNC-01")

    def test_stop_and_start_transition_affect_status(self) -> None:
        factory = build_default_factory(run_id="states", seed=22)
        factory.apply_command("STAMPING-01", CommandRequest(command=CommandName.STOP))
        stopped = sample_for(factory.tick(dt_s=1.0), "STAMPING-01")
        factory.apply_command("STAMPING-01", CommandRequest(command=CommandName.START))
        started = sample_for(factory.tick(dt_s=1.0), "STAMPING-01")

        self.assertEqual(stopped.status, MachineStatus.IDLE)
        self.assertEqual(started.status, MachineStatus.RUNNING)

    def test_factory_rejects_non_positive_time_step(self) -> None:
        with self.assertRaises(ValueError):
            build_default_factory().tick(dt_s=0)

    def test_material_flow_conservation_along_sequential_line(self) -> None:
        factory = build_default_factory(seed=42)
```

```
        for _ in range(120):
            tick = factory.tick(dt_s=1.0)
            counts = {s.machine_id: s.production_count for s in tick.telemetry}
            st = counts["STAMPING-01"]
            cn = counts["CNC-01"]
            we = counts["WELDING-01"]
            ins = counts["INSPECTION-01"]
            pk = counts["PACKAGING-01"]
            # Strict material flow conservation invariant: downstream never exceeds upstream
            self.assertGreaterEqual(st, cn, f"CNC ({cn}) exceeded Stamping ({st})")
            self.assertGreaterEqual(cn, we, f"Welding ({we}) exceeded CNC ({cn})")
            self.assertGreaterEqual(we, ins, f"Inspection ({ins}) exceeded Welding ({we})")
            self.assertGreaterEqual(ins, pk, f"Packaging ({pk}) exceeded Inspection ({ins})")


if __name__ == "__main__":
    unittest.main()
```

Source listing: dashboard/backend/tests/test_live_twin.py

```
0001 """Tests for persistent Live Twin reduction and idempotency semantics."""
0002
0003 from __future__ import annotations
0004
0005 from datetime import timedelta
0006 import unittest
0007 from uuid import uuid4
0008
0009 from repositories.twin_repository import InMemoryTwinRepository
0010 from services.live_twin import LiveTwinService
0011 from simulation.causal import build_default_factory
0012
0013
0014 class LiveTwinServiceTests(unittest.TestCase):
0015     def setUp(self) -> None:
0016         self.repository = InMemoryTwinRepository()
0017         self.service = LiveTwinService(self.repository)
0018         self.factory = build_default_factory(run_id="live-twin-test", seed=123)
0019
0020     def cnc_event(self):
0021         tick = self.factory.tick(dt_s=1.0)
0022         return next(event for event in tick.events if event.machine_id == "CNC-01")
0023
0024     def test_consuming_telemetry_materializes_complete_twin_document(self) -> None:
0025         event = self.cnc_event()
0026         received_at = event.generated_at + timedelta(milliseconds=17)
0027
0028         result = self.service.consume(event, received_at=received_at)
0029
0030         self.assertTrue(result.event_persisted)
0031         self.assertTrue(result.state_updated)
0032         self.assertFalse(result.out_of_order)
0033         self.assertEqual(result.document.identity.machine_id, "CNC-01")
0034         self.assertEqual(result.document.identity.station_index, 2)
0035         self.assertEqual(result.document.revision, 1)
0036         self.assertEqual(result.document.synchronization.latest_sequence, 1)
0037         self.assertEqual(result.document.synchronization.delivery_latency_ms, 17.0)
0038         self.assertEqual(result.document.telemetry.machine_id, "CNC-01")
0039
0040     def test_duplicate_event_is_an_idempotent_no_op(self) -> None:
0041         event = self.cnc_event()
0042         first = self.service.consume(event, received_at=event.generated_at + timedelta(milliseconds=4))
0043         second = self.service.consume(event, received_at=event.generated_at +
     timedelta(milliseconds=9))
0044
0045         self.assertTrue(first.event_persisted)
0046         self.assertFalse(second.event_persisted)
0047         self.assertFalse(second.state_updated)
0048         self.assertEqual(second.document.revision, 1)
0049         self.assertEqual(len(self.service.telemetry_history("CNC-01")), 1)
0050
0051     def test_late_new_event_is_retained_without_rolling_back_current_state(self) -> None:
0052         first = self.cnc_event()
0053         second_event = self.cnc_event()
0054         self.service.consume(first, received_at=first.generated_at + timedelta(milliseconds=1))
0055         self.service.consume(second_event, received_at=second_event.generated_at +
     timedelta(milliseconds=1))
0056
0057         late = first.model_copy(update={"event_id": str(uuid4())})
0058         result = self.service.consume(late, received_at=second_event.generated_at +
     timedelta(milliseconds=3))
0059
0060         self.assertTrue(result.event_persisted)
0061         self.assertFalse(result.state_updated)
```

```
0062         self.assertTrue(result.out_of_order)
0063         self.assertEqual(result.document.last_event_id, second_event.event_id)
0064         # The current document stays on sequence two, even though the late
0065         # event is retained for forensic telemetry history.
0066         current = self.service.get_twin("CNC-01")
0067         assert current is not None
0068         self.assertEqual(current.synchronization.latest_sequence, 2)
0069         self.assertEqual(current.last_event_id, second_event.event_id)
0070         # Repository returns the unchanged current document, not the stale one.
0071         self.assertEqual(len(self.service.telemetry_history("CNC-01")), 3)
0072 
0073     def test_machine_id_mismatch_is_rejected(self) -> None:
0074         event = self.cnc_event()
0075         invalid = event.model_copy(update={"machine_id": "STAMPING-01"})
0076 
0077         with self.assertRaises(ValueError):
0078             self.service.consume(invalid, received_at=event.generated_at + timedelta(milliseconds=1))
0079 
0080     def test_non_telemetry_event_is_rejected(self) -> None:
0081         event = self.cnc_event()
0082         from domain.contracts import EventKind
0083         invalid = event.model_copy(update={"kind": EventKind.STATE})
0084 
0085         with self.assertRaises(ValueError):
0086             self.service.consume(invalid)
0087 
0088     def test_unaware_received_at_is_rejected(self) -> None:
0089         event = self.cnc_event()
0090         from datetime import datetime
0091         naive = datetime.now()
0092 
0093         with self.assertRaises(ValueError):
0094             self.service.consume(event, received_at=naive)
0095 
0096     def test_list_twins_is_ordered_by_station_index(self) -> None:
0097         tick = self.factory.tick(dt_s=1.0)
0098         # Consume in reverse order
0099         for event in reversed(tick.events):
0100             self.service.consume(event)
0101 
0102         twins = self.service.list_twins()
0103         self.assertEqual(len(twins), 5)
0104         station_indices = [twin.identity.station_index for twin in twins]
0105         self.assertEqual(station_indices, [1, 2, 3, 4, 5])
0106 
0107     def test_telemetry_history_bounds_validation(self) -> None:
0108         with self.assertRaises(ValueError):
0109             self.service.telemetry_history("CNC-01", limit=0)
0110         with self.assertRaises(ValueError):
0111             self.service.telemetry_history("CNC-01", limit=10001)
0112 
0113     def test_create_twin_repository_factory_memory(self) -> None:
0114         from config import Settings
0115         from repositories.factory import create_twin_repository
0116         settings = Settings(
0117             persistence_backend="memory",
0118             mongo_uri="mongodb://localhost:27017",
0119             mongo_database="test",
0120             mqtt_host="localhost",
0121             mqtt_port=1883,
0122             mqtt_live_twin_enabled=True,
0123         )
0124         repo = create_twin_repository(settings)
0125         self.assertIsInstance(repo, InMemoryTwinRepository)
```

```
0126
0127     def test_mqtt_consumer_on_message(self) -> None:
0128         from unittest.mock import MagicMock
0129         from services.mqtt_live_twin import LiveTwinMqttConsumer
0130         consumer = LiveTwinMqttConsumer(self.service, host="localhost", port=1883)
0131         event = self.cnc_event()
0132         mock_msg = MagicMock()
0133         mock_msg.payload = event.model_dump_json().encode("utf-8")
0134         mock_msg.topic = "factory/v1/CNC-01/telemetry"
0135
0136         # Directly invoke the logic via consumer's handler or start mock
0137         # Test that consume processes valid message without exception
0138         self.service.consume(event)
0139         twin = self.service.get_twin("CNC-01")
0140         self.assertIsNotNone(twin)
0141         assert twin is not None
0142         self.assertEqual(twin.identity.machine_id, "CNC-01")
0143
0144
0145 if __name__ == "__main__":
0146     unittest.main()
0147
```

Source listing: dashboard/backend/tests/test_process_twin.py

```
0001 """Tests for Phase 3 Process Twin: DAG topology, frozen snapshot, tokens, bottleneck, defects."""
0002
0003 from __future__ import annotations
0004
0005 from datetime import timedelta
0006 import unittest
0007
0008 from domain.contracts import (
0009     CommandName,
0010     CommandRequest,
0011     FaultSeverity,
0012     MachineStatus,
0013     StationType,
0014 )
0015 from repositories.twin_repository import InMemoryTwinRepository
0016 from services.live_twin import LiveTwinService
0017 from services.process_twin import ProcessTwinService
0018 from simulation.causal import build_default_factory
0019
0020
0021 class ProcessTwinServiceTests(unittest.TestCase):
0022     def setUp(self) -> None:
0023         self.repository = InMemoryTwinRepository()
0024         self.live_twin_service = LiveTwinService(self.repository)
0025         self.process_twin_service = ProcessTwinService(self.live_twin_service)
0026         self.factory = build_default_factory(run_id="process-twin-test", seed=42)
0027
0028     def _populate_live_twins(self, ticks: int = 1) -> None:
0029         """Advance the factory and feed all station events into LiveTwinService."""
0030         for _ in range(ticks):
0031             tick = self.factory.tick(dt_s=1.0)
0032             for event in tick.events:
0033                 self.live_twin_service.consume(event)
0034
0035     def test_dag_topology_structure(self) -> None:
0036         topo = self.process_twin_service.topology
0037         self.assertEqual(topo.version, "five-station-v1")
0038         self.assertEqual(len(topo.nodes), 5)
0039         self.assertEqual(len(topo.edges), 4)
0040         self.assertEqual(topo.order, [
0041             "STAMPING-01", "CNC-01", "WELDING-01", "INSPECTION-01", "PACKAGING-01"
0042         ])
0043
0044         # Test node connections
0045         stamping = next(n for n in topo.nodes if n.machine_id == "STAMPING-01")
0046         cnc = next(n for n in topo.nodes if n.machine_id == "CNC-01")
0047         packaging = next(n for n in topo.nodes if n.machine_id == "PACKAGING-01")
0048
0049         self.assertEqual(stamping.upstream_machine_ids, [])
0050         self.assertEqual(stamping.downstream_machine_ids, ["CNC-01"])
0051         self.assertEqual(cnc.upstream_machine_ids, ["STAMPING-01"])
0052         self.assertEqual(cnc.downstream_machine_ids, ["WELDING-01"])
0053         self.assertEqual(packaging.downstream_machine_ids, [])
0054
0055     def test_capture_snapshot_requires_live_twins(self) -> None:
0056         # Before populating twins, snapshot must raise ValueError
0057         with self.assertRaises(ValueError):
0058             self.process_twin_service.capture_snapshot()
0059
0060     def test_capture_snapshot_frozen_semantics(self) -> None:
0061         self._populate_live_twins(1)
0062         snapshot = self.process_twin_service.capture_snapshot()
0063
0064         self.assertEqual(len(snapshot.twins), 5)
```

```
0065         self.assertEqual(snapshot.topology_version, "five-station-v1")
0066         self.assertIsNotNone(snapshot.snapshot_id)
0067         self.assertIsNotNone(snapshot.captured_at.tzinfo)
0068 
0069         # Ordering check
0070         indices = [t.identity.station_index for t in snapshot.twins]
0071         self.assertEqual(indices, [1, 2, 3, 4, 5])
0072 
0073     def test_token_pipeline_progression(self) -> None:
0074         self._populate_live_twins(1)
0075         token = self.process_twin_service.emit_token_through_pipeline()
0076 
0077         self.assertTrue(token.token_id.startswith("tok-"))
0078         self.assertEqual(token.source_station, StationType.STAMPING)
0079         self.assertEqual(token.quality, "pass")
0080         self.assertIn("STAMPING-01", token.lineage)
0081         self.assertIn("CNC-01", token.lineage)
0082         self.assertIn("WELDING-01", token.lineage)
0083         self.assertIn("INSPECTION-01", token.lineage)
0084         self.assertIn("PACKAGING-01", token.lineage)
0085 
0086         flags = token.carried_flags
0087         self.assertIn("stamping_quality", flags)
0088         self.assertIn("machining_quality", flags)
0089         self.assertIn("weld_quality", flags)
0090         self.assertIn("defect_probability", flags)
0091         self.assertEqual(flags["inspection_verdict"], "ACCEPTED")
0092         self.assertEqual(flags["disposition"], "DISPATCHED")
0093 
0094         # Retrieval check
0095         retrieved = self.process_twin_service.get_token(token.token_id)
0096         self.assertIsNotNone(retrieved)
0097         assert retrieved is not None
0098         self.assertEqual(retrieved.token_id, token.token_id)
0099 
0100     def test_token_filtering_and_bounds(self) -> None:
0101         self._populate_live_twins(1)
0102         for _ in range(5):
0103             self.process_twin_service.emit_token_through_pipeline()
0104 
0105         tokens = self.process_twin_service.list_tokens(limit=10)
0106         self.assertEqual(len(tokens), 5)
0107 
0108         pass_tokens = self.process_twin_service.list_tokens(limit=10, quality="pass")
0109         self.assertEqual(len(pass_tokens), 5)
0110 
0111         fail_tokens = self.process_twin_service.list_tokens(limit=10, quality="fail")
0112         self.assertEqual(len(fail_tokens), 0)
0113 
0114         # Bounds validation
0115         with self.assertRaises(ValueError):
0116             self.process_twin_service.list_tokens(limit=0)
0117         with self.assertRaises(ValueError):
0118             self.process_twin_service.list_tokens(limit=10001)
0119 
0120     def test_bottleneck_analysis_nominal_line(self) -> None:
0121         self._populate_live_twins(1)
0122         report = self.process_twin_service.analyze_bottleneck()
0123 
0124         # CNC nominal cycle time is 42s, which is the largest on the line
0125         self.assertEqual(report.binding_machine_id, "CNC-01")
0126         self.assertEqual(report.binding_station, StationType.CNC)
0127         self.assertAlmostEqual(report.binding_cycle_time_s, 42.0, delta=1.0)
0128         self.assertAlmostEqual(report.max_line_capacity_per_hour, round(3600.0 / 42.0, 2), delta=2.0)
```

```
0129
0130        cnc_detail = next(d for d in report.stations if d.machine_id == "CNC-01")
0131        self.assertTrue(cnc_detail.is_bottleneck)
0132        self.assertEqual(cnc_detail.slack_time_s, 0.0)
0133
0134        stamping_detail = next(d for d in report.stations if d.machine_id == "STAMPING-01")
0135        self.assertFalse(stamping_detail.is_bottleneck)
0136        self.assertGreater(stamping_detail.slack_time_s, 20.0)
0137
0138    def test_bottleneck_dynamic_shift_on_stamping_fault(self) -> None:
0139        self._populate_live_twins(1)
0140        # Inject fault into STAMPING-01 and advance past fault development threshold (60s)
0141        self.factory.apply_command(
0142            "STAMPING-01",
0143            CommandRequest(
0144                command=CommandName.INJECT_FAULT,
0145                fault_type="feed_jam",
0146                severity=FaultSeverity.CRITICAL,
0147            ),
0148        )
0149        tick = self.factory.tick(dt_s=60.0)
0150        for event in tick.events:
0151            self.live_twin_service.consume(event)
0152
0153        report = self.process_twin_service.analyze_bottleneck()
0154        # With FAULT operational status, effective cycle time multiplier is 10.0x (8.5s * 10 = 85s >
     42s)
0155        self.assertEqual(report.binding_machine_id, "STAMPING-01")
0156        self.assertEqual(report.binding_station, StationType.STAMPING)
0157        stamping_detail = next(d for d in report.stations if d.machine_id == "STAMPING-01")
0158        self.assertTrue(stamping_detail.is_bottleneck)
0159
0160    def test_defect_tracing_under_cnc_degradation(self) -> None:
0161        self._populate_live_twins(1)
0162        # Healthy baseline check
0163        healthy_report = self.process_twin_service.trace_defects()
0164        self.assertEqual(healthy_report.defective_tokens_count, 0)
0165        self.assertIsNone(healthy_report.primary_root_cause_station)
0166
0167        # Severely degrade CNC-01
0168        self.factory.apply_command(
0169            "CNC-01",
0170            CommandRequest(
0171                command=CommandName.INJECT_FAULT,
0172                fault_type="tool_wear",
0173                severity=FaultSeverity.CRITICAL,
0174            ),
0175        )
0176        # Tick factory so CNC tool wear accumulates and quality plummets
0177        for _ in range(50):
0178            self.factory.tick(dt_s=10.0)
0179        self._populate_live_twins(1)
0180
0181        # Emit tokens under degraded conditions
0182        for _ in range(10):
0183            self.process_twin_service.emit_token_through_pipeline()
0184
0185        defect_report = self.process_twin_service.trace_defects(limit=20)
0186        self.assertGreater(defect_report.defective_tokens_count, 0)
0187        self.assertEqual(defect_report.primary_root_cause_station, StationType.CNC)
0188        self.assertEqual(defect_report.primary_root_cause_machine_id, "CNC-01")
0189
0190        cnc_attr = next(a for a in defect_report.attributions if a.machine_id == "CNC-01")
0191        self.assertGreater(cnc_attr.attribution_percent, 50.0)
```

```
0192
0193     def test_process_view_composition(self) -> None:
0194         self._populate_live_twins(1)
0195         view = self.process_twin_service.get_process_view()
0196
0197         self.assertEqual(len(view.snapshot.twins), 5)
0198         self.assertEqual(view.metrics.bottleneck_machine_id, "CNC-01")
0199         self.assertEqual(view.metrics.total_stations, 5)
0200         self.assertEqual(view.metrics.available_stations, 5)
0201         self.assertGreater(view.metrics.line_capacity_per_hour, 0.0)
0202         self.assertGreater(len(view.metrics.notes), 0)
0203
0204
0205 if __name__ == "__main__":
0206     unittest.main()
```

Source listing: dashboard/backend/tests/test_scenario_worker.py

```
"""Unit tests for Phase 4 Scenario Worker (§4.3, §4.4).

Tests:
- Strict Fork Immutability: verifying running scenarios never mutates Live Twins.
- Deterministic Replay: identical snapshot + seed produces identical output.
- Counterfactual Intervention Sensitivity: tool change restores health, coolant failure triggers thermal defect.
- Line Cadence Adjustment: speedup increases throughput and energy.
- Scenario Repository persistence and retrieval.
- Research Preset retrieval and execution.
"""

from __future__ import annotations

from datetime import datetime, timezone
import unittest
from uuid import uuid4

from domain.contracts import (
    CommandName,
    EventEnvelope,
    EventKind,
    FaultSeverity,
    FrozenLineSnapshot,
    ScenarioForkRequest,
    StationOverride,
    StationType,
    TwinDocument,
    event_id_for,
    utc_now,
)
from repositories.scenario_repository import InMemoryScenarioRepository
from repositories.twin_repository import InMemoryTwinRepository
from services.live_twin import LiveTwinService
from services.process_twin import ProcessTwinService
from services.scenario_worker import ScenarioWorkerService
from simulation.causal import build_default_factory


def _populate_test_twins(live_twin_service: LiveTwinService, factory_ticks: int = 5) -> None:
    factory = build_default_factory(seed=12345)
    for _ in range(factory_ticks):
        tick = factory.tick(dt_s=1.0)
        for event in tick.events:
            live_twin_service.consume(event)


class ScenarioWorkerServiceTests(unittest.TestCase):
    def setUp(self) -> None:
        self.twin_repo = InMemoryTwinRepository()
        self.live_twin_service = LiveTwinService(self.twin_repo)
        self.process_twin_service = ProcessTwinService(self.live_twin_service)
        self.scenario_repo = InMemoryScenarioRepository()
        self.service = ScenarioWorkerService(
            process_twin_service=self.process_twin_service,
            live_twin_service=self.live_twin_service,
            scenario_repository=self.scenario_repo,
        )
        _populate_test_twins(self.live_twin_service, factory_ticks=5)

    def test_scenario_fork_immutability(self) -> None:
        """Scenario execution MUST NEVER mutate Live Twin state or repository events."""
        before_twins = {t.identity.machine_id: t.model_dump(mode="json") for t in self.live_twin_service.list_twins()}
```

```
0063         self.assertEqual(len(before_twins), 5)
0064
0065         request = ScenarioForkRequest(
0066             name="Immutability Proof",
0067             horizon_seconds=60.0,
0068             seed=20260920,
0069             overrides=[
0070                 StationOverride(
0071                     machine_id="CNC-01",
0072                     command=CommandName.INJECT_FAULT,
0073                     fault_type="severe_crash",
0074                     fault_severity=FaultSeverity.CRITICAL,
0075                     load_factor=0.99,
0076                 ),
0077                 StationOverride(
0078                     machine_id="STAMPING-01",
0079                     cycle_time_multiplier=0.5,
0080                 ),
0081             ],
0082         )
0083
0084         result = self.service.run_scenario(request)
0085         self.assertIsNotNone(result.scenario_id)
0086
0087         # Verify Live Twins are completely unchanged
0088         after_twins = {t.identity.machine_id: t.model_dump(mode="json") for t in
     self.live_twin_service.list_twins()}
0089         self.assertEqual(before_twins, after_twins)
0090
0091     def test_deterministic_replay(self) -> None:
0092         """Two forward runs with identical snapshot and seed produce bit-for-bit identical results."""
0093         request = ScenarioForkRequest(
0094             name="Determinism Check",
0095             horizon_seconds=45.0,
0096             seed=99991,
0097             overrides=[
0098                 StationOverride(machine_id="CNC-01", load_factor=0.75),
0099             ],
0100         )
0101
0102         run1 = self.service.run_scenario(request)
0103         run2 = self.service.run_scenario(request)
0104
0105         self.assertEqual(run1.baseline.total_output, run2.baseline.total_output)
0106         self.assertEqual(run1.counterfactual.total_output, run2.counterfactual.total_output)
0107         self.assertEqual(run1.comparison.output_delta, run2.comparison.output_delta)
0108         self.assertEqual(run1.comparison.quality_delta, run2.comparison.quality_delta)
0109         self.assertEqual(run1.comparison.energy_delta_kwh, run2.comparison.energy_delta_kwh)
0110         self.assertEqual(len(run1.trajectory), len(run2.trajectory))
0111
0112     def test_counterfactual_tool_change_preserves_health(self) -> None:
0113         """Immediate tool change at CNC-01 restores tool wear and preserves health vs degradation."""
0114         result = self.service.run_preset("preventive_tool_change", horizon_seconds=90.0, seed=777)
0115
0116         self.assertIn("CNC-01", [s.machine_id for s in result.counterfactual.stations])
0117         base_cnc = next(s for s in result.baseline.stations if s.machine_id == "CNC-01")
0118         cf_cnc = next(s for s in result.counterfactual.stations if s.machine_id == "CNC-01")
0119
0120         # Counterfactual tool change should yield higher CNC health than letting it run
0121         self.assertGreaterEqual(cf_cnc.final_health_score, base_cnc.final_health_score)
0122         self.assertGreaterEqual(result.comparison.health_delta, 0.0)
0123
0124     def test_counterfactual_coolant_failure_triggers_degradation(self) -> None:
0125         """Coolant failure should cause thermal rise, lower health, and negative delta."""
```

```
        result = self.service.run_preset("coolant_pump_failure", horizon_seconds=80.0, seed=42)

        base_cnc = next(s for s in result.baseline.stations if s.machine_id == "CNC-01")
        cf_cnc = next(s for s in result.counterfactual.stations if s.machine_id == "CNC-01")

        # Health should drop under coolant failure
        self.assertLess(cf_cnc.final_health_score, base_cnc.final_health_score)
        self.assertTrue(cf_cnc.fault_occurred)
        self.assertLess(result.comparison.health_delta, 0.0)

    def test_line_cadence_boost_increases_energy(self) -> None:
        """Speeding up machines (lower cycle time) increases energy consumption."""
        result = self.service.run_preset("line_cadence_boost", horizon_seconds=60.0, seed=101)

        self.assertGreater(result.counterfactual.total_energy_kwh, 0.0)
        # Accelerated pace consumes more electrical energy
        self.assertGreaterEqual(result.counterfactual.total_energy_kwh,
    result.baseline.total_energy_kwh)

    def test_scenario_presets_metadata(self) -> None:
        """Presets list must return all predefined research scenarios."""
        presets = self.service.get_presets()
        self.assertGreaterEqual(len(presets), 4)
        preset_ids = [p["preset_id"] for p in presets]
        self.assertIn("preventive_tool_change", preset_ids)
        self.assertIn("coolant_pump_failure", preset_ids)
        self.assertIn("line_cadence_boost", preset_ids)
        self.assertIn("welding_electrode_drift", preset_ids)

    def test_scenario_repository_crud(self) -> None:
        """Repository must support saving, retrieving, listing, and deleting scenarios."""
        result = self.service.run_preset("preventive_tool_change", horizon_seconds=30.0)

        retrieved = self.scenario_repo.get(result.scenario_id)
        self.assertIsNotNone(retrieved)
        self.assertEqual(retrieved.name, result.name)

        all_scenarios = self.scenario_repo.list_all(limit=10)
        self.assertGreaterEqual(len(all_scenarios), 1)
        self.assertEqual(all_scenarios[0].scenario_id, result.scenario_id)

        deleted = self.scenario_repo.delete(result.scenario_id)
        self.assertTrue(deleted)
        self.assertIsNone(self.scenario_repo.get(result.scenario_id))


if __name__ == "__main__":
    unittest.main()
```

Source listing: dashboard/backend/tests/test_api.py

```
0001 """HTTP-level checks for the dashboard compatibility API."""
0002
0003 from __future__ import annotations
0004
0005 import unittest
0006
0007 from fastapi.testclient import TestClient
0008
0009 from main import app
0010 from simulation.causal import build_default_factory
0011
0012
0013 class ApiTests(unittest.TestCase):
0014     def setUp(self) -> None:
0015         self.client = TestClient(app)
0016         self.client.__enter__()
0017
0018     def tearDown(self) -> None:
0019         self.client.__exit__(None, None, None)
0020
0021     def test_health_and_readiness(self) -> None:
0022         health = self.client.get("/health")
0023         readiness = self.client.get("/ready")
0024
0025         self.assertEqual(health.status_code, 200)
0026         self.assertEqual(health.json()["status"], "ok")
0027         self.assertEqual(readiness.status_code, 200)
0028         self.assertTrue(readiness.json()["simulation_running"])
0029
0030     def test_machine_summary_and_fault_control_flow(self) -> None:
0031         machines = self.client.get("/machines")
0032         summary = self.client.get("/summary")
0033         fault = self.client.post(
0034             "/machines/CNC-01/inject-fault",
0035             json={"fault_type": "tool_wear", "severity": "Critical"},
0036         )
0037         state = self.client.get("/simulate/state")
0038
0039         self.assertEqual(machines.status_code, 200)
0040         self.assertEqual(len(machines.json()), 5)
0041         self.assertEqual(summary.status_code, 200)
0042         self.assertEqual(fault.status_code, 200)
0043         self.assertTrue(state.json()["cnc_degrading"])
0044
0045     def test_unknown_machine_returns_not_found(self) -> None:
0046         response = self.client.get("/machines/NOT-A-MACHINE")
0047         self.assertEqual(response.status_code, 404)
0048
0049     def test_versioned_event_ingestion_materializes_live_twin(self) -> None:
0050         factory = build_default_factory(run_id="api-live-twin", seed=500)
0051         tick = factory.tick()
0052         event = next(item for item in tick.events if item.machine_id == "CNC-01")
0053
0054         ingestion = self.client.post("/api/v1/events", json=event.model_dump(mode="json"))
0055         twin = self.client.get("/api/v1/twins/CNC-01")
0056         history = self.client.get("/api/v1/twins/CNC-01/telemetry?limit=10")
0057
0058         self.assertEqual(ingestion.status_code, 202)
0059         self.assertEqual(ingestion.json()["identity"]["machine_id"], "CNC-01")
0060         self.assertEqual(twin.status_code, 200)
0061         self.assertGreaterEqual(len(history.json()), 1)
0062
0063         events_res = self.client.get("/api/v1/events?limit=10")
0064         self.assertEqual(events_res.status_code, 200)
```

```
0065         self.assertGreaterEqual(len(events_res.json()), 1)
0066
0067     def test_live_twin_list_and_not_found_handling(self) -> None:
0068         twins_response = self.client.get("/api/v1/twins")
0069         self.assertEqual(twins_response.status_code, 200)
0070         self.assertIsInstance(twins_response.json(), list)
0071
0072         not_found_twin = self.client.get("/api/v1/twins/NON-EXISTENT")
0073         self.assertEqual(not_found_twin.status_code, 404)
0074
0075         not_found_telemetry = self.client.get("/api/v1/twins/NON-EXISTENT/telemetry")
0076         self.assertEqual(not_found_telemetry.status_code, 404)
0077
0078     def test_invalid_event_ingestion_returns_422(self) -> None:
0079         factory = build_default_factory(run_id="api-invalid-test", seed=501)
0080         tick = factory.tick()
0081         event = next(item for item in tick.events if item.machine_id == "CNC-01")
0082         # Invalidate machine_id
0083         invalid_event = event.model_copy(update={"machine_id": "STAMPING-01"})
0084         response = self.client.post("/api/v1/events", json=invalid_event.model_dump(mode="json"))
0085         self.assertEqual(response.status_code, 422)
0086
0087     def test_process_twin_api_endpoints(self) -> None:
0088         # 1. Topology
0089         topology_res = self.client.get("/api/v1/process/topology")
0090         self.assertEqual(topology_res.status_code, 200)
0091         self.assertEqual(len(topology_res.json()["nodes"]), 5)
0092         self.assertEqual(len(topology_res.json()["edges"]), 4)
0093
0094         # Ingest events for all 5 stations so Live Twin repository is fully populated
0095         factory = build_default_factory(run_id="api-process-test", seed=600)
0096         tick = factory.tick()
0097         for ev in tick.events:
0098             self.client.post("/api/v1/events", json=ev.model_dump(mode="json"))
0099
0100         # 2. Frozen snapshot
0101         snapshot_res = self.client.get("/api/v1/process/snapshot")
0102         self.assertEqual(snapshot_res.status_code, 200)
0103         self.assertEqual(len(snapshot_res.json()["twins"]), 5)
0104
0105         # 3. Process view
0106         view_res = self.client.get("/api/v1/process/view")
0107         self.assertEqual(view_res.status_code, 200)
0108         self.assertIn("metrics", view_res.json())
0109         self.assertIn("snapshot", view_res.json())
0110
0111         # 4. Bottleneck analysis
0112         bottleneck_res = self.client.get("/api/v1/process/bottleneck")
0113         self.assertEqual(bottleneck_res.status_code, 200)
0114         self.assertEqual(bottleneck_res.json()["binding_machine_id"], "CNC-01")
0115
0116         # 5. Token emission and retrieval
0117         token_res = self.client.post("/api/v1/process/tokens")
0118         self.assertEqual(token_res.status_code, 201)
0119         token_id = token_res.json()["token_id"]
0120
0121         tokens_list = self.client.get("/api/v1/process/tokens?limit=10")
0122         self.assertEqual(tokens_list.status_code, 200)
0123         self.assertGreaterEqual(len(tokens_list.json()), 1)
0124
0125         single_token = self.client.get(f"/api/v1/process/tokens/{token_id}")
0126         self.assertEqual(single_token.status_code, 200)
0127         self.assertEqual(single_token.json()["token_id"], token_id)
0128
```

```
        not_found_token = self.client.get("/api/v1/process/tokens/non-existent-token")
        self.assertEqual(not_found_token.status_code, 404)

        # 6. Defect tracing
        defects_res = self.client.get("/api/v1/process/defects/trace?limit=50")
        self.assertEqual(defects_res.status_code, 200)
        self.assertIn("attributions", defects_res.json())

    def test_scenario_worker_api_endpoints(self) -> None:
        """Verify Phase 4 Scenario Worker REST endpoints (§4.3, §4.4)."""
        # 1. Preset listing
        presets_res = self.client.get("/api/v1/scenarios/presets")
        self.assertEqual(presets_res.status_code, 200)
        presets = presets_res.json()
        self.assertGreaterEqual(len(presets), 4)

        # 2. Run preset scenario
        run_preset_res =
    self.client.post("/api/v1/scenarios/presets/preventive_tool_change/run?horizon_seconds=30")
        self.assertEqual(run_preset_res.status_code, 200)
        scenario_data = run_preset_res.json()
        scenario_id = scenario_data["scenario_id"]
        self.assertIn("baseline", scenario_data)
        self.assertIn("counterfactual", scenario_data)
        self.assertIn("comparison", scenario_data)

        # 3. List scenarios
        list_res = self.client.get("/api/v1/scenarios?limit=10")
        self.assertEqual(list_res.status_code, 200)
        scenarios_list = list_res.json()
        self.assertGreaterEqual(len(scenarios_list), 1)

        # 4. Get specific scenario
        get_res = self.client.get(f"/api/v1/scenarios/{scenario_id}")
        self.assertEqual(get_res.status_code, 200)
        self.assertEqual(get_res.json()["scenario_id"], scenario_id)

        # 5. Run custom scenario fork request
        custom_payload = {
            "name": "Custom Speed Test",
            "horizon_seconds": 25.0,
            "seed": 12345,
            "overrides": [
                {"machine_id": "CNC-01", "cycle_time_multiplier": 0.85},
            ],
        }
        custom_res = self.client.post("/api/v1/scenarios/run", json=custom_payload)
        self.assertEqual(custom_res.status_code, 200)
        custom_id = custom_res.json()["scenario_id"]

        # 6. Delete scenario
        del_res = self.client.delete(f"/api/v1/scenarios/{custom_id}")
        self.assertEqual(del_res.status_code, 200)

        not_found_res = self.client.get(f"/api/v1/scenarios/{custom_id}")
        self.assertEqual(not_found_res.status_code, 404)


if __name__ == "__main__":
    unittest.main()
```